System Design Interviews Apple & Google

> An expanded director-level practice guide with 28 design prompts, first-person model answers, a full-page architecture diagram, dedicated trade-off breakdowns, and end-to-end request flows plus API contracts for every prompt — with a repeatable four-week practice system grounded in your real platform, AI, security, and modernization experience.

28	28	28	140	4
DESIGN PROMPTS	MODEL ANSWERS	ARCHITECTURE DIAGRAMS	TRADE-OFF DECISIONS	WEEKS OF DRILLS

Contents

How Google runs system design interviews 2
How Apple's interviews differ 3
The five-step answer framework 4
Google-style questions: plan, diagram, trade-offs, flows, answers 5–64
Apple-style questions: plan, diagram, trade-offs, flows, answers 65–104
Eight additional questions + answers 105–144
How to practice 145–152
Director-level twist + sources 153

Use the model answers to learn structure and language, not to memorize a fixed diagram. Apple and Google do not publish fixed question banks; actual prompts vary by team, level, and domain.

How Google runs system design interviews in 2026

Google's current pattern is a collaborative, approximately 45-minute deep dive into one ambiguous system. The interviewer evaluates how you frame the problem and adapt—not whether your diagram resembles a memorized reference architecture.

0–5 min	Clarify users, scope, scale, service-level objectives, and exclusions.
5–18 min	Propose the high-level architecture and narrate the critical read/write paths.
18–38 min	Deep-dive into one or two hard areas: sharding, consistency, ranking, reliability, or model serving.
38–45 min	Absorb a new constraint, surface trade-offs, identify bottlenecks, and summarize.

Five dimensions that separate strong answers

1 REQUIREMENTS
Narrows ambiguity into measurable functional and non-functional goals.

2 DECOMPOSITION
Creates clean components, interfaces, ownership boundaries, and data flows.

3 TECHNICAL DEPTH
Can descend into storage, consistency, algorithms, failure modes, and operations.

4 TRADE-OFFS
Compares viable options against latency, availability, cost, and complexity.

5 COMMUNICATION
Drives the conversation, checks alignment, and explains decisions crisply.

2025–2026 shift. Candidate-oriented reports describe a return to at least one in-person round and increased use of AI/ML design scenarios. For L6+ candidates, expect more than one design conversation and a higher bar for operational realism, not just component knowledge.

Director signal

After the architecture is credible, widen the lens: team boundaries, migration sequence, risks, on-call model, data governance, cost envelope, and how you would make the decision reversible.

How Apple's system design interviews differ

Apple-style design is often grounded in the experience of a real device and its user. A generic "client → load balancer → service → database" diagram is insufficient if it ignores what happens offline, under resource pressure, or across several devices.

Start with the device

- What must work with no network?
- What is persisted locally, and for how long?
- What happens in Low Power Mode?
- How does the design behave when storage is nearly full?

Then cross the boundary

- How are changes ordered and deduplicated?
- How are concurrent edits reconciled?
- What data can remain on-device?
- What metadata is visible to the service?

Conflict resolution is a product decision



Version vectors can detect causality: one version supersedes another when its counters dominate; incomparable vectors indicate concurrent updates. The architecture still needs a product policy—field-level merge, last-writer-wins, duplicate preservation, or explicit user resolution.

> **Apple's signature angles:** offline-first behavior, multi-device sync, storage and power constraints, unreliable networks, privacy, on-device processing, and graceful degradation. Explain the user consequence of every technical choice.

How to use your own background honestly

You have deep distributed-platform and security experience, but the profile does not claim consumer mobile-sync ownership. Make the parallel explicit: "I have handled consistency and failure boundaries in event-driven platforms; I would transfer those principles here, then partner with device specialists on OS-specific behavior."

A framework for answering any design question

Clarify 01

Define the primary user journey, scale, availability, latency, consistency, privacy, abuse, and what is out of scope. State assumptions and ask for confirmation.

Estimate 02

Calculate orders of magnitude: requests per second, peak multiplier, payload size, storage growth, bandwidth, cache working set, and cost-sensitive resources.

Design 03

Draw the minimum viable end-to-end path. Name APIs, state stores, asynchronous boundaries, partition keys, and where correctness is enforced.

Deep-dive 04

Choose the two hardest risks. Compare alternatives; discuss failure handling, observability, security, data lifecycle, and rollout.

Stress & close 05

Test hot keys, regional failure, retries, backlog, model drift, abuse, and 10× growth. Summarize the decision and remaining risks.

Director-level overlay

ORGANIZATION

Which teams own which boundaries? What interfaces allow parallel delivery? Where is a platform team justified?

OPERABILITY

Who is on call? What are the service-level objectives, error budgets, runbooks, and safe rollback paths?

ECONOMICS

What dominates cost? What should be built, bought, or managed? Which optimization can fund the next capability?

EVOLUTION

What ships in 90 days, what arrives in year one, and what architecture survives the three-year horizon?

> **A strong transition line:** “The architecture is viable. Before deep-diving, I want to identify the highest-risk assumption, the team boundary that contains it, and the metric that tells us whether the investment is working.”

Question 1 of 28

G1. Design YouTube

WHY THEY ASK IT

Tests globally distributed storage, asynchronous media processing, content delivery, hot-object handling, and the ability to separate the upload path from the playback path. Senior candidates must make reliability and cost explicit.

CLARIFYING QUESTIONS TO ASK

- Video on demand only, or live streaming too?
- Target regions, active users, and peak plays per second?
- Required startup latency and rebuffering target?
- Supported resolutions, codecs, and creator controls?
- How quickly must a new upload become playable?

SCALE ESTIMATION

Assume 100M daily viewers, 1B plays/day (~12K average; 10× peak), 5M uploads/day, and 300 MB average source size. That is ~1.5 PB/day before renditions; CDN egress dominates cost.

HIGH-LEVEL DESIGN

```
Creator → Upload API → Object store → Transcode queue → Renditions
Viewer → Metadata API → CDN/edge → origin shield → Object store
```

Use resumable multipart uploads, immutable object storage, a durable workflow for validation/transcoding, metadata in a sharded store, and signed manifests. Route playback through CDN edges with an origin shield; publish only after required renditions pass checks.

DEEP DIVES

- Hot videos: multi-tier caching and request coalescing reduce origin load; pre-warming improves latency but spends bandwidth speculatively.
- Processing reliability: idempotent jobs, content-addressed outputs, dead-letter queues, and per-rendition status prevent duplicate work.
- Global availability: replicate metadata and media by demand; choose regional consistency separately for counters, comments, and takedowns.

LIKELY FOLLOW-UPS

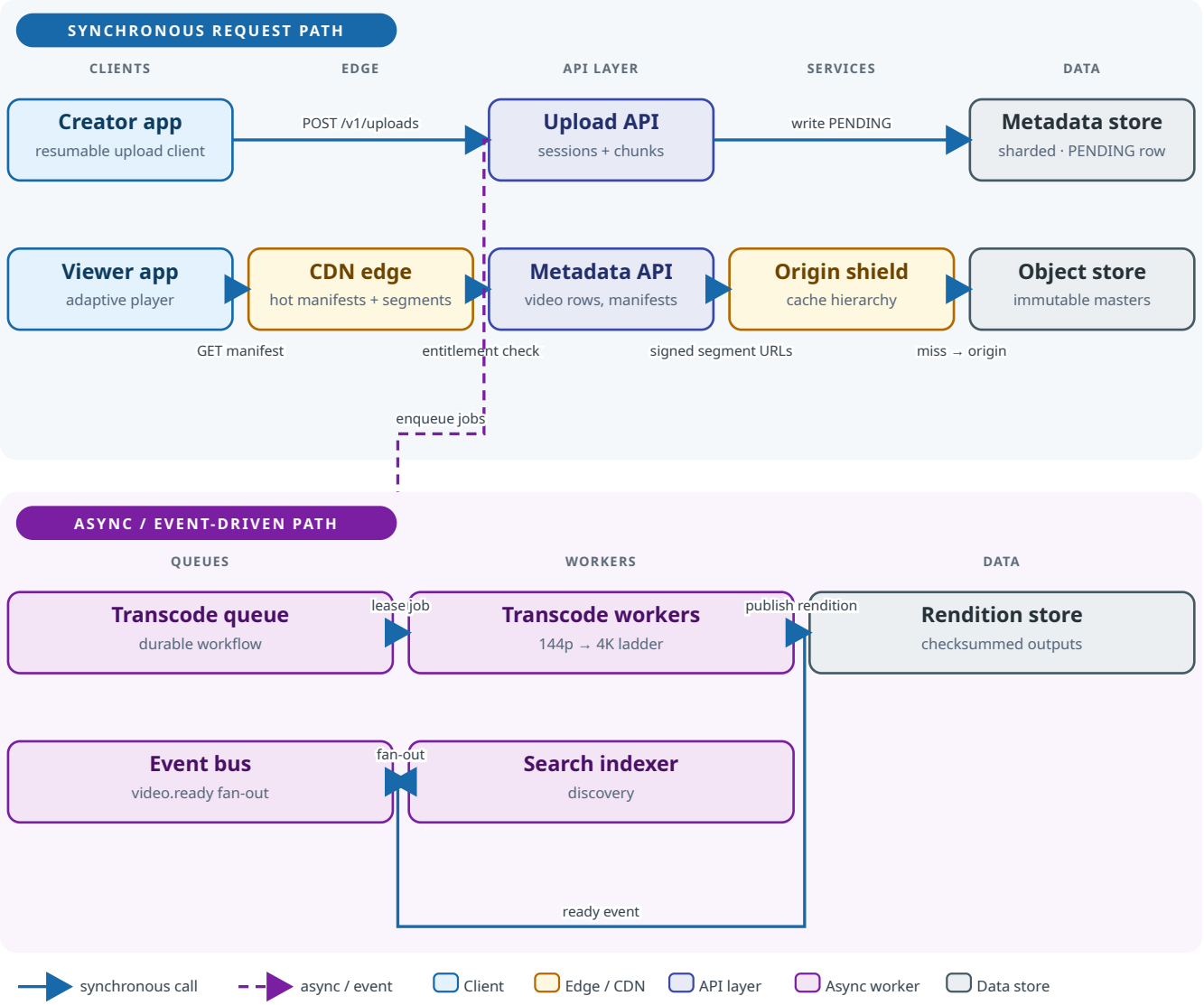
- A celebrity upload creates 100× traffic—what changes?
- How do you enforce deletion or regional policy quickly?
- How would live streaming alter the architecture?

TIE IT TO YOUR BACKGROUND

- Lead with your AWS monolith-to-microservices migration: clear service boundaries improved latency by 30% and APIs by 40%.
- Use your FinOps result—30% infrastructure-cost reduction—to discuss CDN, storage tiering, and transcoding economics.
- Connect asynchronous transcoding to your event-driven architecture and message-broker/OpenShift automation experience.

Architecture diagram — question 1

YouTube — upload & playback



Uploads take an async, durable path; playback is a hot read path served from the edge.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 1

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Transcode synchronously during upload vs durable async transcode jobs** → chosen async durable queue because uploads must acknowledge in milliseconds while transcoding is bursty, long-running work; a durable workflow retries per rendition and survives worker crashes without losing the upload.

2. **Decision: Serve playback from origin vs multi-tier CDN edge cache** → chosen CDN edge with an origin shield because video bytes dominate egress cost; edge hits cut time-to-first-byte and the shield collapses concurrent misses so a viral video never melts the origin.

3. **Decision: Relational metadata vs NoSQL document store** → chosen NoSQL keyed by video_id because the access pattern is key lookup at massive scale with a flexible schema; the only strongly consistent transition (PENDING → READY) is guarded by a conditional write, so full ACID buys nothing.

4. **Decision: Mutable segments with invalidation vs immutable segments with versioned manifests** → chosen immutable segments because a segment never changes after encoding; edge caches never need invalidation and only the small manifest is versioned per publish.

5. **Decision: Single-shot upload vs resumable chunked upload sessions** → chosen resumable chunked sessions because creators upload multi-gigabyte files over unreliable networks; per-chunk checksums let a dropped connection resume instead of restarting.

End-to-end flows and API contracts — question 1

End-to-end request flows

JOURNEY 1 — CREATOR UPLOADS A VIDEO

1. App calls POST /v1/uploads; Upload API returns upload_id, session URI, and chunk size.
2. App PUTs each chunk with Content-Range; the API appends to a staging object, resumable on failure.
3. On the final chunk, POST /v1/videos commits: checksum validated, master object marked immutable.
4. Metadata row (PENDING) is written to the sharded store; transcode jobs enter the durable queue.
5. Workers produce the rendition ladder (144p-4K); each rendition is checksummed and scanned.
6. When required renditions pass, the row flips to READY and video.ready events fan out to indexers.
7. The manifest service mints short-lived signed playback URLs; clients never see raw object paths.

JOURNEY 2 — VIEWER WATCHES A VIDEO

1. Client calls GET /v1/videos/{id}; Metadata API serves the sharded row, edge-cached when hot.
2. GET /v1/videos/{id}/manifest returns signed segment URLs after entitlement and region checks.
3. Playback hits the nearest CDN edge; misses cascade edge to origin shield to object storage.
4. The player adapts bitrate from buffer and throughput, switching renditions without rebuffering.
5. View events stream async to analytics; they never block the playback path.
6. Viral spikes are absorbed by edge cache; the metadata row is served cached with a short TTL.
7. Regional origin failure fails over to a replica; already-cached edge content keeps playing.

API contracts

POST `/v1/uploads` Create a resumable upload session

Auth: OAuth 2.0 Bearer · Content-Type: application/json

```
→ {"title":"My trip to Banff","privacy":"public","expected_bytes":2147483648}
← 201 {"upload_id":"upl_9f3","session_uri":"https://up.example.com/s/upl_9f3","chunk_size":8388608}
```

PUT `/s/{upload_id}` Upload one chunk; positional and resumable

Auth: Session token (query param) · Content-Range: bytes 0-8388607/2147483648

```
→ <binary chunk bytes>
← 200 {"upload_id":"upl_9f3","bytes_received":8388608,"complete":false}
```

POST `/v1/videos` Commit upload; start transcode workflow

Auth: OAuth 2.0 Bearer · Idempotency-Key: <uuid>

```
→ {"upload_id":"upl_9f3","checksum_sha256":"e3b0..."}
← 202 {"video_id":"vid_77q","status":"processing"}
```

GET `/v1/videos/{id}/manifest` Signed HLS/DASH manifest

Auth: OAuth 2.0 Bearer or signed cookie

```
→ —
← 200 {"manifest_url":"https://cdn.example.com/m/vid_77q/master.m3u8?sig=...","ladder":["1080p","720p"]}
```

How I would answer question 1

MODEL ANSWER - AS THE INTERVIEWEE

G1. Design YouTube

I'll start by narrowing the problem before drawing architecture. For Design YouTube, I would ask: Video on demand only, or live streaming too; Target regions, active users, and peak plays per second; Required startup latency and rebuffering target; Supported resolutions, codecs, and creator controls? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 100M daily viewers, 1B plays/day (~12K average; 10× peak), 5M uploads/day, and 300 MB average source size. That is ~1.5 PB/day before renditions; CDN egress dominates cost. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Creator → Upload API → Object store → Transcode queue → Renditions Then Viewer → Metadata API → CDN/edge → origin shield → Object store. The end-to-end design is: Use resumable multipart uploads, immutable object storage, a durable workflow for validation/transcoding, metadata in a sharded store, and signed manifests. Route playback through CDN edges with an origin shield; publish only after required renditions pass checks. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Hot videos: multi-tier caching and request coalescing reduce origin load; pre-warming improves latency but spends bandwidth speculatively. The second is Processing reliability: idempotent jobs, content-addressed outputs, dead-letter queues, and per-rendition status prevent duplicate work. The third is Global availability: replicate metadata and media by demand; choose regional consistency separately for counters, comments, and takedowns. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "A celebrity upload creates 100× traffic—what changes?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Lead with your AWS monolith-to-microservices migration: clear service boundaries improved latency by 30% and APIs by 40%. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 2 of 28

G2. Design Google Maps

WHY THEY ASK IT

Tests spatial modeling, read-heavy global serving, graph routing, streaming traffic updates, and graceful degradation. The key signal is knowing which work is precomputed and which must remain real time.

CLARIFYING QUESTIONS TO ASK

- Navigation, map rendering, place search, or all three?
- Driving only, or walking and transit too?
- Freshness target for incidents and traffic?
- Offline maps required?
- Accuracy, latency, and privacy constraints for location data?

SCALE ESTIMATION

Assume 50M daily users, 10 route requests/user/day, 5× peak (~30K route QPS), and billions of road edges. Tile reads are far higher and should be edge-cached.

HIGH-LEVEL DESIGN

GPS probes → stream → traffic aggregator → edge weights
Client → Tile CDN / Route API → graph shards → route engine

Serve versioned vector tiles from a CDN. Partition the road graph by geography, precompute hierarchy shortcuts, and apply near-real-time traffic as dynamic edge weights. Use a place index for geocoded search and return cached alternatives when live routing degrades.

DEEP DIVES

- Routing: contraction hierarchies or multi-level Dijkstra trade preprocessing cost for query speed; traffic updates must not invalidate the whole graph.
- Geospatial partitioning: S2/geohash cells improve locality but create boundary and density-skew problems.
- Privacy: aggregate probes, minimize retention, apply access controls, and avoid exposing individual trajectories.

LIKELY FOLLOW-UPS

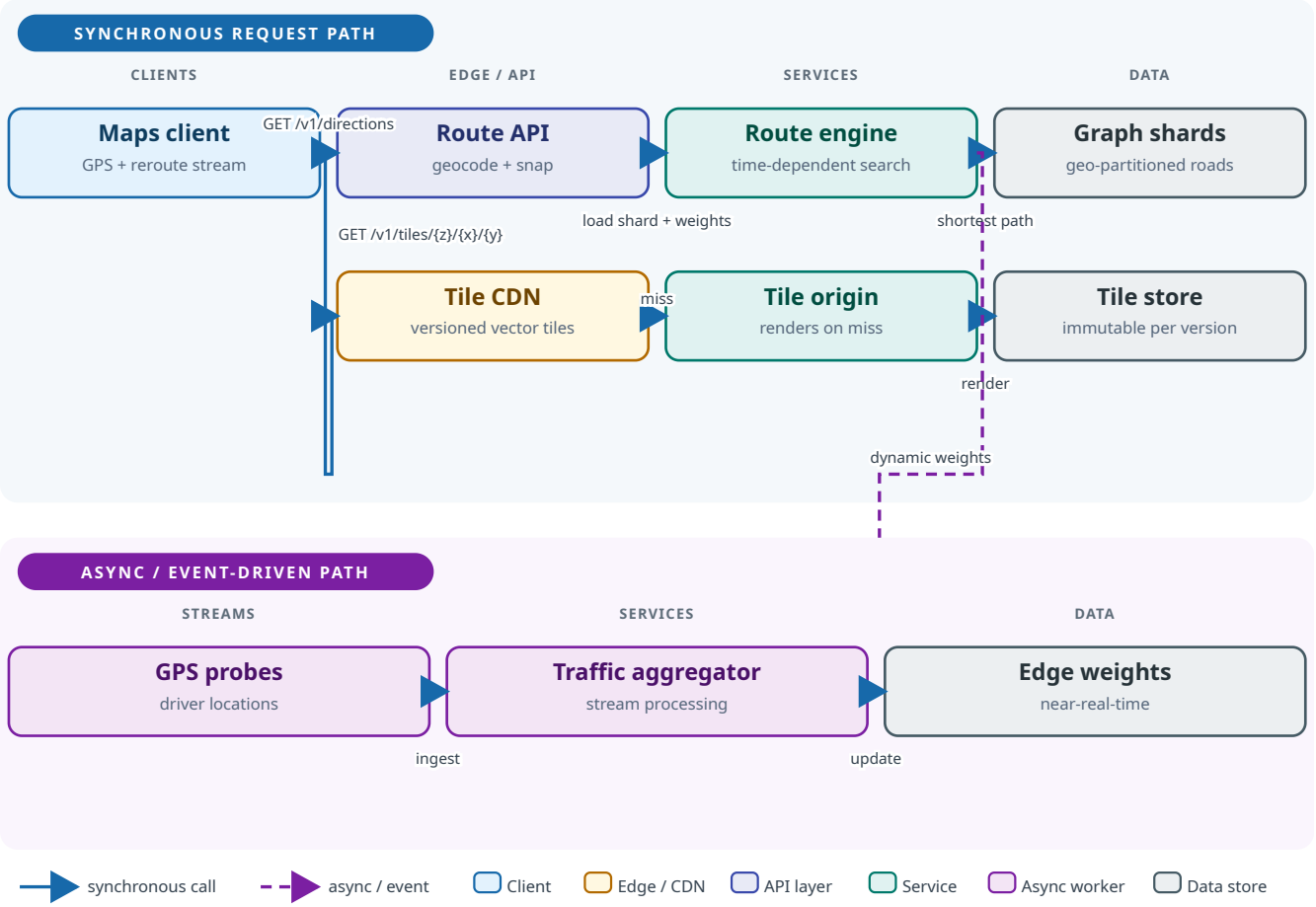
- A major event changes traffic in minutes—how do updates propagate?
- How do cross-region routes traverse graph partitions?
- How does the client behave with no connectivity?

TIE IT TO YOUR BACKGROUND

- Reference your low-latency API architecture and 40% API improvement when explaining route-service budgets and cache placement.
- Map streaming updates to your message-broker and event-driven platform work; emphasize backpressure and idempotency.
- Use your global 80+ engineer organization to discuss separate ownership for tiles, routing, traffic, and privacy controls.

Architecture diagram — question 2

Google Maps — tiles & routing



Tiles are precomputed and cached; routing blends precomputed shortcuts with live traffic weights.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 2

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Run Dijkstra per request vs precomputed contraction hierarchies** → chosen precomputed hierarchies with a live traffic overlay because per-request shortest-path over a planetary graph cannot meet millisecond budgets; the hierarchy answers in milliseconds and traffic deltas apply as edge-weight overlays.

2. **Decision: Mutable shared tiles vs versioned tile generations** → chosen versioned immutable tile generations because map tiles are read-heavy and cacheable; new map data publishes as a generation and clients cut over atomically instead of seeing torn tiles.

3. **Decision: Client-computed vs server-computed ETAs** → chosen server-computed with edge-cached segments because ETAs need live fusion of millions of probe reports; clients pull compact deltas rather than each phone re-solving traffic.

4. **Decision: Relational vs NoSQL store for POIs** → chosen NoSQL documents with a geospatial index because POI reads are key- and range-based with flexible attributes; S2-cell sharding keeps geographically nearby data co-located for fast region queries.

5. **Decision: Push every probe vs pull traffic deltas** → chosen pull of compact tile deltas because millions of clients pushing continuous GPS is a write flood; the server aggregates probes and clients pull deltas on a short interval.

End-to-end flows and API contracts — question 2

End-to-end request flows

JOURNEY 1 — DRIVER REQUESTS TURN-BY-TURN NAVIGATION

1. App calls GET /v1/directions with origin, destination, and departure time.
2. Route API geocodes endpoints via the place index and snaps GPS to road segments.
3. The engine loads the regional graph shard: hierarchy shortcuts plus live traffic edge weights.
4. Time-dependent shortest-path search returns 2-3 candidates with ETA, distance, and tolls.
5. The response is cached per origin/destination cell; the client subscribes to reroute updates.
6. GPS probes stream in; the traffic aggregator refreshes edge weights in near-real time.
7. If congestion appears ahead, a reroute candidate is pushed over the open stream.

JOURNEY 2 — MAP TILES RENDER WHILE PANNING

1. Client requests tile coordinates (z/x/y) from the tile CDN.
2. Hits return versioned vector tiles in milliseconds; misses render from the tile origin.
3. Tiles are immutable per data version, so edges cache them for weeks with no invalidation.
4. Traffic and incidents arrive on a separate stream and composite client-side.
5. Place search hits the geohash-sharded index, biased to the visible viewport.
6. Offline packs take over automatically when connectivity drops.

API contracts

GET `/v1/directions` Compute routes between two points

Auth: API key

```
→ ?origin=43.65,-79.38&destination=43.70,-79.40&mode=driving&alternatives=2
← 200 {"routes":[{"eta_s":1320,"distance_m":18400,"tolls":2.50}],"data_version":"2026.38"}
```

GET `/v1/tiles/{z}/{x}/{y}.mvt` Versioned vector tile

Auth: None (CDN) · X-Data-Version: 2026.38

```
→ —
← 200 <protobuf tile> · Cache-Control: public, max-age=1209600, immutable
```

POST `/v1/traffic/probes` Ingest anonymized GPS probe batch

Auth: Device token · Content-Type: application/json

```
→ {"trip_id":"t_881","points":[{"lat":43.65,"lng":-79.38,"speed":12.5}]}
← 202 {"accepted":3}
```

GET `/v1/places:search` Geocoded place search

Auth: API key

```
→ ?q=blue+jays+stadium&near=43.65,-79.38&limit=5
← 200 {"places":[{"place_id":"pl_991","name":"Rogers Centre","score":0.98}]}
```

How I would answer question 2

MODEL ANSWER - AS THE INTERVIEWEE

G2. Design Google Maps

I'll start by narrowing the problem before drawing architecture. For Design Google Maps, I would ask: Navigation, map rendering, place search, or all three; Driving only, or walking and transit too; Freshness target for incidents and traffic; Offline maps required? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 50M daily users, 10 route requests/user/day, 5× peak (~30K route QPS), and billions of road edges. Tile reads are far higher and should be edge-cached. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: GPS probes → stream → traffic aggregator → edge weights Then Client → Tile CDN / Route API → graph shards → route engine. The end-to-end design is: Serve versioned vector tiles from a CDN. Partition the road graph by geography, precompute hierarchy shortcuts, and apply near-real-time traffic as dynamic edge weights. Use a place index for geocoded search and return cached alternatives when live routing degrades. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Routing: contraction hierarchies or multi-level Dijkstra trade preprocessing cost for query speed; traffic updates must not invalidate the whole graph. The second is Geospatial partitioning: S2/geohash cells improve locality but create boundary and density-skew problems. The third is Privacy: aggregate probes, minimize retention, apply access controls, and avoid exposing individual trajectories. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "A major event changes traffic in minutes—how do updates propagate?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Reference your low-latency API architecture and 40% API improvement when explaining route-service budgets and cache placement. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 3 of 28

G3. Design Google Search autocomplete / typeahead

WHY THEY ASK IT

Tests extreme read latency, ranking, prefix retrieval, freshness, and abuse resistance. A strong answer distinguishes offline model generation from the tiny online critical path.

CLARIFYING QUESTIONS TO ASK

- Prefix suggestions only, or personalized history too?
- Languages, spelling correction, and safety policy?
- Latency target from keystroke to render?
- How fresh must trending queries be?
- Can user history leave the device?

SCALE ESTIMATION

Assume 1B searches/day and 6 keystrokes/search: ~70K average suggestion QPS, 10× peak. Responses are small, but p99 latency should stay below roughly 100 ms end to end.

HIGH-LEVEL DESIGN

Query logs → filter/aggregate → ranker build → versioned trie
Keystroke → edge cache → prefix service → ranked top K

Build compact weighted prefix indexes offline, shard by prefix/language, and deploy immutable versions to regional memory. Overlay a small streaming trend index and optionally merge privacy-safe personalization from the client.

DEEP DIVES

- Data structure: trie or finite-state transducer offers fast prefix lookup; top-K lists increase memory but shrink online compute.
- Freshness: batch rankers are stable; streaming overlays capture trends but increase abuse and quality risk.
- Safety: remove sensitive/low-frequency queries, apply policy filters, and monitor harmful suggestion rates.

LIKELY FOLLOW-UPS

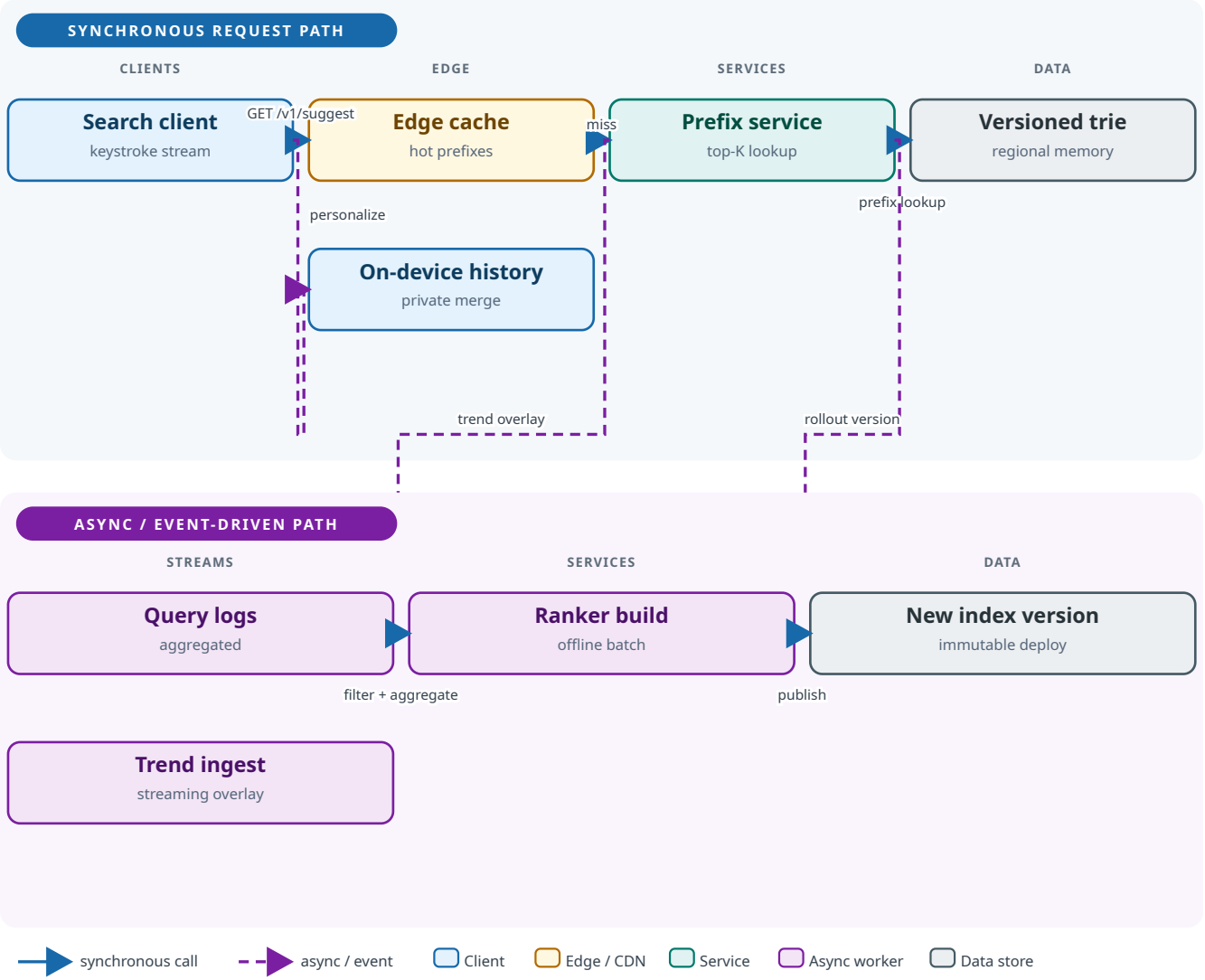
- How do you handle a breaking-news spike?
- How do you support misspellings and multilingual input?
- What is cached when prefixes are highly skewed?

TIE IT TO YOUR BACKGROUND

- Cite your API-first modernization and 40% faster APIs to frame the end-to-end latency budget.
- Connect the streaming overlay to your event-driven architecture; describe isolation so trend failures do not break baseline suggestions.
- Use OWASP and bug-bounty remediation experience when discussing query abuse, injection, and safe logging.

Architecture diagram — question 3

Search typeahead — suggest as you type



A tiny online critical path over immutable indexes; all heavy ranking stays offline.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 3

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Scan candidates per keystroke vs precomputed in-memory trie shards** → chosen precomputed trie shards because p99 must stay under ~50ms; a per-keystroke scan over millions of queries cannot meet that budget at peak QPS.

2. **Decision: Strongly consistent popularity counts vs eventually consistent** → chosen eventually consistent because a slightly stale trending list is invisible to users, while serializing every impression write would throttle the write path.

3. **Decision: Origin-only vs edge-plus-application cache** → chosen edge plus application cache because typeahead is extremely read-heavy with a Zipf distribution; the top prefixes are served from edge memory and the tail falls through.

4. **Decision: Server-side personalization per keystroke vs global list with client re-rank** → chosen global list with client-side re-rank because per-keystroke personalization explodes server state; the client re-ranks the global candidates using local history for free.

5. **Decision: Synchronous logging of every keystroke vs async batched analytics** → chosen async batched because logging must never add latency to the critical path; batches feed popularity counters and spell models offline.

End-to-end flows and API contracts — question 3

End-to-end request flows

JOURNEY 1 — USER TYPES A QUERY PREFIX

1. Each debounced keystroke calls GET /v1/suggest with prefix, language, and region.
2. The edge cache serves popular prefixes from memory in single-digit milliseconds.
3. On a miss, the regional prefix service looks up the versioned trie shard for that prefix range.
4. Baseline completions merge with the streaming trend index for breaking queries.
5. On-device history optionally re-ranks locally; raw history never leaves the device.
6. The top-K list returns; the client prefetches results for the top item.
7. If the service is slow, the edge serves a slightly stale list rather than blocking typing.

JOURNEY 2 — OFFLINE INDEX REBUILD AND ROLLOUT

1. Query logs pass through PII/bot filtering, then aggregate counts per prefix and region.
2. The ranker build emits compact weighted trie segments, sharded by prefix and language.
3. A new immutable version publishes with a manifest; regions swap via blue/green.
4. Canary regions validate click-through and latency before the global rollout.
5. The trend overlay supplies freshness between full rebuilds.
6. Spam completions are blocklisted at build time and enforced at serve time.

API contracts

GET `/v1/suggest` Ranked completions for a prefix

Auth: API key (optional) · Accept-Language: en-US

```
→ ?q=best+cof&hl=en&gl=us&limit=8
← 200 {"completions":[{"text":"best coffee near me","score":0.97}],"index_version":"2026-09-18T06:00Z"}
```

POST `/v1/suggest/feedback` Log impression/click for ranking eval

Auth: API key · Content-Type: application/json

```
→ {"query":"best cof","clicked":"best coffee near me","latency_ms":14}
← 202 {"accepted":true}
```

GET `/v1/index/versions` Published trie versions (ops)

Auth: Service account

```
→ —
← 200 {"versions":[{"id":"2026-09-18T06:00Z","status":"serving","traffic_pct":100}]}
```

POST `/v1/trends/ingest` Push breaking-query spike overlay

Auth: Service account · Content-Type: application/json

```
→ {"query":"eclipse tickets","region":"us","spike_score":0.93,"ttl_s":3600}
← 202 {"overlayed":true}
```

How I would answer question 3

MODEL ANSWER - AS THE INTERVIEWEE

G3. Design Google Search autocomplete / typeahead

I'll start by narrowing the problem before drawing architecture. For Design Google Search autocomplete / typeahead, I would ask: Prefix suggestions only, or personalized history too; Languages, spelling correction, and safety policy; Latency target from keystroke to render; How fresh must trending queries be? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 1B searches/day and 6 keystrokes/search: ~70K average suggestion QPS, 10× peak. Responses are small, but p99 latency should stay below roughly 100 ms end to end. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Query logs → filter/aggregate → ranker build → versioned trie Then Keystroke → edge cache → prefix service → ranked top K. The end-to-end design is: Build compact weighted prefix indexes offline, shard by prefix/language, and deploy immutable versions to regional memory. Overlay a small streaming trend index and optionally merge privacy-safe personalization from the client. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Data structure: trie or finite-state transducer offers fast prefix lookup; top-K lists increase memory but shrink online compute. The second is Freshness: batch rankers are stable; streaming overlays capture trends but increase abuse and quality risk. The third is Safety: remove sensitive/low-frequency queries, apply policy filters, and monitor harmful suggestion rates. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "How do you handle a breaking-news spike?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Cite your API-first modernization and 40% faster APIs to frame the end-to-end latency budget. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 4 of 28

G4. Design a URL shortener

WHY THEY ASK IT

Looks simple but exposes read/write asymmetry, identifier generation, cache strategy, abuse, and operational correctness. Interviewers test whether you find the hot path before adding infrastructure.

CLARIFYING QUESTIONS TO ASK

- Custom aliases and expiration required?
- Redirect latency and availability targets?
- Analytics synchronous or asynchronous?
- Can destinations be changed?
- Malware detection and rate limits?

SCALE ESTIMATION

Assume 100M new links/day (~1.2K writes/s) and 10B redirects/day (~116K reads/s), with a 10× peak. Ten years of mappings is ~365B rows before retention.

HIGH-LEVEL DESIGN

```
Create → ID allocator → mapping store
Redirect → edge/cache → mapping store → 301/302
                    └→ async analytics
```

Generate collision-free IDs from allocated ranges or randomized keys with collision checks. Partition by short code, cache popular mappings at the edge, and send click events asynchronously. Treat blocklists and deletion as fast invalidation paths.

DEEP DIVES

- ID strategy: centralized counters are simple but can bottleneck; range allocation removes the hot coordinator while preserving uniqueness.
- Caching: cache-first redirects reduce latency; negative caching and invalidation prevent abuse or deleted links from lingering.
- Redirect semantics: 301 improves caching but reduces control; 302 preserves measurement and destination changes.

LIKELY FOLLOW-UPS

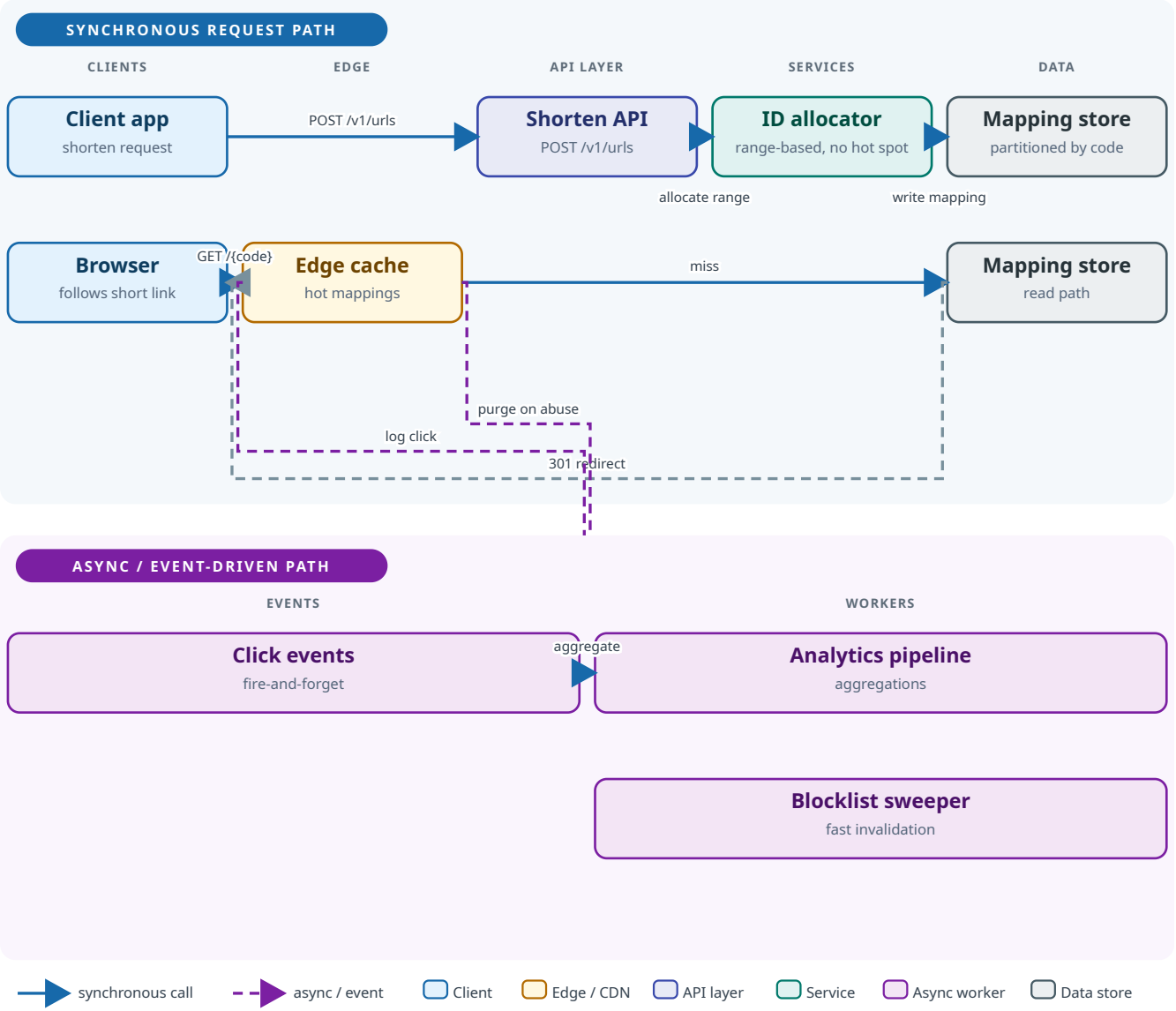
- One link becomes globally viral—what breaks first?
- How do you delete a malicious link immediately?
- How would custom aliases change consistency?

TIE IT TO YOUR BACKGROUND

- Anchor the answer in the partner onboarding platform: API access in under two hours and seven-system integration taught you to keep contracts simple.
- Use the 40% faster API result to explain cache hit-rate and p99 targets.
- Bring in OWASP and bug-bounty remediation for phishing, open-redirect abuse, and administrative takedowns.

Architecture diagram — question 4

URL shortener — create & redirect



Reads dominate: edge cache first, async analytics, fast invalidation for abuse.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 4

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Counter-based IDs vs random/hash-based IDs** → chosen counter-based IDs in base62 because sequential IDs are compact and collision-free by construction; random IDs need collision checks on every creation and longer keys for the same collision odds.

2. **Decision: Relational vs key-value store for mappings** → chosen replicated key-value store because the access pattern is pure key lookup at huge read volume; relational joins and transactions buy nothing for id → URL resolution.

3. **Decision: Synchronous write-through vs async write-back on creation** → chosen synchronous write with quorum because a short link must resolve immediately after it is shared; async creation risks 'link not found' seconds after the user posts it.

4. **Decision: HTTP 301 vs 302 redirects** → chosen 301 for permanent links because browsers and CDNs cache the redirect, slashing origin load; 302 is reserved for links whose destination genuinely changes.

5. **Decision: One namespace vs reserved custom aliases** → chosen a separate alias reservation index because vanity URLs are a product requirement; isolating them keeps the counter ID space dense and makes alias uniqueness a cheap indexed check.

End-to-end flows and API contracts — question 4

End-to-end request flows

JOURNEY 1 — USER SHORTENS A LONG URL

1. Client calls POST /v1/urls; the API validates, blocklist-checks, and normalizes the URL.
2. The allocator assigns a code from a pre-allocated range or a collision-checked random key.
3. The mapping is written to the store partitioned by code, with write-through to cache.
4. The API returns the short URL; Idempotency-Key makes retries safe.
5. Click analytics emit asynchronously; the create path never waits on them.
6. Custom aliases pass availability and reserved-word checks before allocation.
7. Abuse reports trigger fast invalidation: flag mapping, purge cache, update blocklist.

JOURNEY 2 — VISITOR FOLLOWS A SHORT LINK

1. Browser requests GET /{code}; the edge cache is checked first.
2. On a hit, the edge issues the 301/302 immediately; the origin sees nothing.
3. On a miss, the redirect service reads the mapping store partition for that code.
4. The service confirms the mapping is active, then returns the redirect and caches it.
5. A click event queues asynchronously for analytics.
6. Deleted codes return 410 Gone; hot codes get longer edge TTLs to absorb spikes.

API contracts

POST `/v1/urls` Create a short URL

Auth: API key · Content-Type: application/json · Idempotency-Key: <uuid>

```
→ {"long_url":"https://example.com/articles/cache-guide","ttl_days":365}
← 201 {"code":"aB3x9Z","short_url":"https://sho.rt/aB3x9Z"}
```

GET `/{code}` Redirect to the long URL

Auth: None

```
→ —
← 301 Location: https://example.com/articles/cache-guide
```

GET `/v1/urls/{code}/stats` Click analytics for a code

Auth: API key

```
→ ?from=2026-09-01&group_by=day
← 200 {"code":"aB3x9Z","series":[{"day":"2026-09-17","clicks":1204}]}
```

DELETE `/v1/urls/{code}` Delete mapping; purge edge cache

Auth: API key

```
→ —
← 204 (mapping tombstoned; cache purged within seconds)
```

How I would answer question 4

MODEL ANSWER - AS THE INTERVIEWEE

G4. Design a URL shortener

I'll start by narrowing the problem before drawing architecture. For Design a URL shortener, I would ask: Custom aliases and expiration required; Redirect latency and availability targets; Analytics synchronous or asynchronous; Can destinations be changed? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 100M new links/day (~1.2K writes/s) and 10B redirects/day (~116K reads/s), with a 10× peak. Ten years of mappings is ~365B rows before retention. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Create → ID allocator → mapping store Then Redirect → edge/cache → mapping store → 301/302 Then └→ async analytics. The end-to-end design is: Generate collision-free IDs from allocated ranges or randomized keys with collision checks. Partition by short code, cache popular mappings at the edge, and send click events asynchronously. Treat blocklists and deletion as fast invalidation paths. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is ID strategy: centralized counters are simple but can bottleneck; range allocation removes the hot coordinator while preserving uniqueness. The second is Caching: cache-first redirects reduce latency; negative caching and invalidation prevent abuse or deleted links from lingering. The third is Redirect semantics: 301 improves caching but reduces control; 302 preserves measurement and destination changes. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "One link becomes globally viral—what breaks first?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Anchor the answer in the partner onboarding platform: API access in under two hours and seven-system integration taught you to keep contracts simple. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 5 of 28

G5. Design WhatsApp

WHY THEY ASK IT

Tests long-lived connections, message ordering, offline delivery, fan-out, presence, and end-to-end encryption boundaries. Product semantics matter as much as throughput.

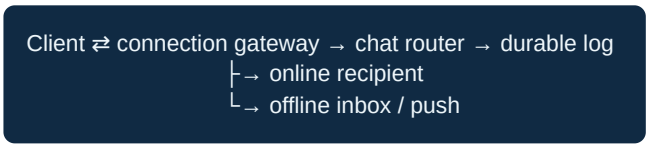
CLARIFYING QUESTIONS TO ASK

- One-to-one only, or groups and media too?
- Ordering scope: per chat or global?
- Delivery/read receipts required?
- Multi-device history and offline retention?
- Metadata and privacy constraints?

SCALE ESTIMATION

Assume 500M daily users, 50 messages/user/day: 25B/day (~290K average; 5× peak). At 1 KB/message, payload is ~25 TB/day before replicas and media.

HIGH-LEVEL DESIGN



Maintain regional WebSocket gateways, assign each conversation to an ordering partition, persist encrypted envelopes in an append-only log, and deliver through online sessions or durable offline inboxes. Keys remain client-controlled; the service sees routing metadata, not plaintext.

DEEP DIVES

- Ordering and dedupe: per-conversation sequence numbers and idempotency keys simplify correctness without global ordering.
- Groups: sender keys reduce encryption overhead; fan-out-on-write improves reads but amplifies writes for large groups.
- Presence: ephemeral and eventually consistent; avoid making it a dependency for message delivery.

LIKELY FOLLOW-UPS

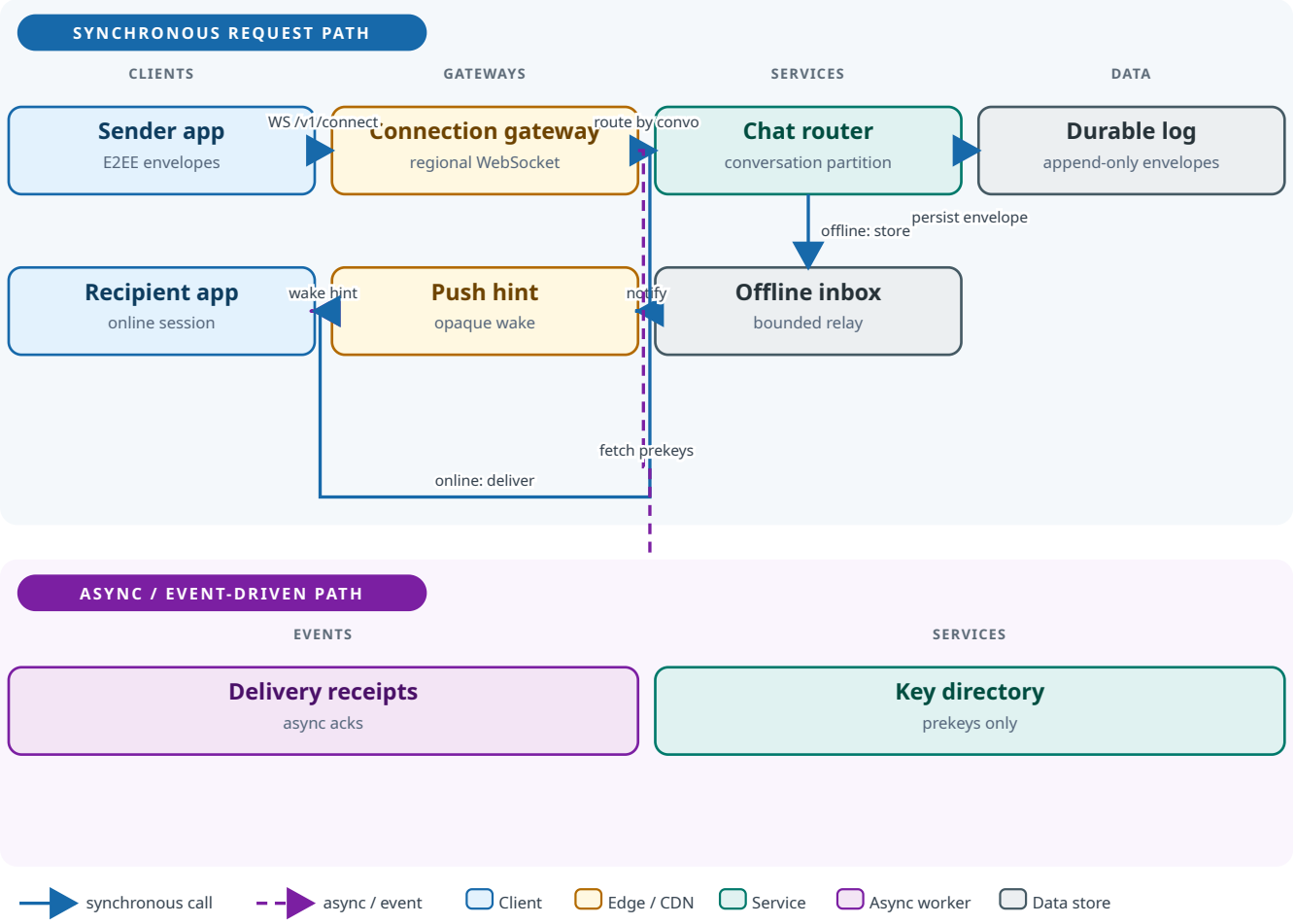
- A user has five devices—how are sessions and keys handled?
- What happens when receipts arrive out of order?
- How do you support abuse reporting with E2E encryption?

TIE IT TO YOUR BACKGROUND

- Use AT&T native-mobile security architecture: OAuth 2.0, JWT, 2FA, step-up auth, and TLS 1.2 establish credible identity expertise.
- Connect real-time routing to your low-latency API and event-driven platform work; do not claim messaging-product ownership.
- Discuss threat modeling with OWASP Top 10 and bug-bounty remediation experience.

Architecture diagram — question 5

WhatsApp — messaging with E2E encryption



E2EE: the service routes envelopes and metadata — never plaintext.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 5

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: TLS only vs Signal-protocol end-to-end encryption** → chosen E2E because the server must never see plaintext; TLS alone leaves message content exposed to server compromise or insider access.

2. **Decision: Direct peer-to-peer delivery vs store-and-forward relay** → chosen store-and-forward relay because recipients are offline most of the time; the relay holds bounded envelopes until the recipient's cursor fetch, then deletes them.

3. **Decision: Push on every message vs pull with cursor** → chosen opaque push hint plus cursor pull because pure push wakes the radio constantly (battery drain) and pure pull adds latency; a tiny hint triggers exactly one fetch.

4. **Decision: Fan-out on write vs fan-out on read for groups** → chosen fan-out on write into per-recipient queues for normal groups because reads become single-queue fetches; very large groups fall back to a shared cursor so one message doesn't multiply into millions of writes.

5. **Decision: Relational vs LSM NoSQL for envelopes** → chosen LSM-backed NoSQL keyed by (recipient, cursor) because writes are append-only and reads are range scans; relational MVCC overhead buys nothing for an ephemeral queue.

End-to-end flows and API contracts — question 5

End-to-end request flows

JOURNEY 1 — USER SENDS A MESSAGE TO AN ONLINE CONTACT

1. Client opens a WebSocket to the regional gateway and authenticates with a session token.
2. The client encrypts on-device and sends a frame with a client-generated message_id.
3. The gateway forwards the envelope to the chat router for the conversation partition.
4. The router appends the encrypted envelope to the append-only log with a sequence number.
5. It pushes the envelope to the recipient's live gateway session; the service sees only metadata.
6. The recipient decrypts, renders, and returns a delivery receipt recorded against the log entry.
7. Duplicate frames are deduplicated by message_id before appending; sends are idempotent.

JOURNEY 2 — RECIPIENT COMES BACK ONLINE

1. Client reconnects and calls GET /v1/inbox with its last acknowledged cursor.
2. The bounded offline inbox returns envelopes after the cursor, in conversation order.
3. The client decrypts, renders in sequence, then POSTs acknowledgements.
4. Acknowledged envelopes become eligible for inbox compaction.
5. Push carries only an opaque wake hint; presence updates are best-effort on the same socket.
6. Group messages fan out as one envelope per recipient device, each separately encrypted.

API contracts

WS `/v1/connect` Open persistent session

Auth: Session token (subprotocol) · Sec-WebSocket-Protocol: chat.v1

```
→ → {"type":"auth","token":"sess_...","last_cursor":"184422"}
← ← {"type":"auth_ok","pending":3}
```

WS `/v1/connect (frame)` Send an encrypted message frame

Auth: Established session

```
→ → {"type":"send","message_id":"m_c8f1","to":"+14155550132","ciphertext":"base64..."}
← ← {"type":"ack","message_id":"m_c8f1","seq":184423}
```

GET `/v1/inbox` Fetch missed envelopes after cursor

Auth: Session token

```
→ ?cursor=184420&limit=100
← 200 {"envelopes":[{"seq":184421,"ciphertext":"base64..."}],"next_cursor":184423}
```

GET `/v1/keys/prekeys/{user}` Recipient prekey bundle for E2EE setup

Auth: Session token

```
→ —
← 200 {"identity_key":"base64...","signed_prekey":{"id":12,"key":"base64..."}}
```

How I would answer question 5

MODEL ANSWER - AS THE INTERVIEWEE

G5. Design WhatsApp

I'll start by narrowing the problem before drawing architecture. For Design WhatsApp, I would ask: One-to-one only, or groups and media too; Ordering scope: per chat or global; Delivery/read receipts required; Multi-device history and offline retention? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 500M daily users, 50 messages/user/day: 25B/day (~290K average; 5× peak). At 1 KB/message, payload is ~25 TB/day before replicas and media. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Client ⇄ connection gateway → chat router → durable log Then ├→ online recipient Then └→ offline inbox / push. The end-to-end design is: Maintain regional WebSocket gateways, assign each conversation to an ordering partition, persist encrypted envelopes in an append-only log, and deliver through online sessions or durable offline inboxes. Keys remain client-controlled; the service sees routing metadata, not plaintext. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Ordering and dedupe: per-conversation sequence numbers and idempotency keys simplify correctness without global ordering. The second is Groups: sender keys reduce encryption overhead; fan-out-on-write improves reads but amplifies writes for large groups. The third is Presence: ephemeral and eventually consistent; avoid making it a dependency for message delivery. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "A user has five devices—how are sessions and keys handled?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Use AT&T native-mobile security architecture: OAuth 2.0, JWT, 2FA, step-up auth, and TLS 1.2 establish credible identity expertise. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 6 of 28

G6. Design a distributed rate limiter

WHY THEY ASK IT

Tests correctness under concurrency, latency, fairness, hot keys, and failure policy. The best answer ties the algorithm to a real enforcement point and service objective.

CLARIFYING QUESTIONS TO ASK

- Limit by user, API key, tenant, IP, or endpoint?
- Hard enforcement or approximate protection?
- Global, regional, or per-instance quota?
- Burst allowance and refill policy?
- Fail open or fail closed when state is unavailable?

SCALE ESTIMATION

Assume 1M requests/s, 100M active identities, and a 1-ms limiter budget. Only active counters should occupy the hot working set; enforcement must not require a cross-region round trip.

HIGH-LEVEL DESIGN

```
Request → local token cache → regional quota shard → service
                                  └→ async usage / policy stream
```

Store policies in a control plane. Enforce token buckets locally with leased quota from regional shards, partitioned by identity. Stream usage for reconciliation and expose headers describing remaining quota and retry time.

DEEP DIVES

- Accuracy versus availability: local leases can temporarily overspend; synchronous central checks are precise but fragile and slow.
- Hot tenants: isolate heavy keys, shard counters, and use hierarchical tenant/user limits.
- Failure mode: security endpoints may fail closed; low-risk reads may fail open with conservative local caps.

LIKELY FOLLOW-UPS

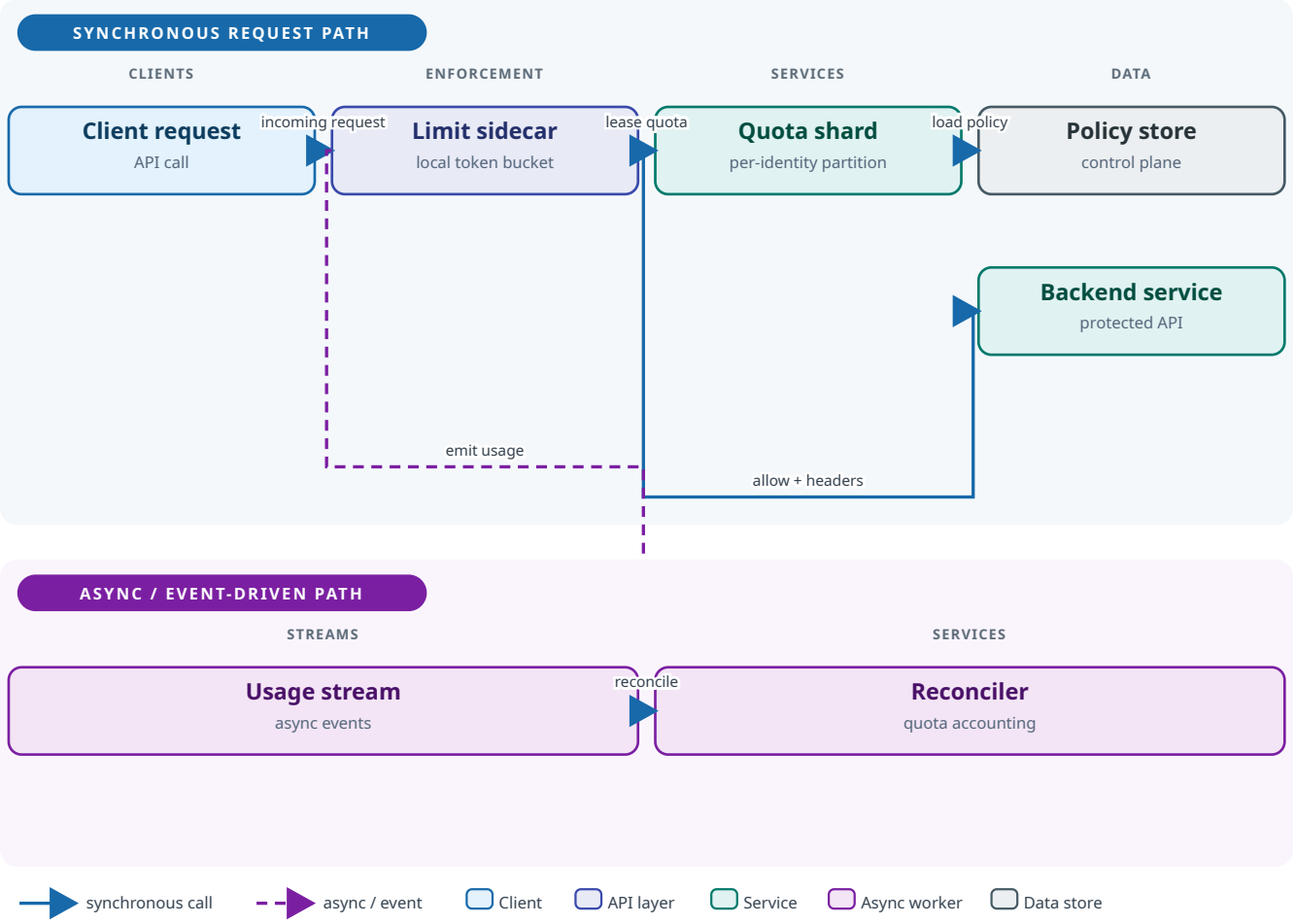
- How do you change limits without restarting clients?
- How do you prevent clock skew issues?
- What if one region loses quota-store connectivity?

TIE IT TO YOUR BACKGROUND

- Use your seven-system onboarding APIs to discuss tenant-level quotas and predictable partner contracts.
- Point to 40% faster APIs: a limiter must protect that latency gain, not consume it.
- Use zero-trust and step-up-auth experience to explain stricter policies for sensitive operations.

Architecture diagram — question 6

Distributed rate limiter



Enforce locally from leased quota; reconcile centrally — never on the hot path.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 6

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Central counter on the hot path vs locally enforced leased quota** → chosen leased quota with central reconciliation because a central counter on every request adds latency and a single point of contention; edge nodes enforce from a lease and reconcile usage asynchronously.

2. **Decision: Fixed window vs token bucket** → chosen token bucket because fixed windows allow a 2× burst at window boundaries; the bucket smooths traffic while staying O(1) per check.

3. **Decision: Synchronous check vs async sampling** → chosen synchronous check in the gateway data path because limits must be enforced before expensive work starts; async sampling cannot shed load in time during a spike.

4. **Decision: Exact global counts vs bounded-error approximate** → chosen bounded-error approximate because exact counts need cross-node coordination on every request; a small over-admission is acceptable, systematic under-admission is not.

5. **Decision: Queue excess requests vs reject with 429** → chosen reject with 429 and Retry-After because queueing excess during overload deepens the failure; fast rejection protects the system and tells well-behaved clients when to retry.

End-to-end flows and API contracts — question 6

End-to-end request flows

JOURNEY 1 — API REQUEST PASSES THE RATE LIMITER

1. The service calls the limiter with the caller identity (API key, user, or IP).
2. The local in-memory token bucket is checked first: no network hop in the common case.
3. If tokens exist, one is consumed and usage batches into the async usage stream.
4. If empty, the limiter requests a quota lease from the regional shard for that identity.
5. The shard grants a bounded lease based on policy and reconciled global usage.
6. Policy changes propagate from the control plane in seconds; granted leases are honored.
7. Every decision emits X-RateLimit-Remaining and Retry-After headers for the client.

JOURNEY 2 — QUOTA SHARD FAILS OVER

1. Health checks mark a shard unhealthy; the control plane promotes its replica.
2. In-flight leases stay valid; they are time-bounded and pre-issued.
3. Local buckets keep enforcing, so the data plane does not hard-stop during failover.
4. If no shard is reachable, the configured failure policy applies: fail-open or fail-closed.
5. The usage stream replays after recovery; reconciliation corrects any over-admission.
6. Canary policy changes roll to one region first, then globally on healthy rejection rates.

API contracts

POST `/v1/policies` Create a rate-limit policy

Auth: Admin token · Content-Type: application/json

```
→ {"name":"partner-tier-2","scope":"api_key","limit":1000,"window_s":60,"burst":200}
← 201 {"policy_id":"pol_221","version":1}
```

GET `/v1/quota/{identity}` Inspect remaining quota

Auth: API key (self) or admin

```
→ ?policy_id=pol_221
← 200 {"identity":"key_abc","limit":1000,"remaining":742,"resets_at":"..."}
```

POST `/internal/v1/leases` Shard grants a quota lease (internal)

Auth: mTLS service identity · Content-Type: application/json

```
→ {"identity":"key_abc","requested":500,"ttl_s":10}
← 200 {"granted":500,"lease_id":"lse_991"}
```

GET `/v1/usage` Usage time series for analysis

Auth: Admin token

```
→ ?identity=key_abc&granularity=1m
← 200 {"points":[{"t":"...","allowed":812,"rejected":3}]}
```

How I would answer question 6

MODEL ANSWER - AS THE INTERVIEWEE

G6. Design a distributed rate limiter

I'll start by narrowing the problem before drawing architecture. For Design a distributed rate limiter, I would ask: Limit by user, API key, tenant, IP, or endpoint; Hard enforcement or approximate protection; Global, regional, or per-instance quota; Burst allowance and refill policy? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 1M requests/s, 100M active identities, and a 1-ms limiter budget. Only active counters should occupy the hot working set; enforcement must not require a cross-region round trip. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Request → local token cache → regional quota shard → service Then └→ async usage / policy stream. The end-to-end design is: Store policies in a control plane. Enforce token buckets locally with leased quota from regional shards, partitioned by identity. Stream usage for reconciliation and expose headers describing remaining quota and retry time. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Accuracy versus availability: local leases can temporarily overspend; synchronous central checks are precise but fragile and slow. The second is Hot tenants: isolate heavy keys, shard counters, and use hierarchical tenant/user limits. The third is Failure mode: security endpoints may fail closed; low-risk reads may fail open with conservative local caps. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "How do you change limits without restarting clients?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Use your seven-system onboarding APIs to discuss tenant-level quotas and predictable partner contracts. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 7 of 28

G7. Design a distributed cache

WHY THEY ASK IT

Tests partitioning, eviction, consistency, replication, hot-key mitigation, and failure recovery. The interviewer wants a cache contract, not merely “use Redis.”

CLARIFYING QUESTIONS TO ASK

- Cache-aside, write-through, or primary data store?
- Object size and access distribution?
- Staleness tolerance and invalidation needs?
- Regional or global scope?
- Required behavior during node failure?

SCALE ESTIMATION

Assume a 100 TB working set, 5M operations/s, 90% reads, and 1 ms median target. With 256 GB usable/node and replication factor two, capacity alone needs ~800 nodes plus headroom.

HIGH-LEVEL DESIGN

```
Client library → consistent-hash ring → cache shard + replica
                                      └→ source of truth on miss
```

Use client-side or proxy routing over a versioned hash ring, replicated shards, TTL plus explicit invalidation, and cache-aside reads. Protect the backing store with request coalescing, bounded retries, and admission control.

DEEP DIVES

- Eviction: LRU is simple; TinyLFU-style admission better protects the working set from one-hit scans.
- Rebalancing: virtual nodes smooth distribution, but movement can cause miss storms; warm gradually and dual-read briefly.
- Consistency: versioned values and event-driven invalidation reduce stale reads without turning the cache into a database.

LIKELY FOLLOW-UPS

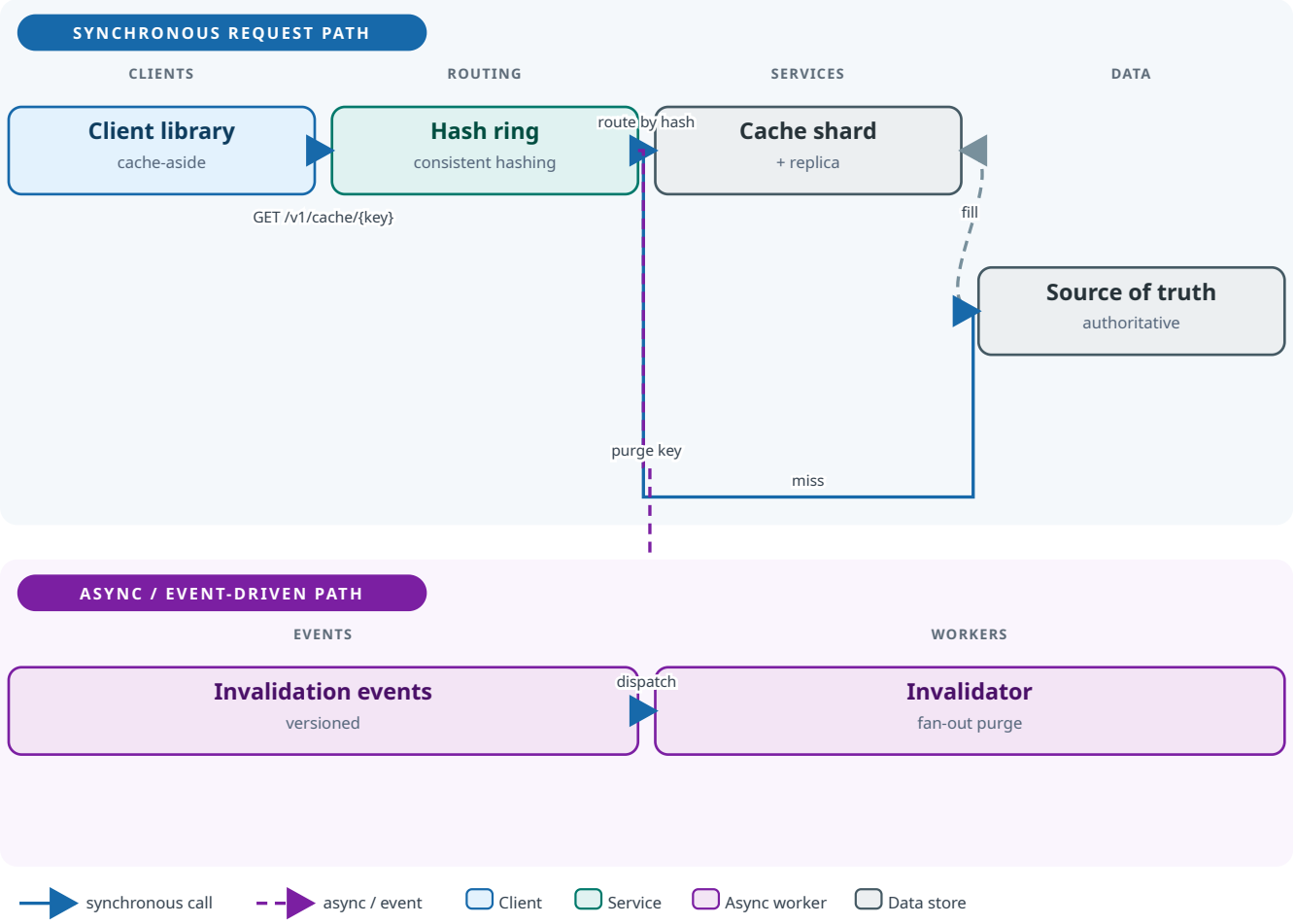
- A single key receives 10% of traffic—what do you do?
- How do you recover from a cold regional restart?
- How do you avoid cache stampedes?

TIE IT TO YOUR BACKGROUND

- Connect cache placement to the 30% latency reduction and 40% faster APIs from your AWS modernization.
- Use FinOps discipline: memory is expensive, so show how hit rate and avoided origin cost justify capacity.
- Relate invalidation to your event-driven architecture and message-broker experience.

Architecture diagram — question 7

Distributed cache — cache-aside



Cache-aside with versioned invalidation; shield the source on miss storms.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 7

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Cache-aside vs read-through vs write-through** → chosen cache-aside because the application controls freshness per key and the cache stays a pure performance layer; the database remains the source of truth, never the cache.

2. **Decision: Centralized directory vs consistent hashing** → chosen consistent hashing because a directory is a coordination bottleneck and a single point of failure; the ring lets any client route to the owning shard with no lookup.

3. **Decision: TTL expiry vs explicit versioned invalidation** → chosen versioned invalidation events because TTLs either serve stale data or miss too often; versioned events give precise freshness with per-key control.

4. **Decision: Single copy vs primary-plus-replica shards** → chosen primary plus replica per shard because losing a shard without a replica turns into a miss storm on the source of truth; replicas absorb failover without a cold start.

5. **Decision: Synchronous fill on miss vs async background fill** → chosen synchronous fill with singleflight because async fill serves empty or stale; singleflight coalesces concurrent misses so exactly one request hits the source while the rest wait on it.

End-to-end flows and API contracts — question 7

End-to-end request flows

JOURNEY 1 — READ WITH A CACHE HIT

1. The client library hashes the key and walks the versioned hash ring to the primary shard.
2. It sends GET to the shard owner, which serves the value from memory with TTL metadata.
3. Hot keys are detected by counters; replicas begin serving reads to spread load.
4. Stale-while-revalidate may serve a slightly expired value while refreshing in background.
5. Slow primaries trigger hedged requests to a replica; the first response wins.
6. Ring updates arrive versioned; the library drains old connections gracefully.
7. All reads carry a deadline so cache trouble never stalls the caller past its fallback budget.

JOURNEY 2 — READ WITH A MISS, THEN FILL (CACHE-ASIDE)

1. On a miss, request coalescing lets concurrent getters share one in-flight fill.
2. One thread loads the value from the source of truth.
3. The value is PUT back with a TTL; the primary shard replicates asynchronously.
4. Waiting getters all receive the value; errors return once and never poison the cache.
5. Negative caching briefly marks absent keys to shield the source from miss storms.
6. Invalidation arrives via explicit DELETE or pub/sub; versioned keys self-correct stale reads.

API contracts

GET `/v1/cache/{key}` Read a cached value

Auth: Service token

```
→ —
← 200 {"key":"user:88231","value":"{...}","ttl_s":295} · X-Cache: HIT
```

PUT `/v1/cache/{key}` Write a value with TTL

Auth: Service token · Content-Type: application/json

```
→ {"value":"{...}","ttl_s":300,"version":15}
← 200 {"key":"user:88231","stored":true,"replicas":2}
```

DELETE `/v1/cache/{key}` Invalidate a key cluster-wide

Auth: Service token

```
→ —
← 202 {"key":"user:88231","invalidated_shards":3}
```

POST `/v1/cache/batch` Multi-get for hydration

Auth: Service token · Content-Type: application/json

```
→ {"keys":["user:88231","user:88232"],"coalesce":true}
← 200 {"items":{"user:88231":{"hit":true},"user:88232":{"hit":false}}}
```

How I would answer question 7

MODEL ANSWER - AS THE INTERVIEWEE

G7. Design a distributed cache

I'll start by narrowing the problem before drawing architecture. For Design a distributed cache, I would ask: Cache-aside, write-through, or primary data store; Object size and access distribution; Staleness tolerance and invalidation needs; Regional or global scope? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume a 100 TB working set, 5M operations/s, 90% reads, and 1 ms median target. With 256 GB usable/node and replication factor two, capacity alone needs ~800 nodes plus headroom. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Client library → consistent-hash ring → cache shard + replica Then └→ source of truth on miss. The end-to-end design is: Use client-side or proxy routing over a versioned hash ring, replicated shards, TTL plus explicit invalidation, and cache-aside reads. Protect the backing store with request coalescing, bounded retries, and admission control. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Eviction: LRU is simple; TinyLFU-style admission better protects the working set from one-hit scans. The second is Rebalancing: virtual nodes smooth distribution, but movement can cause miss storms; warm gradually and dual-read briefly. The third is Consistency: versioned values and event-driven invalidation reduce stale reads without turning the cache into a database. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "A single key receives 10% of traffic—what do you do?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Connect cache placement to the 30% latency reduction and 40% faster APIs from your AWS modernization. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 8 of 28

G8. Design a notification system

WHY THEY ASK IT

Tests multi-channel orchestration, user preferences, prioritization, retries, provider failures, and observability. Strong answers protect users from duplicates and notification fatigue.

CLARIFYING QUESTIONS TO ASK

- Push, email, SMS, and in-app channels?
- Transactional versus marketing priorities?
- Per-user preferences and quiet hours?
- Delivery guarantee and latency target?
- Global compliance and opt-out requirements?

SCALE ESTIMATION

Assume 1B notifications/day (~12K average; 20× campaign peak), 80% push, 15% email, 5% SMS. Provider rate limits and fan-out spikes dominate design.

HIGH-LEVEL DESIGN

```
Producer → intake → preference/policy → priority queues
                                  └→ channel workers → providers
                                                  └→ receipts
```

Accept idempotent events, enrich with templates and preferences, route to priority/channel queues, and isolate provider adapters. Track lifecycle state, deduplicate by event and recipient, and ingest delivery receipts asynchronously.

DEEP DIVES

- Backpressure: transactional alerts get reserved capacity; bulk campaigns degrade or pause.
- Exactly-once illusion: at-least-once transport plus idempotency and user-visible dedupe is practical.
- Preferences: keep a strongly consistent opt-out path and cache reads carefully.

LIKELY FOLLOW-UPS

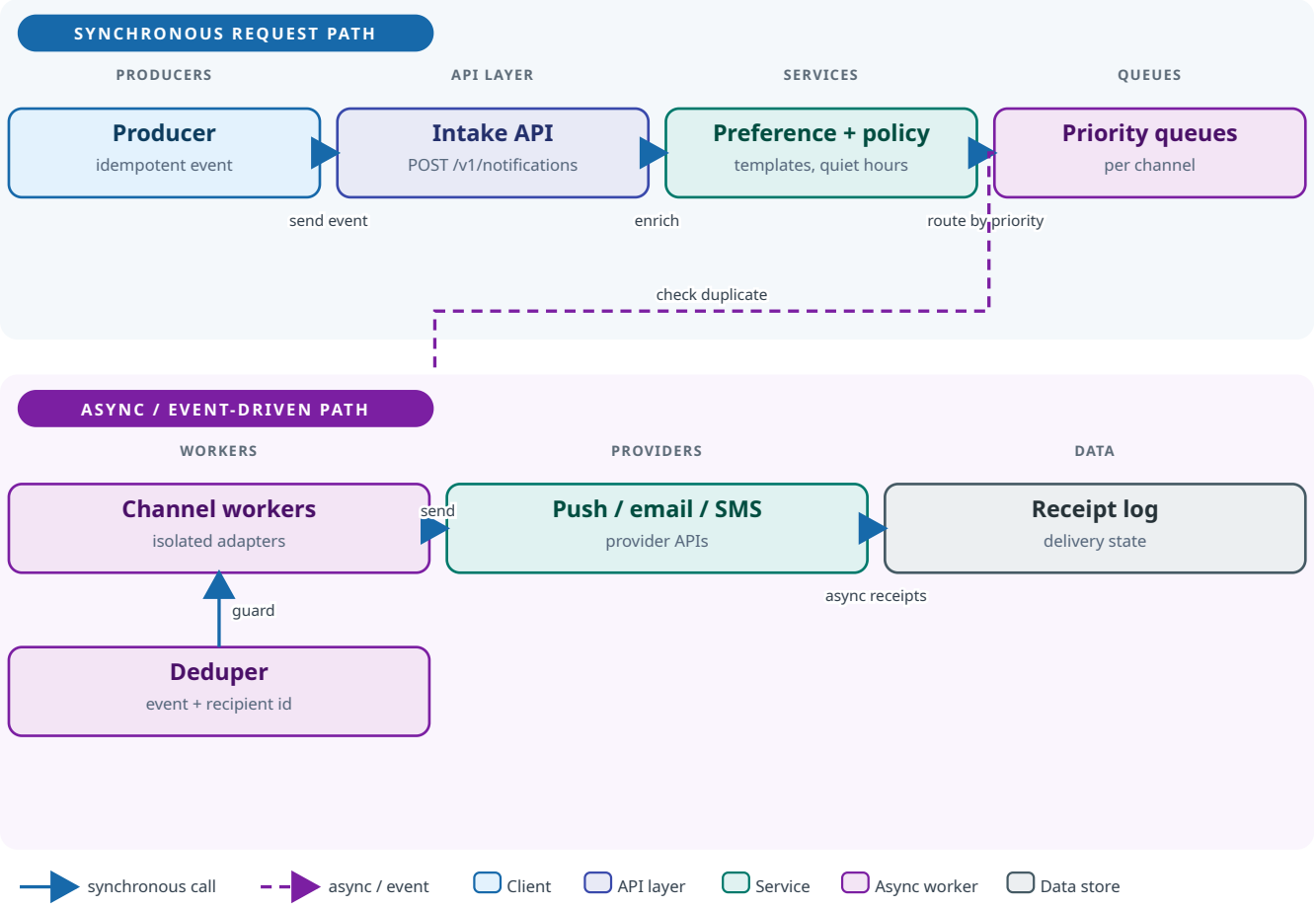
- An SMS provider fails during an incident—how do you fail over?
- How do you prevent a retry storm?
- How would you measure usefulness, not just delivery?

TIE IT TO YOUR BACKGROUND

- Use AT&T mobile identity/API work to discuss secure device registration and token lifecycle.
- Connect durable channel queues and retries to your message-broker/OpenShift automation that eliminated 99% of manual work.
- Use your 80+ engineer organization to define platform, channel, and product-team ownership.

Architecture diagram — question 8

Notification system — multi-channel



Transactional traffic gets reserved capacity; bulk campaigns degrade first.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 8

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: One pipeline per channel vs unified event pipeline** → chosen unified event pipeline with channel adapters because per-channel pipelines duplicate dedup, preference checks, and rate limiting; adapters isolate SMS/push/email quirks at the edge.

2. **Decision: Exactly-once vs at-least-once delivery** → chosen at-least-once with idempotent sends and dedup keys because exactly-once is impossible across third-party providers; idempotency keys make retries safe instead of duplicate.

3. **Decision: Immediate send vs preference-driven batching** → chosen preference-driven batching because waking users per event destroys trust; low-priority events collapse into digests honoring quiet hours.

4. **Decision: Send synchronously in the request path vs async durable queue** → chosen async durable queue because provider latency (seconds) must never block user requests; the queue also absorbs provider outages and backs off per provider.

5. **Decision: Active-active vs active-standby provider failover** → chosen active-standby with health checks because dual-sending duplicates messages; the standby takes over only when the primary is proven unhealthy.

End-to-end flows and API contracts — question 8

End-to-end request flows

JOURNEY 1 — PASSWORD-RESET EMAIL (TRANSACTIONAL)

1. The auth service POSTs an event with an idempotency key (user_id + event_id) to intake.
2. Intake validates, dedupes against recent event IDs, and persists the request as RECEIVED.
3. Enrichment loads locale, verified contact points, and quiet-hour preferences.
4. Policy checks run: verified email, approved template, suppression and rate limits.
5. The router places the message on the high-priority transactional queue for email.
6. A channel worker renders the template and calls the provider adapter; receipts update the lifecycle state async.
7. Provider failure triggers bounded backoff, then failover to the secondary provider.

JOURNEY 2 — PROMOTIONAL PUSH HONORING PREFERENCES

1. The producer submits a campaign; intake fans out per recipient as deduped tasks.
2. Per-user preference checks run: opted in, channel choice, weekly frequency cap.
3. Ineligible users are dropped with a logged suppression reason.
4. Eligible messages enter the bulk queue, rate-shaped below transactional traffic.
5. Channel workers send via isolated provider adapters with bulkheads.
6. Collapse keys replace older unread promos so banners never stack.
7. Unsubscribes propagate back within seconds, halting the rest of the campaign for that user.

API contracts

POST `/v1/notifications` Submit a notification event

Auth: Service API key · Content-Type: application/json · Idempotency-Key: <uuid>

```
→ {"event_id":"evt_551","user_id":"u_9021","template":"password_reset","priority":"high"}
← 202 {"notification_id":"n_31001","status":"queued"}
```

GET `/v1/notifications/{id}` Check lifecycle status

Auth: Service API key

```
→ —
← 200 {"notification_id":"n_31001","status":"delivered","attempts":1}
```

POST `/v1/users/{id}/preferences` Update channel and category preferences

Auth: User OAuth token · Content-Type: application/json

```
→ {"promos":{"push":false,"email":true},"quiet_hours":{"start":"22:00","end":"07:00"}}
← 200 {"user_id":"u_9021","updated":true}
```

POST `/v1/receipts` Provider delivery webhook (inbound)

Auth: Provider-signed (HMAC) · X-Provider-Signature: <hmac>

```
→ {"notification_id":"n_31001","event":"delivered"}
← 200 {"recorded":true}
```

How I would answer question 8

MODEL ANSWER - AS THE INTERVIEWEE

G8. Design a notification system

I'll start by narrowing the problem before drawing architecture. For Design a notification system, I would ask: Push, email, SMS, and in-app channels; Transactional versus marketing priorities; Per-user preferences and quiet hours; Delivery guarantee and latency target? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 1B notifications/day (~12K average; 20× campaign peak), 80% push, 15% email, 5% SMS. Provider rate limits and fan-out spikes dominate design. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Producer → intake → preference/policy → priority queues Then └→ channel workers → providers Then └→ receipts. The end-to-end design is: Accept idempotent events, enrich with templates and preferences, route to priority/channel queues, and isolate provider adapters. Track lifecycle state, deduplicate by event and recipient, and ingest delivery receipts asynchronously. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Backpressure: transactional alerts get reserved capacity; bulk campaigns degrade or pause. The second is Exactly-once illusion: at-least-once transport plus idempotency and user-visible dedupe is practical. The third is Preferences: keep a strongly consistent opt-out path and cache reads carefully. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "An SMS provider fails during an incident—how do you fail over?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Use AT&T mobile identity/API work to discuss secure device registration and token lifecycle. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 9 of 28

G9. Design sponsored search ads

WHY THEY ASK IT

Tests low-latency retrieval, auction and ranking, budget pacing, measurement, and adversarial integrity. It also reveals whether you can balance revenue, relevance, fairness, and user trust.

CLARIFYING QUESTIONS TO ASK

- Search text ads only?
- Auction model and ranking objective?
- Latency budget inside search response?
- Budget, targeting, and frequency-cap semantics?
- Click attribution and fraud requirements?

SCALE ESTIMATION

Assume 100K peak search QPS, 50 ms ad-service budget, millions of active campaigns, and billions of daily budget/impression updates. Candidate retrieval must reduce the auction set sharply.

HIGH-LEVEL DESIGN

```
Campaigns → policy/index → candidate shards
Query → retrieve → predict CTR/value → auction → render
Click → event log → attribution / billing / model feedback
```

Build an inverted targeting index, retrieve eligible ads, filter policy and budget, score predicted utility, then run the auction. Reserve inventory and budget with bounded staleness; stream impressions/clicks to billing, fraud, and model pipelines.

DEEP DIVES

- Budget pacing: centralized precision is slow; distributed reservations bound overspend while keeping serving available.
- Ranking: optimize long-term user value and advertiser value, not click-through rate alone; calibrate and audit models.
- Measurement: idempotent event IDs, delayed conversion windows, and fraud-resistant attribution protect billing correctness.

LIKELY FOLLOW-UPS

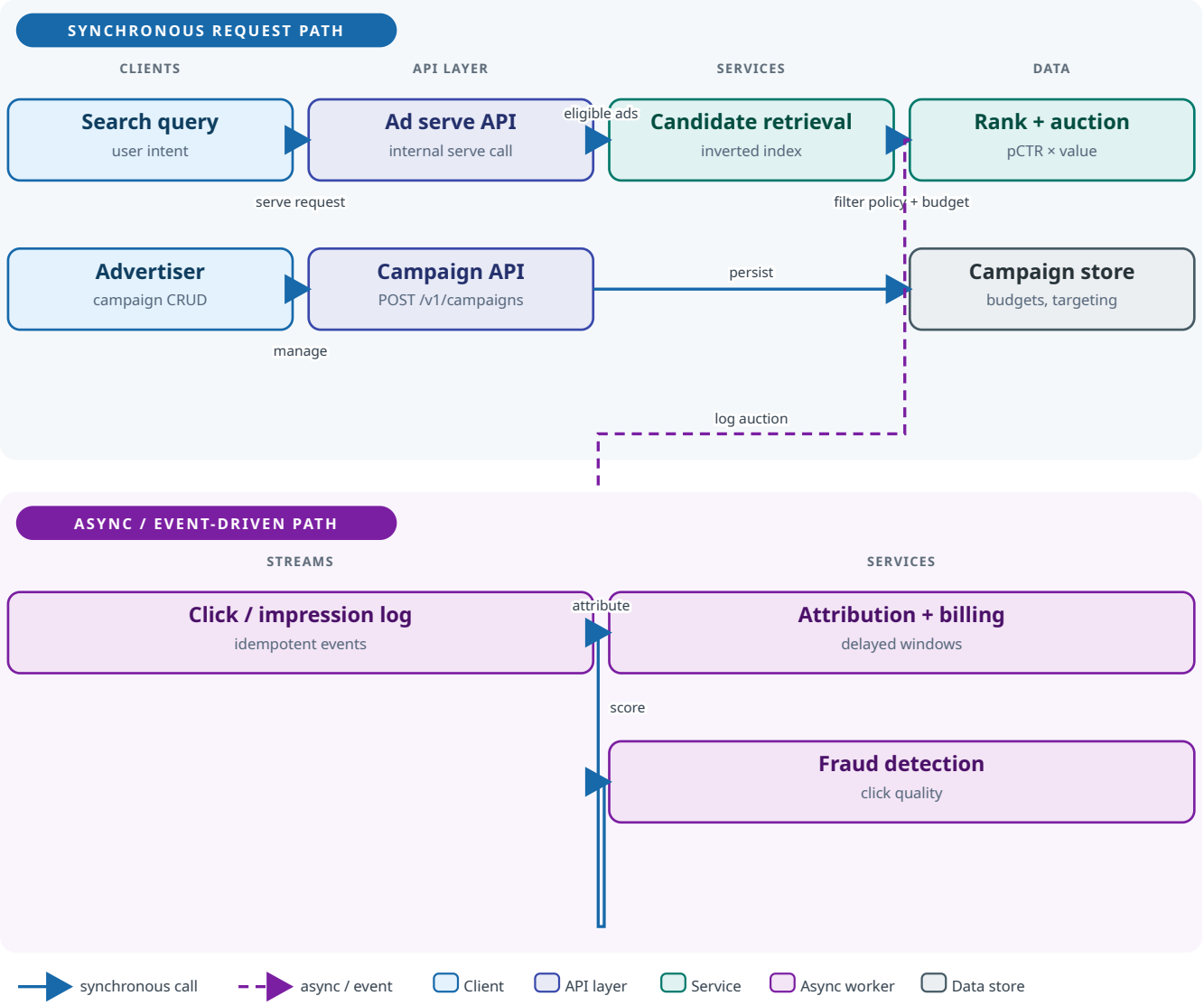
- How do you launch a new ranking model safely?
- What happens when budget data is stale?
- How do you detect click fraud?

TIE IT TO YOUR BACKGROUND

- Map model-serving governance to your $5–7M applied-AI portfolio and MLOps experience.
- Use the $500M business ownership lens to articulate revenue versus customer-trust trade-offs.
- Reference DevSecOps and zero-trust controls for campaign access, billing data, and model deployment.

Architecture diagram — question 9

Sponsored search ads — serve & auction



The serve path is milliseconds; money and models settle asynchronously.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 9

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Real-time auction per query vs fully precomputed** → chosen real-time auction over precomputed candidate sets because bids and budgets change constantly; precompute can't reflect budget pacing, but candidate pruning keeps the live auction within milliseconds.

2. **Decision: First-price vs second-price (GSP) auction** → chosen second-price because it encourages truthful bidding, which stabilizes advertiser behavior and long-term revenue instead of incentivizing bid shading.

3. **Decision: Synchronous auction in the search path vs async** → chosen synchronous with a hard deadline because ads must render with the results; async would delay the entire page, so slow bidders are simply excluded past the deadline.

4. **Decision: Approximate vs exact budget accounting** → chosen exact, strongly consistent budget ledger because overspending an advertiser's money is a financial correctness bug; approximate counters are fine for reporting but never for spend.

5. **Decision: Inline click-fraud blocking vs offline detection** → chosen offline detection with delayed invalidation because inline ML inference would blow the auction latency budget; offline models catch fraud rings and refund invalid clicks asynchronously.

End-to-end flows and API contracts — question 9

End-to-end request flows

JOURNEY 1 — USER SEARCHES; SPONSORED ADS SERVE

1. The frontend calls the internal serving path with query, user, and page context.
2. Candidate retrieval queries the inverted targeting index for keyword, geo, and audience matches.
3. Policy filters drop disapproved creatives; budget filters drop exhausted campaigns.
4. The prediction service scores pCTR and expected value from low-latency feature stores.
5. The auction ranks by bid x quality, prices second-price style, and selects winners.
6. The response returns markup with impression beacons inside the page latency budget.
7. Clicks route through a redirector that logs before landing on the advertiser page.

JOURNEY 2 — ADVERTISER MANAGES A CAMPAIGN

1. The advertiser POSTs budget, bids, targeting, and creatives.
2. Creatives enter policy review (automated plus sampled human); only approved ads serve.
3. The indexer adds targeting entries to the serving shards.
4. Budget pacing spreads spend across the day via per-campaign throttles.
5. Bid or pause edits propagate to serving shards within seconds.
6. Invalid clicks are filtered by the fraud pipeline before billing.
7. Exhausted daily budgets flip the campaign ineligible until reset; no overspend.

API contracts

GET `/internal/v1/ads/serve` Ranked ads for a query (internal)

Auth: mTLS service identity · X-Deadline-Ms: 60

```
→ ?q=running+shoes&geo=us-nc&device=mobile&slots=3
← 200 {"ads":[{"ad_id":"ad_771","price_cpc":1.42}],"auction_id":"a_99120"}
```

POST `/v1/campaigns` Create a campaign

Auth: Advertiser OAuth · Content-Type: application/json

```
→ {"name":"Fall shoes","daily_budget":500.00,"bid_cpc":1.50,"targeting":{"keywords":["running shoes"]}}
← 201 {"campaign_id":"cmp_551","status":"in_review"}
```

GET `/r/click` Click redirector: logs, then lands

Auth: Signed params

```
→ ?ad_id=ad_771&auction_id=a_99120&sig=...
← 302 Location: <advertiser page> (click logged with auction context)
```

GET `/v1/reports/performance` Advertiser performance report

Auth: Advertiser OAuth

```
→ ?campaign_id=cmp_551&from=2026-09-11&group_by=day
← 200 {"rows":[{"day":"2026-09-17","impressions":120400,"clicks":3210,"spend":4812.55}]}
```

How I would answer question 9

MODEL ANSWER - AS THE INTERVIEWEE

G9. Design sponsored search ads

I'll start by narrowing the problem before drawing architecture. For Design sponsored search ads, I would ask: Search text ads only; Auction model and ranking objective; Latency budget inside search response; Budget, targeting, and frequency-cap semantics? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 100K peak search QPS, 50 ms ad-service budget, millions of active campaigns, and billions of daily budget/impression updates. Candidate retrieval must reduce the auction set sharply. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Campaigns → policy/index → candidate shards Then Query → retrieve → predict CTR/value → auction → render Then Click → event log → attribution / billing / model feedback. The end-to-end design is: Build an inverted targeting index, retrieve eligible ads, filter policy and budget, score predicted utility, then run the auction. Reserve inventory and budget with bounded staleness; stream impressions/clicks to billing, fraud, and model pipelines. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Budget pacing: centralized precision is slow; distributed reservations bound overspend while keeping serving available. The second is Ranking: optimize long-term user value and advertiser value, not click-through rate alone; calibrate and audit models. The third is Measurement: idempotent event IDs, delayed conversion windows, and fraud-resistant attribution protect billing correctness. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "How do you launch a new ranking model safely?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Map model-serving governance to your $5–7M applied-AI portfolio and MLOps experience. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 10 of 28

G10. Design a web crawler and search-index pipeline

WHY THEY ASK IT

Tests distributed scheduling, politeness, deduplication, fault-tolerant pipelines, indexing, and freshness. The core trade-off is broad, fresh coverage without harming sites or wasting work.

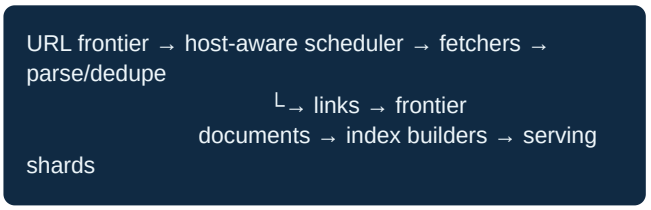
CLARIFYING QUESTIONS TO ASK

- Public web or a bounded enterprise corpus?
- Freshness objective by page class?
- Respect robots.txt and crawl-delay?
- Supported formats and languages?
- Near-real-time indexing required?

SCALE ESTIMATION

Assume 10B known URLs, 1B fetches/day (~12K average), 100 KB average response, and 10× peak. Raw fetch volume is ~100 TB/day before compression and indexes.

HIGH-LEVEL DESIGN



Partition the frontier by host, enforce per-host politeness, fetch through sandboxed workers, fingerprint content, and emit parsed documents/links to durable streams. Build immutable index segments and merge them into versioned serving shards.

DEEP DIVES

- Scheduling: prioritize freshness and importance while preventing one host from monopolizing capacity.
- Deduplication: URL canonicalization catches syntactic duplicates; content fingerprints catch mirrors but risk false positives.
- Index rollout: immutable segments make rollback easy; compaction saves space but consumes I/O.

LIKELY FOLLOW-UPS

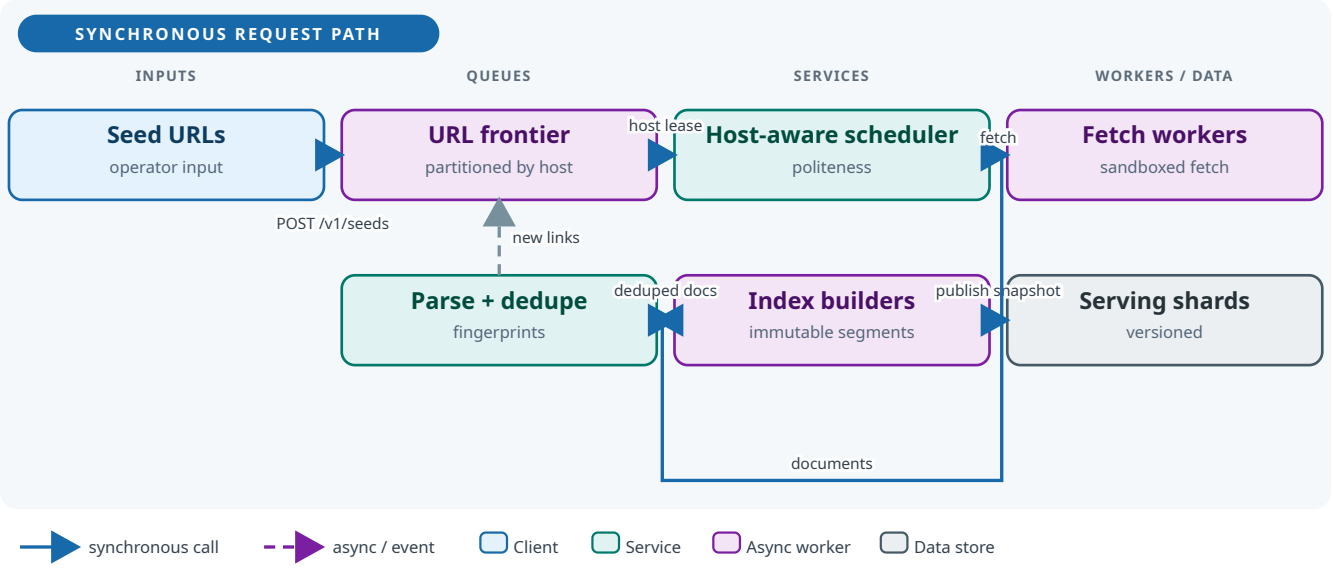
- How do you recrawl frequently changing pages?
- How do you stop crawler traps?
- What happens when parsers produce a bad index?

TIE IT TO YOUR BACKGROUND

- Use your mainframe-to-cloud and monolith-to-microservices migrations to discuss staged replacement and rollbackable pipelines.
- Connect durable queues and worker isolation to message-broker/OpenShift automation.
- Bring in OWASP and bug-bounty work for hostile content, sandboxing, and parser security.

Architecture diagram — question 10

Web crawler & index pipeline



Politeness-bounded crawl up front; immutable index segments behind serving.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 10

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: BFS frontier vs priority queue** → chosen priority queue by PageRank and change rate because BFS wastes fetches on low-value pages; priority focuses bandwidth where the index gains the most freshness per fetch.

2. **Decision: Global rate limit vs per-host politeness queues** → chosen per-host queues with politeness delays because a global limit still hammers small hosts; per-host queues enforce crawl-delay per domain and isolate slow hosts from fast ones.

3. **Decision: Fixed-schedule re-crawl vs change-driven** → chosen change-driven with adaptive intervals because fixed schedules refetch unchanged pages; adaptive intervals learn each page's change rate and skip the stable ones.

4. **Decision: Monolithic index vs term-sharded inverted index** → chosen term-sharded because a monolith can't scale; each shard owns its postings lists independently and query fan-out merges them.

5. **Decision: Coupled crawl-and-index vs decoupled pipeline** → chosen decoupled via a durable queue because fetching is I/O-bound and indexing is CPU-bound; the queue lets each scale independently and survives indexer restarts without losing crawled pages.

End-to-end flows and API contracts — question 10

End-to-end request flows

JOURNEY 1 — NEW URL DISCOVERED AND INDEXED

1. A seed or discovered link enters the frontier, partitioned by host for politeness.
2. The scheduler picks the next due URL per host, honoring crawl-delay and robots.txt.
3. A sandboxed fetcher downloads with timeouts and size caps; unsafe content is rejected early.
4. The parser extracts text and outbound links; links flow back into the frontier.
5. Content fingerprints dedupe near-identical pages; canonical URLs collapse duplicates.
6. Parsed documents emit to a durable stream; index builders write immutable segments.
7. Segments merge in the background; a new versioned snapshot publishes atomically.

JOURNEY 2 — SERVING SHARDS PICK UP THE NEW SNAPSHOT

1. The build pipeline publishes the snapshot manifest with segment locations.
2. Serving shards download new segments while still serving the old snapshot.
3. Each shard swaps to the new version via blue/green; queries never see a half-built index.
4. The old snapshot is retained briefly for instant rollback.
5. Freshness SLOs track median time-to-index per content class.
6. High change-rate hosts get shorter recrawl intervals; conditional fetches keep cost low.

API contracts

POST `/v1/seeds` Submit seed URLs

Auth: Service account · Content-Type: application/json

```
→ {"urls":["https://example.com/blog"],"priority":"normal"}
← 202 {"accepted":1,"frontier_depth":18422003}
```

POST `/internal/v1/fetch/lease` Fetcher claims next URL batch (internal)

Auth: mTLS service identity · Content-Type: application/json

```
→ {"worker_id":"f_221","capacity":50}
← 200 {"leases":[{"url":"https://example.com/a","lease_id":"lz_1"}]}
```

GET `/v1/index/versions` Published index snapshots

Auth: Service account

```
→ —
← 200 {"serving":"idx_2026-09-18T08:00Z","shards":64}
```

GET `/v1/search` Query the serving index

Auth: API key

```
→ ?q=distributed+caching+patterns&limit=10
← 200 {"results":[{"url":"https://...","score":0.93}]}
```

How I would answer question 10

MODEL ANSWER - AS THE INTERVIEWEE

G10. Design a web crawler and search-index pipeline

I'll start by narrowing the problem before drawing architecture. For Design a web crawler and search-index pipeline, I would ask: Public web or a bounded enterprise corpus; Freshness objective by page class; Respect robots.txt and crawl-delay; Supported formats and languages? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 10B known URLs, 1B fetches/day (~12K average), 100 KB average response, and 10× peak. Raw fetch volume is ~100 TB/day before compression and indexes. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: URL frontier → host-aware scheduler → fetchers → parse/dedupe Then └→ links → frontier Then documents → index builders → serving shards. The end-to-end design is: Partition the frontier by host, enforce per-host politeness, fetch through sandboxed workers, fingerprint content, and emit parsed documents/links to durable streams. Build immutable index segments and merge them into versioned serving shards. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Scheduling: prioritize freshness and importance while preventing one host from monopolizing capacity. The second is Deduplication: URL canonicalization catches syntactic duplicates; content fingerprints catch mirrors but risk false positives. The third is Index rollout: immutable segments make rollback easy; compaction saves space but consumes I/O. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "How do you recrawl frequently changing pages?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Use your mainframe-to-cloud and monolith-to-microservices migrations to discuss staged replacement and rollbackable pipelines. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 11 of 28

G11. Design an AI chatbot serving system at scale

WHY THEY ASK IT

A 2025–2026 trend prompt that tests large language model (LLM) inference, latency, cost, safety, evaluation, and model lifecycle—not just API plumbing. Directors must show where human governance sits.

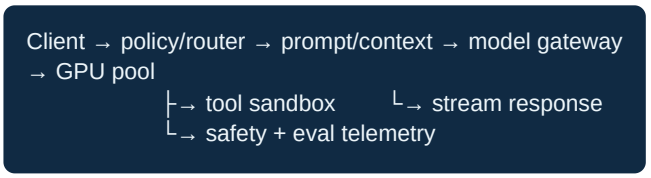
CLARIFYING QUESTIONS TO ASK

- General assistant or domain-specific copilot?
- Latency, quality, and cost target per turn?
- Streaming responses and conversation memory?
- Sensitive data and retention constraints?
- Tools/actions allowed, and which require approval?

SCALE ESTIMATION

Assume 10M daily users, 10 turns/user/day (~1.2K average; 10× peak), 2K input and 500 output tokens/turn. Token generation—not request count—is the primary capacity and cost driver.

HIGH-LEVEL DESIGN



Use a model gateway for routing, quotas, versioning, and streaming. Assemble context under a token budget, isolate tool execution, apply input/output policy checks, and capture privacy-safe traces for offline and online evaluation.

DEEP DIVES

- Inference: continuous batching improves GPU utilization but can hurt tail latency; route by model size and service tier.
- Caching: exact and semantic caches cut cost but create privacy, staleness, and correctness risks.
- Evals: combine task success, groundedness, safety, latency, and cost; use canaries and shadow traffic before promotion.

LIKELY FOLLOW-UPS

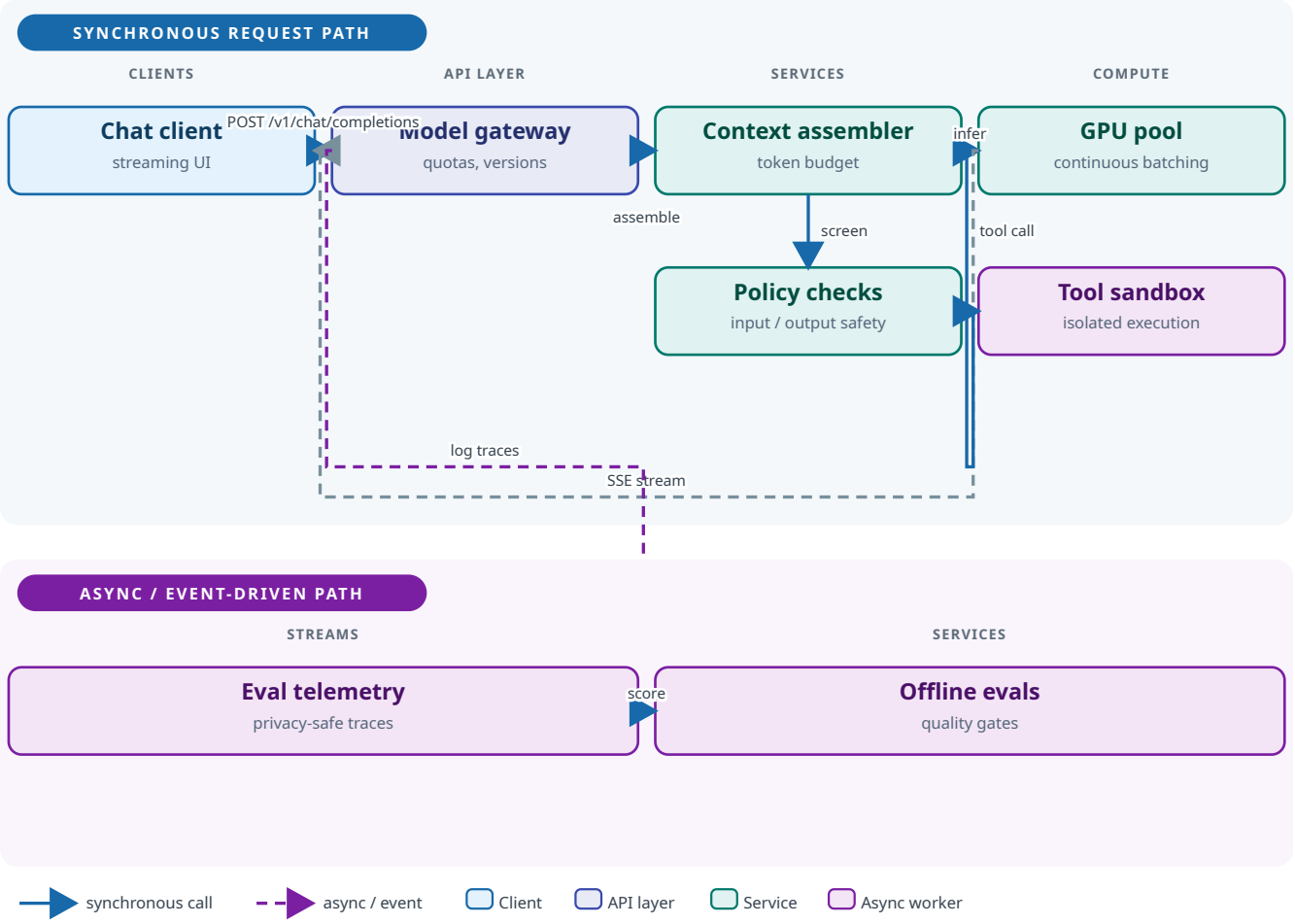
- Traffic doubles while GPU supply is fixed—what degrades?
- How do you roll back a bad model?
- How do you prevent unsafe tool execution?

TIE IT TO YOUR BACKGROUND

- Lead with your $5–7M GenAI, agentic-AI, and intelligent-automation portfolio; frame the problem as product, platform, and governance.
- Use intelligent document processing—turnaround from hours to minutes—as proof of measurable AI workflow impact.
- Connect 30% FinOps savings reinvested into AI to model routing, GPU utilization, and cost-per-successful-task.

Architecture diagram — question 11

AI chatbot serving at scale



Token generation is the scarce resource; route, batch, and gate around it.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 11

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Single giant model vs routed model cascade** → chosen cascade (small model first, escalate hard queries) because most queries are easy; the small model answers cheaply and only the hard tail pays for the large model, cutting GPU cost per successful answer.

2. **Decision: Sticky stateful sessions vs stateless with external context** → chosen stateless with an external context store because sticky sessions break load balancing and failover; external context lets any replica serve any turn of any conversation.

3. **Decision: Wait for the full response vs stream tokens** → chosen streaming because time-to-first-token dominates perceived latency; users start reading while the model is still generating, which hides most of the tail.

4. **Decision: Heavy safety filter before generation vs lightweight pre-filter plus async eval** → chosen lightweight pre-filter with async full evaluation because a heavy pre-filter taxes every request; the pre-filter blocks obvious abuse fast and the full eval runs async with retroactive action.

5. **Decision: Recompute attention vs KV-cache with prefix caching** → chosen KV-cache with prefix caching because recompute wastes FLOPs on shared system prompts; prefix caching shares the system-prompt cache across sessions and slashes per-request compute.

End-to-end flows and API contracts — question 11

End-to-end request flows

JOURNEY 1 — USER SENDS A CHAT MESSAGE (STREAMING)

1. Client POSTs to /v1/chat/completions with history, model alias, and stream=true over SSE.
2. The gateway authenticates, checks quotas and abuse signals, and pins the model version.
3. Input policy checks run: PII handling, prompt-injection screening, blocked categories.
4. The context assembler builds the prompt under a token budget, truncating oldest first.
5. The request routes to the right GPU pool; the scheduler batches it for throughput.
6. Tokens stream back as server-sent events; the client renders progressively.
7. Tool calls pause generation; the sandbox runs the tool, and results re-enter the context.

JOURNEY 2 — LONG DOCUMENT SUMMARIZATION (ASYNC)

1. Client uploads the document and starts an async job; the API returns a job id immediately.
2. The document is chunked deterministically; chunks summarize in parallel map steps.
3. A reduce step merges chunk summaries with citations to chunk offsets.
4. The client polls or receives a webhook; partial progress is visible via job status.
5. Token usage is metered per stage; cost guardrails abort runaway jobs early.
6. Human feedback attaches to the redacted trace for offline eval and data mining.
7. Identical prompts with identical context hashes skip inference via response cache.

API contracts

POST `/v1/chat/completions` Chat completion, optionally streamed

Auth: Bearer API key · Accept: text/event-stream

```
→ {"model":"assistant-2","messages":[{"role":"user","content":"Summarize..."}],"stream":true}
← 200 (SSE) data: {"choices":[{"delta":{"content":"Here is"}}]} ... data: [DONE]
```

POST `/v1/jobs/summarize` Start async summarization job

Auth: Bearer API key · Content-Type: application/json

```
→ {"document_id":"doc_881","webhook":"https://client.example.com/hook"}
← 202 {"job_id":"job_551","status":"queued"}
```

GET `/v1/jobs/{job_id}` Poll async job status

Auth: Bearer API key

```
→ —
← 200 {"job_id":"job_551","status":"succeeded","result":{"summary":"..."}}
```

GET `/v1/models` List model aliases and versions

Auth: Bearer API key

```
→ —
← 200 {"models":[{"alias":"assistant-2","pinned":"asst-2.14","context_window":128000}]}
```

How I would answer question 11

MODEL ANSWER - AS THE INTERVIEWEE

G11. Design an AI chatbot serving system at scale

I'll start by narrowing the problem before drawing architecture. For Design an AI chatbot serving system at scale, I would ask: General assistant or domain-specific copilot; Latency, quality, and cost target per turn; Streaming responses and conversation memory; Sensitive data and retention constraints? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 10M daily users, 10 turns/user/day (~1.2K average; 10× peak), 2K input and 500 output tokens/turn. Token generation—not request count—is the primary capacity and cost driver. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Client → policy/router → prompt/context → model gateway → GPU pool Then ├→ tool sandbox └→ stream response Then └→ safety + eval telemetry. The end-to-end design is: Use a model gateway for routing, quotas, versioning, and streaming. Assemble context under a token budget, isolate tool execution, apply input/output policy checks, and capture privacy-safe traces for offline and online evaluation. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Inference: continuous batching improves GPU utilization but can hurt tail latency; route by model size and service tier. The second is Caching: exact and semantic caches cut cost but create privacy, staleness, and correctness risks. The third is Evals: combine task success, groundedness, safety, latency, and cost; use canaries and shadow traffic before promotion. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "Traffic doubles while GPU supply is fixed—what degrades?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Lead with your $5–7M GenAI, agentic-AI, and intelligent-automation portfolio; frame the problem as product, platform, and governance. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 12 of 28

G12. Design a RAG pipeline for enterprise search

WHY THEY ASK IT

Another 2025–2026 trend prompt. Retrieval-augmented generation (RAG) tests ingestion, permissions, chunking, hybrid retrieval, quality evaluation, freshness, and grounded answers.

CLARIFYING QUESTIONS TO ASK

- Which repositories and document types?
- Permission model and tenant isolation?
- Freshness target after a source update?
- Citations required for every claim?
- Languages, query volume, and quality metrics?

SCALE ESTIMATION

Assume 100M documents, 10 chunks/document, 1B vectors, 1M daily queries, and daily updates to 1% of content. Index build throughput and filtered retrieval are first-class requirements.

HIGH-LEVEL DESIGN

Sources → parse/chunk → ACL enrich → embed → vector + text indexes
Query → auth → hybrid retrieve → rerank → LLM → cited answer

Use connector-specific checkpoints, deterministic chunk IDs, content hashing, and deletion propagation. Enforce access control before retrieval, combine lexical and vector candidates, rerank, then generate only from retrieved evidence with citations.

DEEP DIVES

- Chunking: structural chunks preserve meaning; fixed windows simplify indexing. Test by task, not intuition.
- Security: ACL-aware indexes reduce leakage risk; post-filtering alone can expose scores or exhaust candidate sets.
- Quality: evaluate retrieval recall separately from answer groundedness to locate failures.

LIKELY FOLLOW-UPS

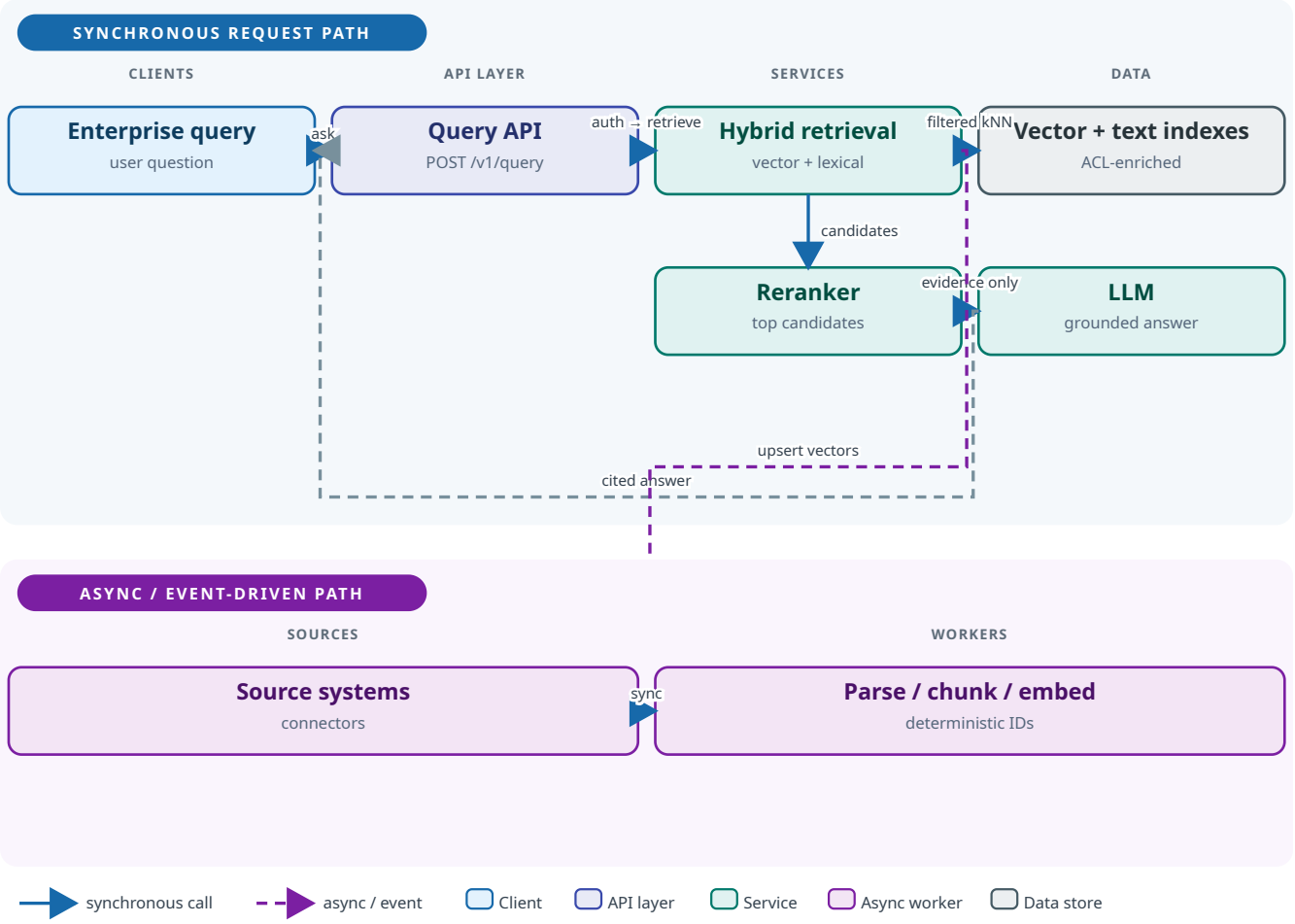
- A document's access is revoked—how fast is removal?
- How do you detect stale embeddings?
- How do you evaluate hallucinations at scale?

TIE IT TO YOUR BACKGROUND

- Use your intelligent-document-processing program as the primary story: hours-to-minutes turnaround and real workflow adoption.
- Name-drop GenAI/LLM orchestration, agentic AI, MLOps, and zero-trust only where they map to the pipeline.
- Relate seven-system integration to connector governance, schema variation, and reliable source-of-truth boundaries.

Architecture diagram — question 12

Enterprise RAG pipeline



Enforce ACLs before retrieval; generate only from retrieved evidence.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 12

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Keyword index only vs hybrid vector plus BM25** → chosen hybrid because pure vector misses exact terms (IDs, error codes) and pure keyword misses semantics; the hybrid covers both and a reranker merges the candidate lists.

2. **Decision: Synchronous re-index on document change vs async versioned re-index** → chosen async with index versions because re-embedding on the write path delays every save; versions keep queries consistent while the new index builds in the background.

3. **Decision: Long-context stuffing vs retrieve-then-read with reranking** → chosen retrieve-then-read because stuffing whole corpora blows context windows and token cost; top-k reranked chunks fit the window and carry citations the answer can quote.

4. **Decision: Per-tenant index vs shared index with tenant filters** → chosen shared index with metadata tenant filters plus post-retrieval ACL enforcement because per-tenant indexes fragment memory and compute; filters isolate tenants while the ANN structure stays shared and efficient.

5. **Decision: SQL for embeddings vs dedicated vector store** → chosen dedicated ANN vector store because SQL cannot do approximate nearest-neighbor search at scale; metadata stays in SQL and joins to vectors by document ID.

End-to-end flows and API contracts — question 12

End-to-end request flows

JOURNEY 1 — ENTERPRISE ASKS A QUESTION OVER ITS DOCUMENTS

1. Client POSTs the question; the API authenticates the user and resolves group memberships.
2. The planner issues hybrid retrieval: vector similarity plus lexical (BM25) candidates.
3. Access-control filtering runs BEFORE ranking; unreadable chunks are removed entirely.
4. The merger dedupes overlapping chunks; the reranker scores the top candidates.
5. The LLM answers strictly from the permitted chunks, with inline citations per claim.
6. Uncovered questions get an explicit refusal, not a hallucinated answer.
7. The trace (query, chunk ids, model version) is logged for audit; content stays secured.

JOURNEY 2 — A DOCUMENT IS UPDATED, THEN DELETED

1. The connector detects the change via its checkpoint and pushes the new version.
2. The pipeline re-chunks deterministically and re-embeds only changed chunks.
3. ACL enrichment re-resolves permissions before the new version becomes searchable.
4. Indexes upsert new chunks; old chunks are marked superseded for in-flight queries.
5. On delete, a tombstone propagates: chunks leave both indexes and the embedding store.
6. A verification job confirms the document is unretrievable and reports compliance evidence.
7. Stale checkpoints alert if a connector stops syncing, bounding index staleness.

API contracts

POST `/v1/query` Ask over permitted documents

Auth: User OAuth 2.0 · Content-Type: application/json

```
→ {"question":"What is our PTO carryover policy?","top_k":8}
← 200 {"answer":"Employees may carry over up to 40 hours [1].","citations":[{"id":"c_881"}]}
```

POST `/v1/connectors/{id}/sync` Trigger a connector sync

Auth: Service account · Content-Type: application/json

```
→ {"full":false}
← 202 {"sync_id":"sy_221","status":"running"}
```

DELETE `/v1/documents/{id}` Delete document; propagate tombstone

Auth: Service account

```
→ —
← 202 {"doc_id":"d_551","tombstone":"ts_77"}
```

GET `/v1/eval/citation-quality` Answer quality metrics (ops)

Auth: Admin token

```
→ ?from=2026-09-11
← 200 {"citation_precision":0.94,"p95_latency_ms":1810}
```

How I would answer question 12

MODEL ANSWER - AS THE INTERVIEWEE

G12. Design a RAG pipeline for enterprise search

I'll start by narrowing the problem before drawing architecture. For Design a RAG pipeline for enterprise search, I would ask: Which repositories and document types; Permission model and tenant isolation; Freshness target after a source update; Citations required for every claim? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 100M documents, 10 chunks/document, 1B vectors, 1M daily queries, and daily updates to 1% of content. Index build throughput and filtered retrieval are first-class requirements. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Sources → parse/chunk → ACL enrich → embed → vector + text indexes Then Query → auth → hybrid retrieve → rerank → LLM → cited answer. The end-to-end design is: Use connector-specific checkpoints, deterministic chunk IDs, content hashing, and deletion propagation. Enforce access control before retrieval, combine lexical and vector candidates, rerank, then generate only from retrieved evidence with citations. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Chunking: structural chunks preserve meaning; fixed windows simplify indexing. Test by task, not intuition. The second is Security: ACL-aware indexes reduce leakage risk; post-filtering alone can expose scores or exhaust candidate sets. The third is Quality: evaluate retrieval recall separately from answer groundedness to locate failures. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "A document's access is revoked—how fast is removal?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Use your intelligent-document-processing program as the primary story: hours-to-minutes turnaround and real workflow adoption. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 13 of 28

A1. Design iCloud Photos multi-device sync

WHY THEY ASK IT

Tests offline-first behavior, conflict resolution, storage pressure, media transfer, privacy, and user trust. The hard part is preserving intent when several devices change state independently.

CLARIFYING QUESTIONS TO ASK

- Which edits sync: originals, metadata, albums, deletions?
- How many devices per account?
- Must edits work fully offline?
- Storage-optimization behavior on each device?
- Shared libraries and end-to-end encryption scope?

SCALE ESTIMATION

Assume 1B accounts, 50K uploads/s at peak, 8 MB average media, and 5 devices/account. Metadata operations are tiny but correctness-sensitive; media egress and duplicate storage dominate cost.

HIGH-LEVEL DESIGN

```
Device journal → sync API → change log → metadata store
    └→ chunked media → object store/CDN
Other devices ← cursor / push hint ← change log
```

Record local changes in a durable journal, upload media in resumable encrypted chunks, and sync metadata through per-library ordered change logs. Use stable asset IDs, content hashes, tombstones, and device cursors. Version vectors detect concurrent edits; product policy determines merge or preservation.

DEEP DIVES

- Conflicts: field-level merge for metadata, preserve both for destructive media edits, and never silently lose an original.
- Storage/power: keep thumbnails and metadata locally, evict full-resolution files under pressure, and batch work in favorable network/power conditions.
- Privacy: encrypt transport and storage; minimize server-visible metadata and keep sensitive analysis on-device where possible.

LIKELY FOLLOW-UPS

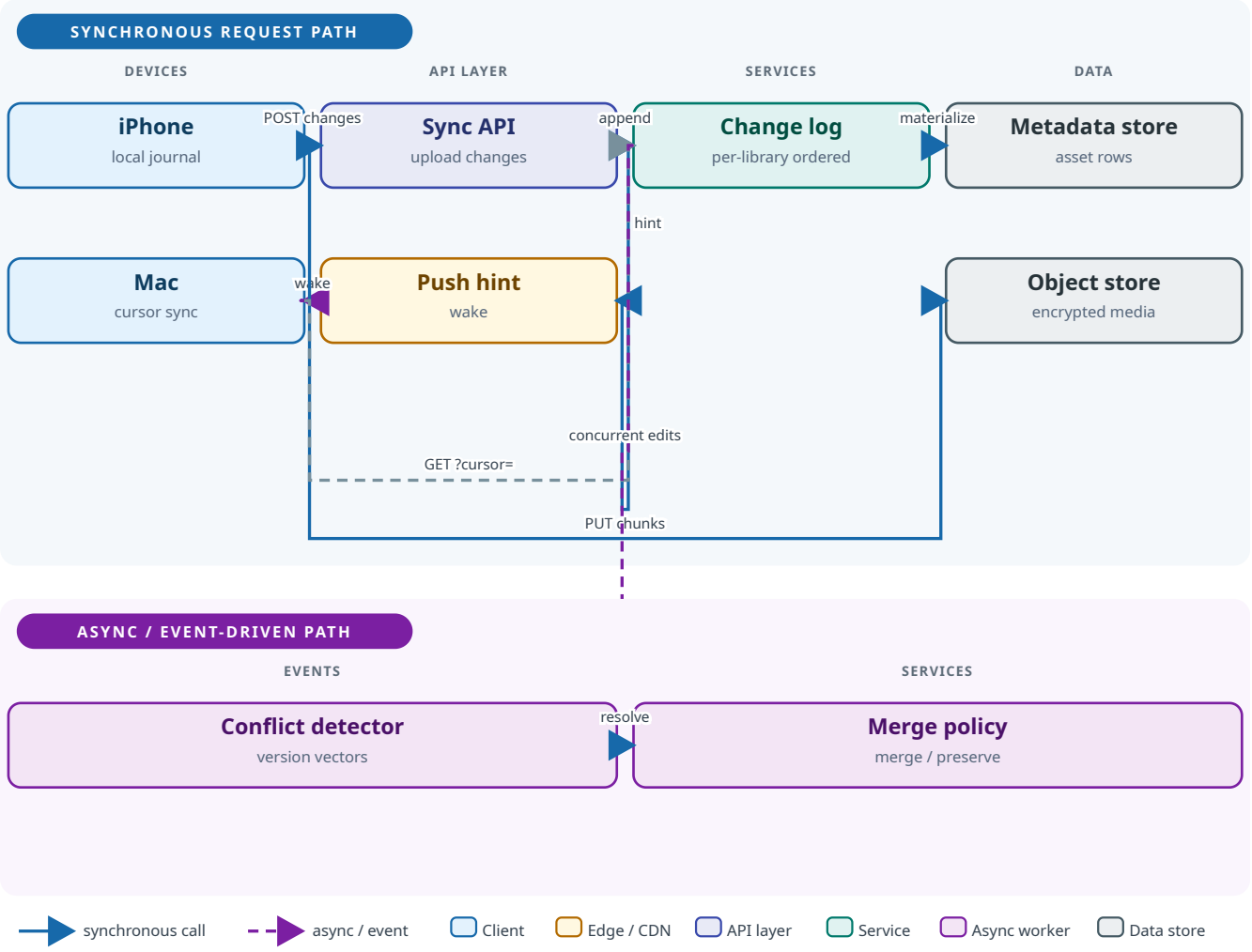
- Two devices edit the same photo offline—what happens?
- A phone is nearly full and enters Low Power Mode—what pauses?
- How does permanent deletion propagate safely?

TIE IT TO YOUR BACKGROUND

- Draw an honest parallel to consistency and replay in your event-driven Hartford platforms; explicitly say this is not claimed iCloud-scale ownership.
- Use AWS migration experience to discuss staged cutover, idempotent transfers, and rollback.
- Bring in zero-trust and mobile-security architecture for device identity, authorization, and encrypted sessions.

Architecture diagram — question 13

iCloud Photos multi-device sync



Local journal first; a per-library ordered log fans out to every device.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 13

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Last-writer-wins vs version vectors with a merge policy** → chosen version vectors with a deterministic merge policy because last-writer-wins silently drops edits made on another device; vectors detect true conflicts and the policy preserves both sides instead of losing data.

2. **Decision: Re-upload full files vs chunked content-defined deltas** → chosen chunked deltas because photo edits are usually small; re-uploading full originals wastes bandwidth and battery on metered connections.

3. **Decision: Push every change vs pull with cursor** → chosen opaque push hint plus cursor pull because always-on connections drain battery; a lightweight hint wakes the device to pull exactly what changed, nothing more.

4. **Decision: Device journal as truth vs server-side ordered change log** → chosen per-library ordered change log on the server because multi-device sync needs one total order; the log is the arbiter and every device replays it, with device journals as input only.

5. **Decision: Strong consistency vs eventual consistency across devices** → chosen eventual with monotonic-read sessions because strong consistency would block offline edits; users tolerate seconds of lag but not failed saves, and CAP says a partition must not freeze the camera roll.

End-to-end flows and API contracts — question 13

End-to-end request flows

JOURNEY 1 — PHOTO TAKEN ON IPHONE APPEARS ON MAC

1. The camera writes the photo locally and appends a journal entry with asset id and hash.
2. The sync engine uploads media in resumable encrypted chunks, content-addressed for idempotent retry.
3. The device POSTs a metadata commit: asset id, hash, dimensions, journal sequence.
4. The sync API appends to the per-library ordered change log, the source of truth for changes.
5. The Mac long-polls GET /sync/changes with its cursor and downloads new chunks via CDN.
6. It applies metadata changes in log order; version vectors detect concurrent edits.
7. Concurrent edits merge per product policy (non-destructive merge or keep both).

JOURNEY 2 — USER DELETES A PHOTO ON IPAD

1. The iPad writes a tombstone with server-authoritative expiry into its journal and syncs it.
2. Other devices receive the tombstone via the change log and hide the asset immediately.
3. Encrypted media moves to the retention tier; it is not physically deleted until expiry.
4. Restore within the window writes a newer state transition that resurrects the asset.
5. After expiry, keys are destroyed and the lifecycle scanner reclaims storage.
6. Offline devices reconcile on next sync via cursors; old tombstones compact away.

API contracts

POST `/v1/libraries/{id}/changes` Upload a batch of journal changes

Auth: iCloud token (device) · Content-Type: application/json

```
→ {"base_cursor":"84210","changes":[{"op":"add","asset_id":"as_991","sha256":"e3b0..."}]}
← 200 {"accepted":1,"new_cursor":"84211","conflicts":[]}
```

PUT `/v1/assets/{id}/chunks/{n}` Upload one encrypted chunk (resumable)

Auth: iCloud token (device) · Content-Range: bytes 0-4194303/12582912

```
→ <encrypted chunk bytes>
← 200 {"asset_id":"as_991","chunk":0,"received":3,"total":3}
```

GET `/v1/libraries/{id}/changes` Pull changes after cursor

Auth: iCloud token (device)

```
→ ?cursor=84210&limit=200
← 200 {"changes":[{"seq":84211,"op":"add","asset_id":"as_991"}],"next_cursor":"84211"}
```

POST `/v1/assets/{id}/tombstone` Record a deletion tombstone

Auth: iCloud token (device) · Content-Type: application/json

```
→ {"asset_id":"as_991"}
← 200 {"asset_id":"as_991","state":"tombstoned","expires_at":"2026-10-18T10:00:00Z"}
```

How I would answer question 13

MODEL ANSWER - AS THE INTERVIEWEE

A1. Design iCloud Photos multi-device sync

I'll start by narrowing the problem before drawing architecture. For Design iCloud Photos multi-device sync, I would ask: Which edits sync: originals, metadata, albums, deletions; How many devices per account; Must edits work fully offline; Storage-optimization behavior on each device? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 1B accounts, 50K uploads/s at peak, 8 MB average media, and 5 devices/account. Metadata operations are tiny but correctness-sensitive; media egress and duplicate storage dominate cost. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Device journal → sync API → change log → metadata store Then └→ chunked media → object store/CDN Then Other devices ← cursor / push hint ← change log. The end-to-end design is: Record local changes in a durable journal, upload media in resumable encrypted chunks, and sync metadata through per-library ordered change logs. Use stable asset IDs, content hashes, tombstones, and device cursors. Version vectors detect concurrent edits; product policy determines merge or preservation. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Conflicts: field-level merge for metadata, preserve both for destructive media edits, and never silently lose an original. The second is Storage/power: keep thumbnails and metadata locally, evict full-resolution files under pressure, and batch work in favorable network/power conditions. The third is Privacy: encrypt transport and storage; minimize server-visible metadata and keep sensitive analysis on-device where possible. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "Two devices edit the same photo offline—what happens?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Draw an honest parallel to consistency and replay in your event-driven Hartford platforms; explicitly say this is not claimed iCloud-scale ownership. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 14 of 28

A2. Design a Notes sync system for iCloud

WHY THEY ASK IT

Tests small but correctness-critical data, offline editing, multi-device convergence, collaboration, and clear conflict semantics. Simplicity and data-loss prevention are stronger signals than architectural ornament.

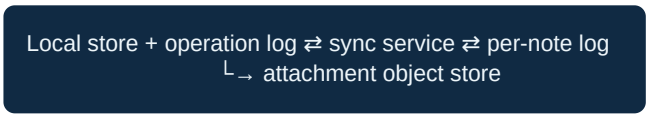
CLARIFYING QUESTIONS TO ASK

- Plain text, rich text, attachments, and shared notes?
- Concurrent character-level collaboration required?
- Expected offline duration?
- Deletion and recovery semantics?
- End-to-end encrypted notes?

SCALE ESTIMATION

Assume 500M users, 100 notes/user, 10 edits/day, and 5× peak. Text deltas are small; attachments and sync fan-out drive bandwidth.

HIGH-LEVEL DESIGN



Persist edits locally first, assign idempotent operation IDs, and sync through a per-note operation log. Use revision vectors for causal ordering. For solo notes, field/block merges may suffice; for collaboration, use operational transformation or conflict-free replicated data types after defining UX needs.

DEEP DIVES

- Conflict model: CRDTs converge but add metadata/complexity; operation transformation needs coordination; duplicate preservation is safest for rare hard conflicts.
- Offline lifecycle: retries are exponential and resumable; user sees sync state without being blocked.
- Storage/power/privacy: delta sync, attachment dedupe, background scheduling, encrypted local data, and minimal server metadata.

LIKELY FOLLOW-UPS

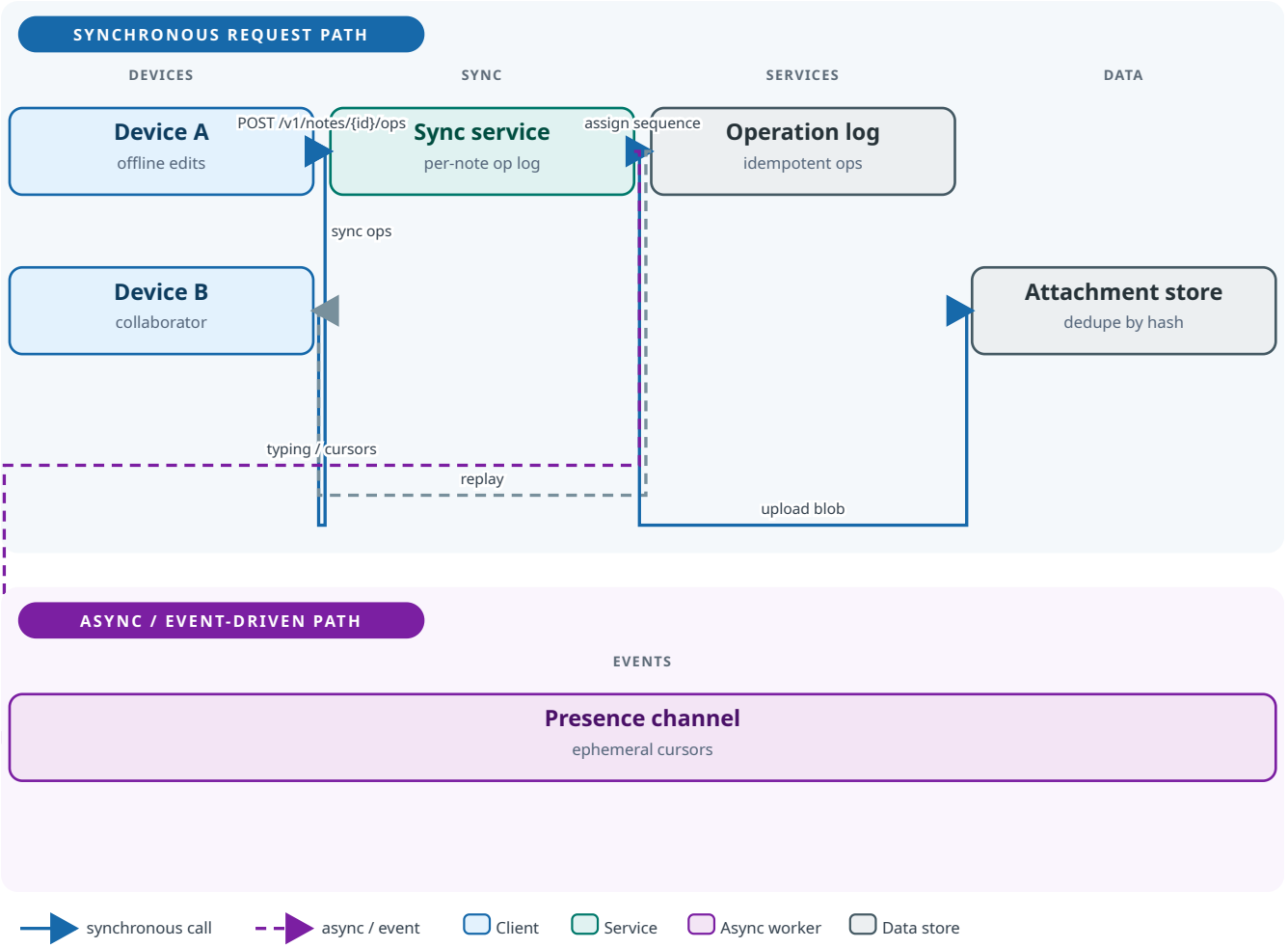
- A device reconnects after six months—replay or snapshot?
- How do you recover a deleted note?
- How does shared-note revocation affect cached copies?

TIE IT TO YOUR BACKGROUND

- Relate operation logs and idempotency to your message-broker and event-driven automation experience.
- Use 99% manual-work elimination to show how reliability and observability turn an automation into a trusted workflow.
- Be explicit that mobile offline UX is a transfer of distributed-systems principles, not prior Notes product experience.

Architecture diagram — question 14

Notes sync — offline-first convergence



Operations are the source of truth; snapshots are just compaction.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 14

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Last-writer-wins vs CRDTs vs operational transform** → chosen CRDTs because notes need offline concurrent edits across devices; last-writer-wins loses keystrokes and OT needs a central ordering server, while CRDTs merge deterministically on the device.

2. **Decision: Sync whole notes vs field-level deltas** → chosen field-level deltas because edits are far smaller than notes; deltas keep sync fast on poor networks and make conflicts finer-grained.

3. **Decision: One socket per note vs single multiplexed sync connection** → chosen single multiplexed connection because per-note sockets multiply radio wakeups and battery drain; one connection carries all entity streams with per-stream cursors.

4. **Decision: Server-resolved vs client-resolved conflicts** → chosen client-resolved against the server log because the server cannot know user intent; clients apply the deterministic merge and surface genuine conflicts in the UI.

5. **Decision: Hard delete vs tombstones with garbage collection** → chosen tombstones with GC because hard deletes resurrect when a stale device syncs; tombstones propagate the deletion everywhere and GC reclaims storage after all devices acknowledge.

End-to-end flows and API contracts — question 14

End-to-end request flows

JOURNEY 1 — USER EDITS A NOTE OFFLINE, THEN RECONNECTS

1. Keystrokes persist to the local store first and append ops to the local op log.
2. The note stays fully usable offline with an honest "waiting to sync" indicator.
3. On reconnect, the client POSTs pending ops; idempotency ids make retries safe.
4. The service appends ops to the per-note authoritative log and returns the new revision.
5. The client GETs missed ops since its revision and applies them locally.
6. Revision vectors detect causality; solo-note merges are field-level and lossless.
7. Attachments upload as separate blobs referenced by content hash, never blocking text sync.

JOURNEY 2 — TWO COLLABORATORS EDIT THE SAME NOTE

1. Both clients hold the note over the sync channel; keystrokes become ops with revision vectors.
2. The service sequences ops per note and broadcasts to the other collaborator.
3. Concurrent ops merge via OT/CRDT policy; both devices converge with no user action.
4. Presence ("Maya is editing paragraph 2") travels on the ephemeral channel only.
5. A disconnecting client queues ops locally; they merge in order on reconnect.
6. Permission revocation closes the socket immediately; new ops from that user are rejected.

API contracts

POST `/v1/notes` Create a note

Auth: iCloud token (device) · Content-Type: application/json

```
→ {"title":"Trip ideas","folder":"travel"}
← 201 {"note_id":"n_771","revision":0}
```

POST `/v1/notes/{id}/ops` Append operations

Auth: iCloud token (device) · Content-Type: application/json

```
→ {"base_revision":14,"ops":[{"op_id":"op_a91","type":"insert","pos":42,"text":"Book flights"}]}
← 200 {"accepted":1,"server_revision":16}
```

GET `/v1/notes/{id}/ops` Fetch ops since a revision

Auth: iCloud token (device)

```
→ ?since=14&limit=500
← 200 {"ops":[{"op_id":"op_b02","type":"insert","pos":10}],"server_revision":16}
```

GET `/v1/notes/{id}/snapshot` Compacted snapshot for bootstrap

Auth: iCloud token (device)

```
→ —
← 200 {"note_id":"n_771","revision":1200,"snapshot_at":"..."}
```

How I would answer question 14

MODEL ANSWER - AS THE INTERVIEWEE

A2. Design a Notes sync system for iCloud

I'll start by narrowing the problem before drawing architecture. For Design a Notes sync system for iCloud, I would ask: Plain text, rich text, attachments, and shared notes; Concurrent character-level collaboration required; Expected offline duration; Deletion and recovery semantics? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 500M users, 100 notes/user, 10 edits/day, and 5× peak. Text deltas are small; attachments and sync fan-out drive bandwidth. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Local store + operation log ⇄ sync service ⇄ per-note log Then └→ attachment object store. The end-to-end design is: Persist edits locally first, assign idempotent operation IDs, and sync through a per-note operation log. Use revision vectors for causal ordering. For solo notes, field/block merges may suffice; for collaboration, use operational transformation or conflict-free replicated data types after defining UX needs. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Conflict model: CRDTs converge but add metadata/complexity; operation transformation needs coordination; duplicate preservation is safest for rare hard conflicts. The second is Offline lifecycle: retries are exponential and resumable; user sees sync state without being blocked. The third is Storage/power/privacy: delta sync, attachment dedupe, background scheduling, encrypted local data, and minimal server metadata. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "A device reconnects after six months—replay or snapshot?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Relate operation logs and idempotency to your message-broker and event-driven automation experience. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 15 of 28

A3. Design FaceTime

WHY THEY ASK IT

Tests real-time media, NAT traversal, adaptive bitrate, call signaling, device handoff, privacy, and graceful degradation. A strong answer protects conversational quality rather than chasing raw resolution.

CLARIFYING QUESTIONS TO ASK

- One-to-one first, or group calls?
- Audio/video quality and join-time targets?
- Cross-device handoff required?
- End-to-end encryption expectations?
- Behavior on weak networks and low battery?

SCALE ESTIMATION

Assume 10M concurrent one-to-one calls at peak. At 1.5 Mbps per participant, relaying all media would exceed 30 Tbps; peer-to-peer paths are preferred where feasible, with relay capacity for fallback.

HIGH-LEVEL DESIGN

```
Caller → signaling / push → callee
Media: peer ⇄ STUN/TURN ⇄ peer
          └→ relay fallback + quality telemetry
```

Separate signaling from encrypted media. Use ICE with STUN/TURN for NAT traversal, short-lived credentials, adaptive codecs, congestion control, and regionally close relays. Push wakes an offline callee; handoff establishes a new encrypted session before retiring the old one.

DEEP DIVES

- Media path: peer-to-peer minimizes cost/latency; relays improve reachability and privacy consistency at significant bandwidth cost.
- Adaptation: reduce resolution/frame rate before audio quality; respect thermal and Low Power Mode constraints.
- Privacy: minimize call metadata, rotate identifiers, encrypt media end to end, and keep diagnostic logs privacy-safe.

LIKELY FOLLOW-UPS

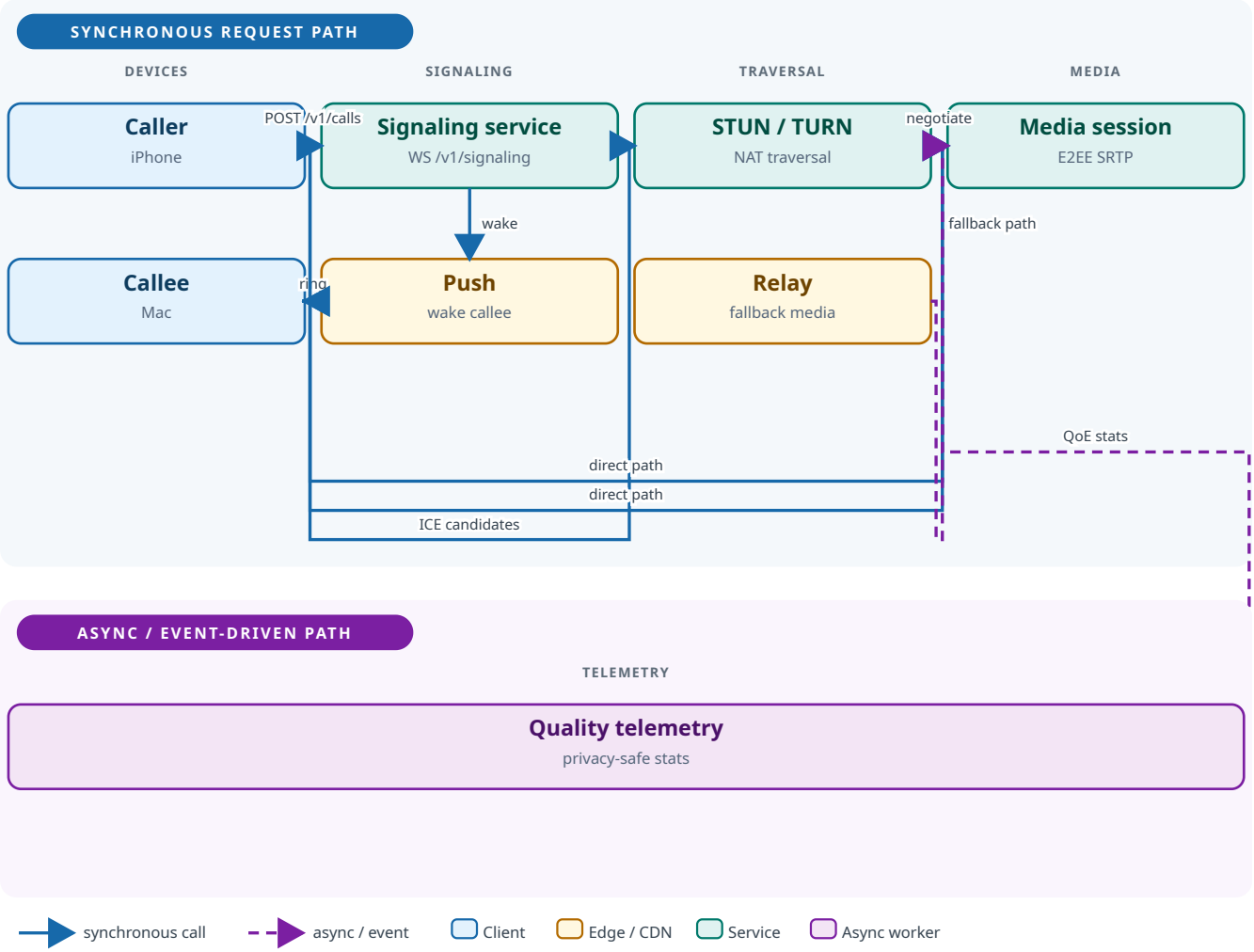
- How do group calls change topology?
- What if network type changes mid-call?
- How do you debug quality without inspecting content?

TIE IT TO YOUR BACKGROUND

- Use AT&T native mobile authentication work to establish credibility on secure session bootstrap, 2FA, and step-up auth.
- Connect low-latency API design to signaling budgets; clearly separate it from hands-on media-stack claims.
- Apply FinOps experience to the peer-to-peer versus relay cost decision.

Architecture diagram — question 15

FaceTime — signaling & media planes



Signaling sets up the call; media stays peer-to-peer whenever possible.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 15

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: P2P mesh vs SFU vs MCU** → chosen SFU because a P2P mesh explodes uplink bandwidth beyond two participants and an MCU re-encode adds latency and cost; the SFU forwards packets without transcoding, scaling to group calls cheaply.

2. **Decision: TCP vs UDP for media** → chosen UDP with NACK and FEC because TCP retransmission delays break real-time interactivity; NACK/FEC recover from loss inside the latency budget instead of stalling the stream.

3. **Decision: Fixed bitrate vs adaptive congestion control** → chosen adaptive via transport-wide congestion control because fixed bitrate either stalls on bad networks or wastes good ones; the sender continuously tracks available bandwidth and switches simulcast layers.

4. **Decision: Hop-by-hop vs end-to-end encryption** → chosen end-to-end (media encrypted before the SFU) because the SFU must route but never see plaintext; per-participant keys keep a compromised relay from decrypting calls.

5. **Decision: Single encode vs simulcast** → chosen simulcast for group calls because one encode forces everyone to the lowest common quality; simulcast lets the SFU forward the right layer to each receiver based on their downlink.

End-to-end flows and API contracts — question 15

End-to-end request flows

JOURNEY 1 — CALLER STARTS A FACETIME CALL

1. Caller POSTs to /v1/calls; signaling creates a call record and short-lived call token.
2. Push wakes the callee with an opaque hint; the callee fetches details over the signaling socket.
3. Both clients gather ICE candidates (host, STUN reflexive, TURN relay) via signaling.
4. Media negotiates SRTP with per-call keys; the best path is peer-to-peer when NAT allows.
5. If direct fails, both fall back to the regional TURN relay, which forwards undecryptable packets.
6. Congestion control and adaptive codecs adjust quality; telemetry streams to the quality service.
7. Either side POSTs /calls/{id}/end; relay allocations are released immediately.

JOURNEY 2 — HANDOFF FROM IPHONE TO MAC MID-CALL

1. The iPhone calls POST /v1/calls/{id}/handoff for a transfer token bound to the user's devices.
2. The signed-in Mac redeems the token and joins the signaling session as a new endpoint.
3. The Mac negotiates ICE and establishes a fresh encrypted media session with the peer.
4. Only after the Mac's media is confirmed does the iPhone relinquish its media path.
5. If the Mac fails within the timeout, the iPhone keeps the call: make-before-break.
6. Telemetry continues across the transition, tagged per device for attribution.

API contracts

POST `/v1/calls` Initiate a call (signaling)

Auth: Device identity token · Content-Type: application/json

```
→ {"callee":"+14155550132","media":["audio","video"]}
← 201 {"call_id":"c_88121","call_token":"ct_...","state":"ringing"}
```

WS `/v1/signaling` SDP/ICE exchange channel

Auth: Call token (subprotocol) · Sec-WebSocket-Protocol: facetime.sig.v1

```
→ → {"type":"ice","call_id":"c_88121","candidate":"candidate:842..."}
← ← {"type":"answer","call_id":"c_88121","sdp":"v=0..."}
```

POST `/v1/turn/credentials` Mint short-lived TURN credentials

Auth: Device identity token · Content-Type: application/json

```
→ {"call_id":"c_88121"}
← 200 {"username":"u_1694:devA","password":"...","ttl_s":3600}
```

POST `/v1/calls/{id}/handoff` Request device handoff token

Auth: Device identity token · Content-Type: application/json

```
→ {"target_device":"mac-4f2a"}
← 200 {"handoff_token":"ht_..."}
```

How I would answer question 15

MODEL ANSWER - AS THE INTERVIEWEE

A3. Design FaceTime

I'll start by narrowing the problem before drawing architecture. For Design FaceTime, I would ask: One-to-one first, or group calls; Audio/video quality and join-time targets; Cross-device handoff required; End-to-end encryption expectations? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 10M concurrent one-to-one calls at peak. At 1.5 Mbps per participant, relaying all media would exceed 30 Tbps; peer-to-peer paths are preferred where feasible, with relay capacity for fallback. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Caller → signaling / push → callee Then Media: peer ⇄ STUN/TURN ⇄ peer Then └→ relay fallback + quality telemetry. The end-to-end design is: Separate signaling from encrypted media. Use ICE with STUN/TURN for NAT traversal, short-lived credentials, adaptive codecs, congestion control, and regionally close relays. Push wakes an offline callee; handoff establishes a new encrypted session before retiring the old one. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Media path: peer-to-peer minimizes cost/latency; relays improve reachability and privacy consistency at significant bandwidth cost. The second is Adaptation: reduce resolution/frame rate before audio quality; respect thermal and Low Power Mode constraints. The third is Privacy: minimize call metadata, rotate identifiers, encrypt media end to end, and keep diagnostic logs privacy-safe. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "How do group calls change topology?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Use AT&T native mobile authentication work to establish credibility on secure session bootstrap, 2FA, and step-up auth. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 16 of 28

A4. Design Apple Push Notification service (APNs) at scale

WHY THEY ASK IT

Tests persistent device connections, routing, fan-out, priority, token lifecycle, privacy, and delivery semantics under mobile constraints. The service must be fast without promising impossible exactly-once delivery.

CLARIFYING QUESTIONS TO ASK

- Transactional and background notifications?
- Target delivery latency and offline retention?
- Topic fan-out and collapse semantics?
- Per-app quotas and priority levels?
- Payload privacy and device-token lifecycle?

SCALE ESTIMATION

Assume 2B devices, 500M concurrent connections, 100B notifications/day (~1.2M average; burstier at events). Payloads are small; connection state and fan-out are the dominant concerns.

HIGH-LEVEL DESIGN

```
Provider → authenticated API → policy/queue → device-route shards
                                              └→ gateways ⇄ devices
```

Authenticate providers, validate compact payloads, apply app/user quotas, and enqueue by priority. Shard device routing state, maintain persistent gateways, collapse replaceable notifications, and retain only bounded offline messages. The payload should reveal minimal sensitive data.

DEEP DIVES

- Delivery semantics: at-least-once with collapse IDs and client dedupe is practical; acknowledgments measure transport, not user action.
- Power: batch low-priority wakeups; reserve immediate delivery for high-value events.
- Token lifecycle: rotate and invalidate tokens, never treat them as user identity, and isolate provider credentials.

LIKELY FOLLOW-UPS

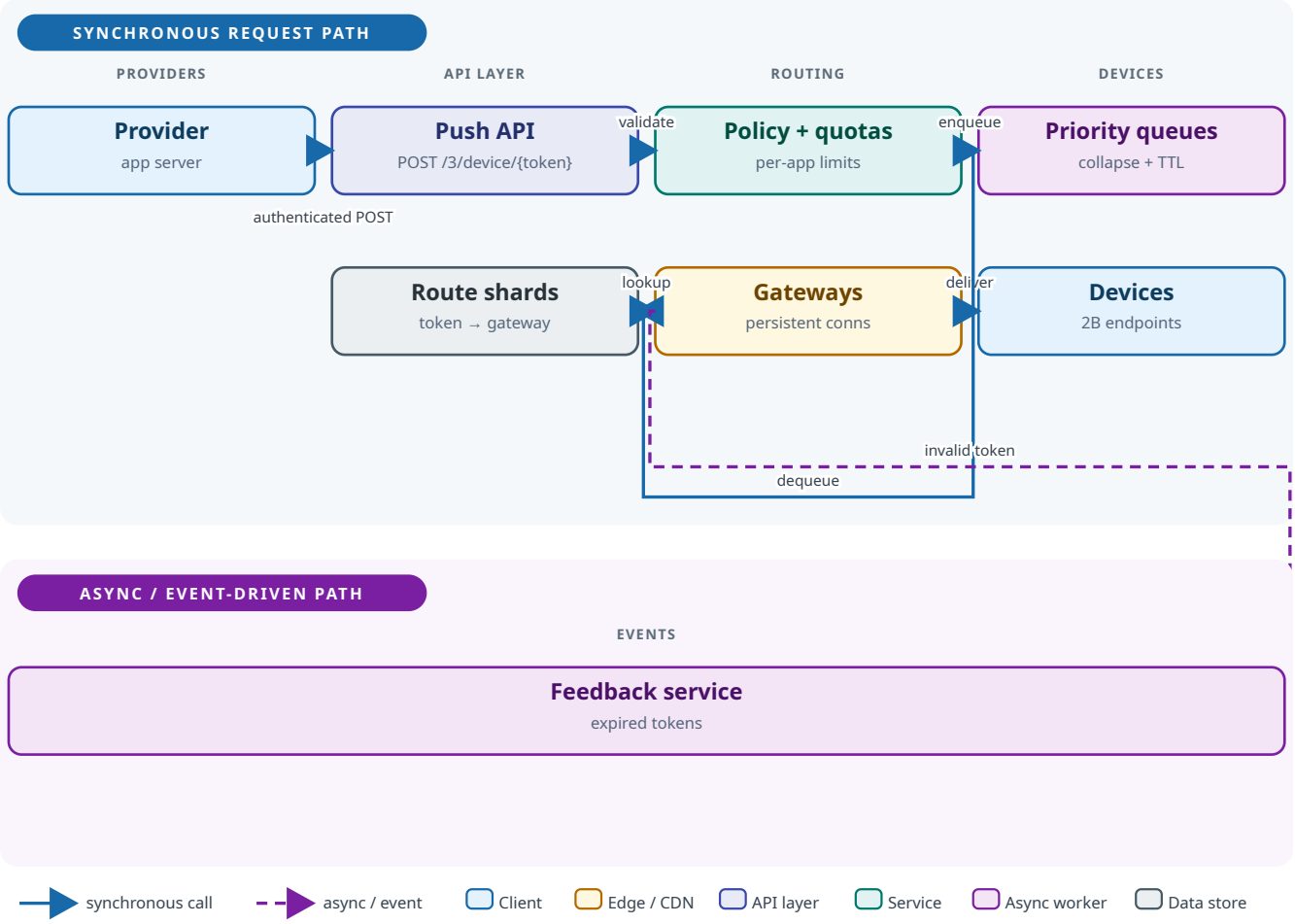
- A news event causes a global burst—how do you protect transactional traffic?
- How do expired tokens get cleaned up?
- What telemetry is safe to retain?

TIE IT TO YOUR BACKGROUND

- Use AT&T mobile API/security work for provider auth, JWT, TLS, and token-lifecycle reasoning.
- Relate fan-out queues and retry isolation to your message-broker/OpenShift automation.
- Use 80+ engineer leadership to define gateway, routing, platform-security, and developer-experience ownership.

Architecture diagram — question 16

APNs — push at planetary scale



At-least-once transport with collapse; acknowledgments measure transport, not taps.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 16

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Polling vs persistent device connections** → chosen persistent multiplexed connections because polling by hundreds of millions of devices is a permanent thundering herd; one long-lived connection per device fans out every app's pushes.

2. **Decision: At-least-once vs at-most-once delivery** → chosen at-most-once with coalescing because duplicate pushes visibly annoy users and exactly-once is impossible; coalescing collapses a burst into the latest payload per app.

3. **Decision: Per-app queues vs per-device queue** → chosen per-device queue with per-app priority because per-app queues complicate device wake scheduling; the device queue orders across apps by priority and collapses duplicates.

4. **Decision: Synchronous provider API vs async intake** → chosen async with token-based auth because provider QPS is extremely spiky; async intake absorbs spikes and token auth avoids a handshake per message.

5. **Decision: Single region vs regional with anycast** → chosen regional endpoints behind anycast because one region is a total blast radius; anycast routes each device to the nearest healthy region automatically.

End-to-end flows and API contracts — question 16

End-to-end request flows

JOURNEY 1 — PROVIDER SENDS A PUSH TO AN ONLINE DEVICE

1. The provider authenticates with a JWT and POSTs to /3/device/{token} over HTTP/2.
2. The API validates payload shape, topic scope, and per-app/per-user quotas.
3. The notification is enqueued by priority; alerts jump ahead of background updates.
4. The router resolves the token in the sharded device-route table to the owning gateway.
5. The gateway writes the frame to the persistent device connection; the device acks.
6. Collapsible payloads replace queued-but-unsent earlier ones with the same collapse id.
7. A 410 Unregistered response tells the provider to delete the dead token.

JOURNEY 2 — DEVICE OFFLINE; NOTIFICATION STORED THEN COLLAPSED

1. With no live connection, the notification goes to the bounded per-device offline store.
2. Only the newest notification per collapse id is kept; the store is size- and time-bounded.
3. On reconnect, the gateway drains the offline store in priority order before live traffic.
4. Payloads stay minimal by design; the app fetches details itself after waking.
5. Expired offline notifications are dropped rather than delivered stale.
6. Token rotation tombstones the old token so in-flight sends fail safe.

API contracts

POST `/3/device/{device_token}` Send a push notification (HTTP/2)

Auth: Provider JWT (ES256) · apns-topic: com.example.app · apns-priority: 10

```
→ {"aps":{"alert":{"title":"Flight delayed","body":"UA 412 boards 4:40 PM"},"badge":1}}
← 200 {"apns-id":"e41f..."} · 410 {"reason":"Unregistered"}
```

GET `/v1/feedback` Poll failed/dead tokens

Auth: Provider JWT

```
→ ?since=2026-09-18T09:00:00Z
← 200 {"tokens":[{"device_token":"dt_abc","reason":"Unregistered"}]}
```

GET `/internal/v1/routes/{token}` Resolve token to gateway shard (internal)

Auth: mTLS service identity

```
→ —
← 200 {"device_token":"dt_abc","shard":"gw-us-east-3","status":"online"}
```

GET `/v1/quotas/{app_id}` Inspect app send quotas (ops)

Auth: Admin token

```
→ —
← 200 {"app_id":"com.example.app","per_device_per_hour":120}
```

How I would answer question 16

MODEL ANSWER - AS THE INTERVIEWEE

A4. Design Apple Push Notification service (APNs) at scale

I'll start by narrowing the problem before drawing architecture. For Design Apple Push Notification service (APNs) at scale, I would ask: Transactional and background notifications; Target delivery latency and offline retention; Topic fan-out and collapse semantics; Per-app quotas and priority levels? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 2B devices, 500M concurrent connections, 100B notifications/day (~1.2M average; burstier at events). Payloads are small; connection state and fan-out are the dominant concerns. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Provider → authenticated API → policy/queue → device-route shards Then └→ gateways ⇄ devices. The end-to-end design is: Authenticate providers, validate compact payloads, apply app/user quotas, and enqueue by priority. Shard device routing state, maintain persistent gateways, collapse replaceable notifications, and retain only bounded offline messages. The payload should reveal minimal sensitive data. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Delivery semantics: at-least-once with collapse IDs and client dedupe is practical; acknowledgments measure transport, not user action. The second is Power: batch low-priority wakeups; reserve immediate delivery for high-value events. The third is Token lifecycle: rotate and invalidate tokens, never treat them as user identity, and isolate provider credentials. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "A news event causes a global burst—how do you protect transactional traffic?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Use AT&T mobile API/security work for provider auth, JWT, TLS, and token-lifecycle reasoning. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 17 of 28

A5. Design Apple Pay

WHY THEY ASK IT

Tests security boundaries, tokenization, device trust, transaction orchestration, replay prevention, privacy, and failure handling. The strongest answer distinguishes card credentials, device tokens, cryptograms, and merchant-facing data.

CLARIFYING QUESTIONS TO ASK

- In-store near-field communication, in-app, web, or all?
- Who tokenizes: network token service or issuer?
- Offline terminal behavior?
- Fraud, dispute, and availability requirements?
- What data may Apple, merchant, and issuer observe?

SCALE ESTIMATION

Assume 100M transactions/day (~1.2K average; 20× peak), strict p99 latency, and financial-grade auditability. Correctness, cryptographic operations, and external network dependencies dominate—not raw storage.

HIGH-LEVEL DESIGN

User auth → Secure Enclave authorizes device credential
Device → merchant/terminal → acquirer/network → issuer
token + one-time cryptogram → authorization

Provision a device-specific token after issuer verification; protect key material in the Secure Enclave. At payment time, require user presence, generate a transaction-specific cryptogram, and send tokenized data through existing payment rails. Make retries idempotent and reconcile authorization outcomes.

DEEP DIVES

- Threat model: stolen device, replay, compromised merchant, rooted client, and account takeover require distinct controls.
- Offline constraints: bounded offline authorization may improve availability but increases issuer and fraud risk.
- Privacy: minimize linkable identifiers and separate operational telemetry from transaction content.

LIKELY FOLLOW-UPS

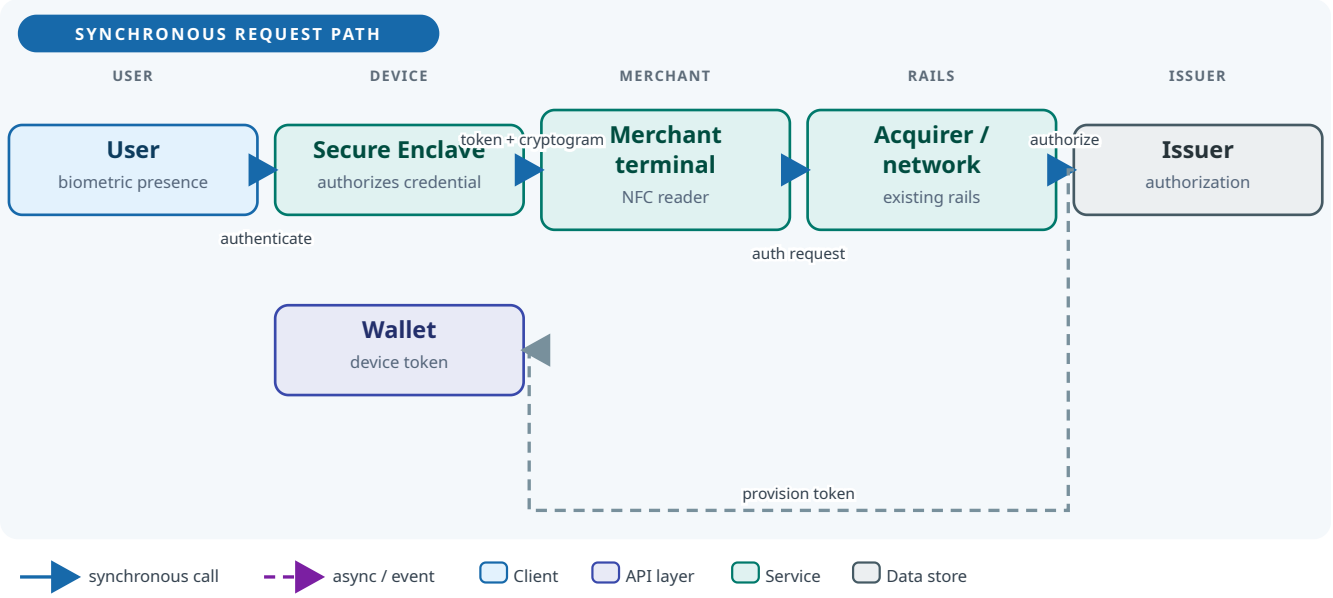
- The authorization response times out—do you retry?
- How is a lost device revoked?
- How do you rotate compromised token keys?

TIE IT TO YOUR BACKGROUND

- Lead with AT&T identity/security architecture: OAuth 2.0, JWT, HMAC-SHA1, TLS 1.2, SSO, 2FA, and step-up authentication.
- Use CEH v10, OWASP, and bug-bounty remediation to frame layered threat modeling and incident response.
- Connect seven-system integration to orchestrating external participants while preserving idempotency and audit trails.

Architecture diagram — question 17

Apple Pay — tokenized payments



Real card numbers never leave the device path; every tap is a one-time cryptogram.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 17

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Real PAN vs tokenized device account number** → chosen tokenized DAN because the real card number must never touch merchants or leave the secure element; tokens are domain-scoped and revocable without reissuing the card.

2. **Decision: Online-only authorization vs offline-capable** → chosen offline-capable with secure-element counters because transit gates and terminals go offline; the secure element enforces spend counters and risk limits without network access.

3. **Decision: Synchronous deep risk check vs lightweight sync plus async deep eval** → chosen lightweight sync with async deep evaluation because the terminal decision must return in milliseconds; deep fraud models run async and feed future risk scores.

4. **Decision: NoSQL vs ACID SQL for the transaction ledger** → chosen ACID SQL because money movement needs exactly-once accounting; eventual consistency risks double-spend or lost authorizations, which are correctness failures, not performance trade-offs.

5. **Decision: Pull provisioning vs push with attestation** → chosen push with hardware attestation because the credential must be bound to a genuine secure element; attestation proves the key was generated in and never left the hardware.

End-to-end flows and API contracts — question 17

End-to-end request flows

JOURNEY 1 — IN-STORE NFC PAYMENT

1. The user authenticates with Face ID; the Secure Enclave authorizes the device credential.
2. The device builds a one-time cryptogram bound to amount, merchant, and counter.
3. Over NFC, the terminal receives the tokenized PAN plus cryptogram; never the real card number.
4. The terminal routes via acquirer and network, which detokenizes and forwards to the issuer.
5. The issuer authorizes (funds, fraud checks) and the decision returns down the rails.
6. Retries reuse the idempotency key; a duplicate cryptogram is recognized, never double-charged.
7. Settlement and reconciliation run asynchronously after the user-facing authorization.

JOURNEY 2 — PROVISIONING A NEW CARD

1. The user adds a card; the app calls POST /v1/tokens/provision with the card reference.
2. The issuer runs verification (OTP or bank-app approval) before minting any token.
3. On approval, a device-specific token is sealed into the Secure Enclave.
4. The real PAN never lands on the device in recoverable form.
5. Each device gets its own token; revoking one device never affects the others.
6. Lost-device mode suspends all tokens remotely via the token service.

API contracts

POST `/v1/tokens/provision` Provision a device token for a card

Auth: Device identity + user auth · Content-Type: application/json

```
→ {"card_ref":"cr_551","verification":{"method":"otp","otp":"482910"}}
← 201 {"token_ref":"tok_991","state":"active"}
```

POST `/v1/payments/authorize` Authorize a tokenized payment

Auth: Acquirer credentials · Content-Type: application/json · Idempotency-Key: <uuid>

```
→ {"token_ref":"tok_991","cryptogram":"9f26...","amount":42.50,"currency":"USD"}
← 200 {"payment_id":"pay_77120","status":"authorized","auth_code":"A12B34"}
```

GET `/v1/payments/{id}` Fetch payment status and receipt

Auth: Device identity token

```
→ —
← 200 {"payment_id":"pay_77120","status":"captured","amount":42.50}
```

DELETE `/v1/tokens/{token_ref}` Suspend or delete a device token

Auth: Device identity or issuer

```
→ {"reason":"device_lost"}
← 200 {"token_ref":"tok_991","state":"suspended"}
```

How I would answer question 17

MODEL ANSWER - AS THE INTERVIEWEE

A5. Design Apple Pay

I'll start by narrowing the problem before drawing architecture. For Design Apple Pay, I would ask: In-store near-field communication, in-app, web, or all; Who tokenizes: network token service or issuer; Offline terminal behavior; Fraud, dispute, and availability requirements? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 100M transactions/day (~1.2K average; 20× peak), strict p99 latency, and financial-grade auditability. Correctness, cryptographic operations, and external network dependencies dominate—not raw storage. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: User auth → Secure Enclave authorizes device credential Then Device → merchant/terminal → acquirer/network → issuer Then token + one-time cryptogram → authorization. The end-to-end design is: Provision a device-specific token after issuer verification; protect key material in the Secure Enclave. At payment time, require user presence, generate a transaction-specific cryptogram, and send tokenized data through existing payment rails. Make retries idempotent and reconcile authorization outcomes. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Threat model: stolen device, replay, compromised merchant, rooted client, and account takeover require distinct controls. The second is Offline constraints: bounded offline authorization may improve availability but increases issuer and fraud risk. The third is Privacy: minimize linkable identifiers and separate operational telemetry from transaction content. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "The authorization response times out—do you retry?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Lead with AT&T identity/security architecture: OAuth 2.0, JWT, HMAC-SHA1, TLS 1.2, SSO, 2FA, and step-up authentication. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 18 of 28

A6. Design an offline-first messaging app with E2E encryption

WHY THEY ASK IT

Combines Apple's device-first lens with messaging correctness and end-to-end encryption (E2EE). It tests whether privacy survives offline queues, multi-device keys, backup, and push notifications.

CLARIFYING QUESTIONS TO ASK

- One-to-one, groups, attachments, and multi-device?
- How long can a device remain offline?
- Message ordering and deletion semantics?
- Encrypted backup and key recovery?
- Metadata minimization requirements?

SCALE ESTIMATION

Assume 100M daily users, 100 messages/user/day (~116K average; 8× peak), five devices/user, and 1 KB median envelope. Media uses a separate encrypted object path.

HIGH-LEVEL DESIGN

Local encrypted DB → outbox → relay store → recipient inbox
Key service: public prekeys only; plaintext stays on devices

Write locally first, assign conversation sequence and idempotency IDs, encrypt on-device, then upload envelopes to a bounded relay. Devices fetch by cursor and acknowledge receipt; push contains only an opaque wake hint. Multi-device sessions receive separately encrypted key material.

DEEP DIVES

- Key management: prekeys enable asynchronous start; device addition and removal require transparent verification and rekeying.
- Sync conflicts: preserve message events; edits/deletes are new signed events, ordered per conversation.
- Power/storage: batch background sync, cap local media, and defer large transfers based on network and battery.

LIKELY FOLLOW-UPS

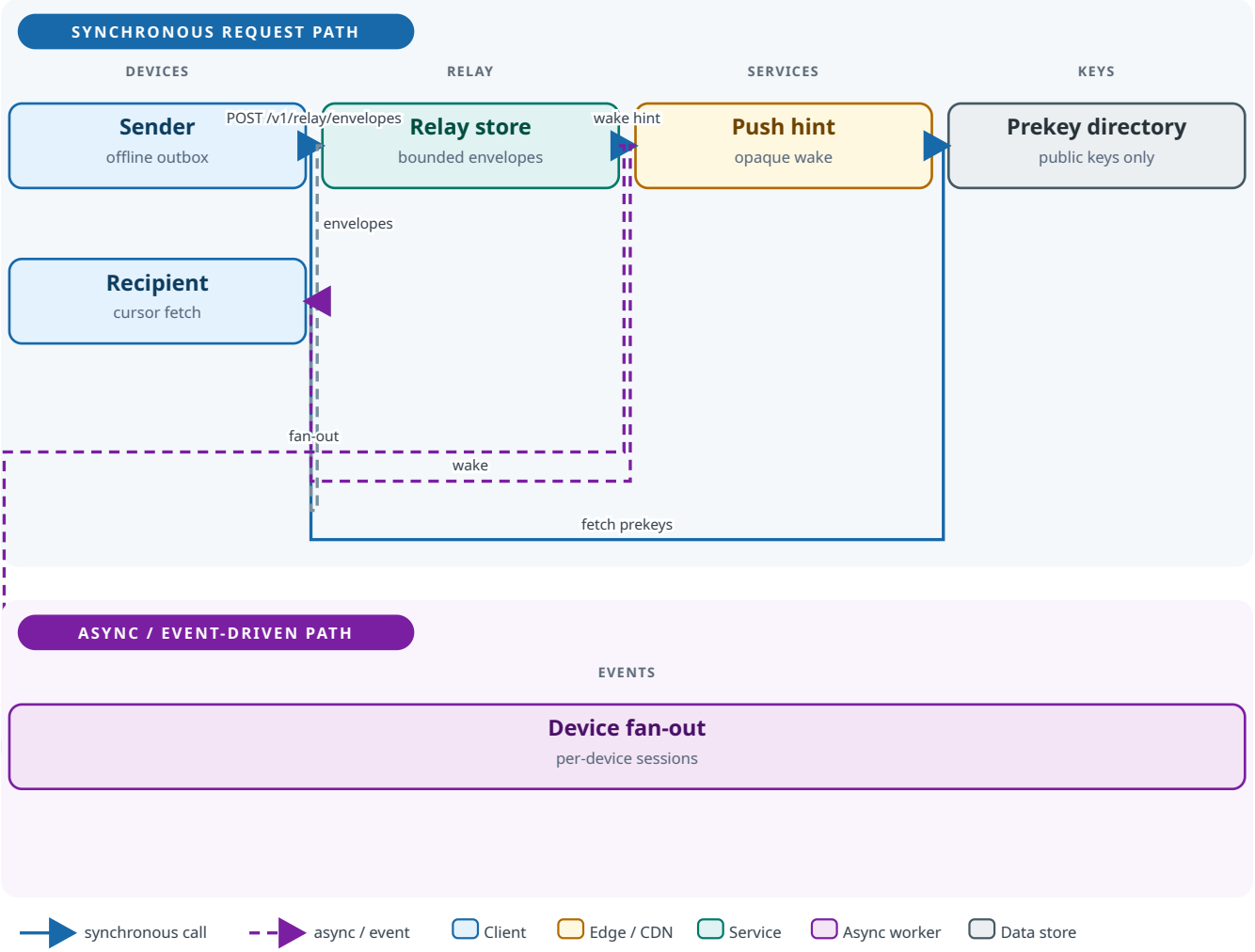
- How does a new device obtain history?
- What if the sender deletes while the recipient is offline?
- How do abuse reports work without server plaintext?

TIE IT TO YOUR BACKGROUND

- Use your AT&T security background for authentication, key-bound sessions, TLS, 2FA, and step-up controls around device enrollment.
- Connect outbox/relay design to event-driven platforms and message brokers; avoid claiming E2EE product delivery.
- Apply zero-trust and DevSecOps practices to key-service deployment and privileged operations.

Architecture diagram — question 18

Offline-first E2E encrypted messaging



Write locally, encrypt on-device; the relay stores envelopes, never plaintext.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 18

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1 **Decision: Server-readable vs Signal-protocol end-to-end encryption** → chosen E2E with a prekey directory because the relay must store but never read; prekeys let a sender bootstrap an encrypted session with a recipient that is offline.

2 **Decision: Retain envelopes vs delete-on-delivery with bounded retention** → chosen delete-on-delivery with bounded retention because keeping ciphertext-equivalent envelopes forever expands the breach surface; bounded retention caps exposure if the relay is compromised.

3 **Decision: Push with content vs opaque wake hint** → chosen opaque wake hint because push payloads leak metadata to the push provider; the hint carries zero content and the client fetches envelopes over its own encrypted channel.

4 **Decision: Per-user session vs per-device sessions with fan-out** → chosen per-device sessions because each device holds its own keys; a per-user session cannot give per-device forward secrecy or survive single-device compromise.

5 **Decision: Synchronous send vs local outbox with async flush** → chosen local outbox because offline-first means the UI can never block on network; the outbox preserves order, retries with backoff, and drains when connectivity returns.

End-to-end flows and API contracts — question 18

End-to-end request flows

JOURNEY 1 — USER SENDS MESSAGES WHILE OFFLINE

1. With no connectivity, messages encrypt on-device into the local outbox with client message ids.
2. The UI shows queued state honestly: stored locally is not delivered.
3. On reconnect, the outbox drains in order; each envelope POSTs with its idempotency id.
4. The relay appends envelopes to the recipient's bounded inbox; retried duplicates are dropped.
5. The sender marks envelopes accepted once the relay acknowledges them.
6. The recipient fetches by cursor, decrypts locally, and renders in conversation order.
7. Push carries only an opaque wake hint; read receipts are encrypted envelopes on the same path.

JOURNEY 2 — READING ON A SECOND DEVICE (MULTI-DEVICE)

1. Each trusted device owns its keypair; the sender encrypts one envelope per recipient device.
2. Reading on the phone emits an E2E sync envelope so the laptop marks the thread read.
3. The laptop fetches via its own cursor; state converges without the server understanding it.
4. New devices enroll via approval from an existing trusted device; keys exchange out-of-band.
5. The server directory holds only public prekeys; it routes but never decrypts.
6. Revoking a device publishes through the trusted-device channel; future envelopes exclude it.

API contracts

POST `/v1/relay/envelopes` Submit encrypted envelopes

Auth: Device session token · Content-Type: application/json

```
→ {"envelopes":[{"message_id":"m_9f2","to_device":"dev_b2","ciphertext":"base64..."}]}
← 202 {"accepted":["m_9f2"],"rejected":[]}
```

GET `/v1/relay/inbox` Fetch envelopes after cursor

Auth: Device session token

```
→ ?cursor=99120&limit=100
← 200 {"envelopes":[{"message_id":"m_9f2","ciphertext":"base64..."}],"next_cursor":99121}
```

POST `/v1/relay/ack` Acknowledge receipt; enables compaction

Auth: Device session token · Content-Type: application/json

```
→ {"cursor":99121}
← 200 {"compacted_through":99121}
```

GET `/v1/keys/prekeys/{device}` Public prekey bundle for a device

Auth: Device session token

```
→ —
← 200 {"device_id":"dev_b2","identity_key":"base64...","signed_prekey":{"id":7}}
```

How I would answer question 18

MODEL ANSWER - AS THE INTERVIEWEE

A6. Design an offline-first messaging app with E2E encryption

I'll start by narrowing the problem before drawing architecture. For Design an offline-first messaging app with E2E encryption, I would ask: One-to-one, groups, attachments, and multi-device; How long can a device remain offline; Message ordering and deletion semantics; Encrypted backup and key recovery? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 100M daily users, 100 messages/user/day (~116K average; 8× peak), five devices/user, and 1 KB median envelope. Media uses a separate encrypted object path. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Local encrypted DB → outbox → relay store → recipient inbox Then Key service: public prekeys only; plaintext stays on devices. The end-to-end design is: Write locally first, assign conversation sequence and idempotency IDs, encrypt on-device, then upload envelopes to a bounded relay. Devices fetch by cursor and acknowledge receipt; push contains only an opaque wake hint. Multi-device sessions receive separately encrypted key material. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Key management: prekeys enable asynchronous start; device addition and removal require transparent verification and rekeying. The second is Sync conflicts: preserve message events; edits/deletes are new signed events, ordered per conversation. The third is Power/storage: batch background sync, cap local media, and defer large transfers based on network and battery. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "How does a new device obtain history?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Use your AT&T security background for authentication, key-bound sessions, TLS, 2FA, and step-up controls around device enrollment. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 19 of 28

A7. Design the App Store backend

WHY THEY ASK IT

Tests global catalog/search, binary delivery, entitlements, ranking, recommendations, review workflows, and developer trust. The key is separating strongly consistent commerce from eventually consistent discovery.

CLARIFYING QUESTIONS TO ASK

- Catalog/search only, or purchases and binary delivery too?
- Global storefront and regional policy differences?
- Review workflow and emergency takedowns?
- Recommendation personalization constraints?
- Update propagation and rollback targets?

SCALE ESTIMATION

Assume 2M apps, 1B active devices, 100K search QPS at peak, and very large binary egress during popular updates. Catalog scale is modest; traffic bursts, policy, and correctness are hard.

HIGH-LEVEL DESIGN

Developer → submission → scan/review → catalog/version store → CDN
User → search/recs → product page → entitlement → signed download

Use an immutable app-version pipeline, automated scans plus human review, a versioned catalog, search/recommendation indexes, strongly consistent entitlements, and signed CDN downloads. Regional policy filters operate at serving time; emergency takedowns have a fast control path.

DEEP DIVES

- Release safety: phased rollout, crash/quality telemetry, automatic pause, and rollback protect devices.
- Search/recs: eventual indexes are acceptable; purchases and entitlements are not. Privacy-safe on-device signals can reduce central profiling.
- Review operations: queues need priority, evidence, auditability, and clear human decision points.

LIKELY FOLLOW-UPS

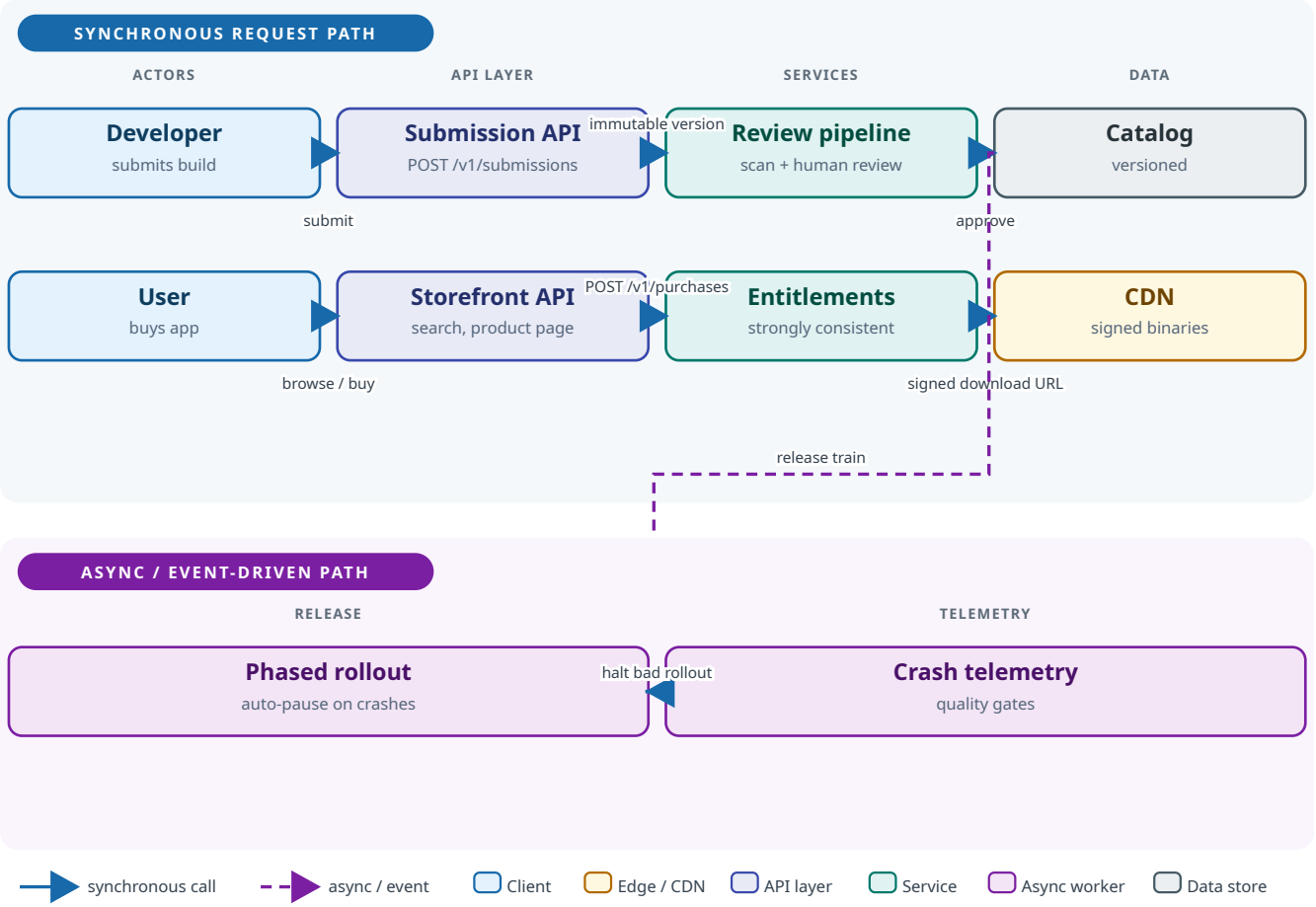
- A critical app update overloads the CDN—what changes?
- How does an emergency malware takedown propagate?
- How do family entitlements sync offline?

TIE IT TO YOUR BACKGROUND

- Use Marriott.com consumer-scale reservations/rewards work to establish customer-facing reliability and high-traffic web judgment.
- Connect automated review to intelligent document processing and agentic-AI governance—automation assists, humans retain policy authority.
- Apply your CI/CD, DevSecOps, and bug-bounty background to artifact scanning, phased rollout, and emergency response.

Architecture diagram — question 19

App Store backend — catalog & commerce



Commerce is strongly consistent; discovery and rollout are eventually consistent.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 19

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Single-stage vs staged review pipeline** → chosen staged (automated screen, then human) because ML pre-screening filters obvious policy violations cheaply; expensive human reviewers see only ambiguous submissions.

2. **Decision: Eventual vs strongly consistent purchase ledger** → chosen strongly consistent (ACID) entitlements because charging without granting access, or granting without charging, is a correctness bug; eventual consistency is fine for charts and reviews but never for money.

3. **Decision: Origin download vs CDN with signed URLs** → chosen CDN with signed URLs because app binaries are large, immutable, and hot; signed URLs keep the origin off the download path while still enforcing purchase entitlement.

4. **Decision: Synchronous review on submit vs async with status tracking** → chosen async because review takes hours; the submit path acknowledges immediately and polling/webhooks track the submission through the pipeline.

5. **Decision: One global catalog vs regional catalogs** → chosen regional catalogs with global fallback because pricing, law, and availability differ by region; regions override the global catalog instead of forking it.

End-to-end flows and API contracts — question 19

End-to-end request flows

JOURNEY 1 — DEVELOPER SUBMITS AN APP UPDATE

1. The developer uploads binary and metadata; the API returns a submission id and starts the pipeline.
2. Automated scans check malware, private API usage, crash-on-launch, and privacy manifests.
3. Clean submissions queue for human review against diffs from the last approved version.
4. On approval, the version is marked immutable and published to the versioned catalog.
5. Search and recommendation indexes ingest the new metadata for discoverability.
6. Regional policy filters attach at serving time; one catalog serves all regions correctly.
7. Emergency takedown can pull a version globally within minutes; every transition is audited.

JOURNEY 2 — USER BUYS AND DOWNLOADS AN APP

1. The product page call returns metadata plus the regional policy decision (available or gated).
2. Tapping buy records the entitlement in the strongly consistent entitlement store.
3. The client requests a download; the service issues a short-lived signed CDN URL.
4. The binary streams from the nearest edge; delta updates fetch only changed chunks.
5. The device verifies the code signature before first launch.
6. Refunds revoke the entitlement; installed features stop working without it.

API contracts

POST `/v1/submissions` Submit a new app version

Auth: Developer token · Content-Type: application/json

```
→ {"app_id":"com.example.coolapp","version":"2.4.0","binary_ref":"bin_881"}
← 202 {"submission_id":"sub_1201","status":"scanning"}
```

GET `/v1/apps/{id}` Product page metadata (regionalized)

Auth: User token (optional) · X-Region: US

```
→ —
← 200 {"app_id":"com.example.coolapp","version":"2.4.0","rating":4.6,"policy":{"available":true}}
```

POST `/v1/purchases` Purchase an app or in-app item

Auth: User token · Content-Type: application/json · Idempotency-Key: <uuid>

```
→ {"app_id":"com.example.coolapp","sku":"pro_monthly"}
← 201 {"purchase_id":"p_771","entitlement_id":"ent_991","status":"active"}
```

GET `/v1/downloads/{version}` Signed download URL

Auth: User token (entitlement-checked)

```
→ —
← 200 {"url":"https://cdn.example.com/dl/...?sig=...","size_bytes":48211200}
```

How I would answer question 19

MODEL ANSWER - AS THE INTERVIEWEE

A7. Design the App Store backend

I'll start by narrowing the problem before drawing architecture. For Design the App Store backend, I would ask: Catalog/search only, or purchases and binary delivery too; Global storefront and regional policy differences; Review workflow and emergency takedowns; Recommendation personalization constraints? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 2M apps, 1B active devices, 100K search QPS at peak, and very large binary egress during popular updates. Catalog scale is modest; traffic bursts, policy, and correctness are hard. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Developer → submission → scan/review → catalog/version store → CDN Then User → search/recs → product page → entitlement → signed download. The end-to-end design is: Use an immutable app-version pipeline, automated scans plus human review, a versioned catalog, search/recommendation indexes, strongly consistent entitlements, and signed CDN downloads. Regional policy filters operate at serving time; emergency takedowns have a fast control path. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Release safety: phased rollout, crash/quality telemetry, automatic pause, and rollback protect devices. The second is Search/recs: eventual indexes are acceptable; purchases and entitlements are not. Privacy-safe on-device signals can reduce central profiling. The third is Review operations: queues need priority, evidence, auditability, and clear human decision points. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "A critical app update overloads the CDN—what changes?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Use Marriott.com consumer-scale reservations/rewards work to establish customer-facing reliability and high-traffic web judgment. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 20 of 28

A8. Design “Recently Deleted” in Photos

WHY THEY ASK IT

A compact feature reveals data lifecycle rigor: 30-day recovery, secure deletion, offline state, storage pressure, and cross-device conflict. The interviewer looks for precise semantics and edge cases.

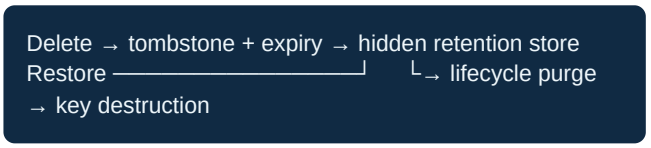
CLARIFYING QUESTIONS TO ASK

- Exactly 30 days from which event and clock?
- Can users restore or delete immediately from any device?
- Does deleted media consume cloud and device quota?
- Shared-library behavior?
- What does “secure deletion” guarantee across backups?

SCALE ESTIMATION

Assume billions of assets enter deletion state daily. Metadata and scheduled-expiry load are manageable, but full-resolution media retention has major storage cost. Avoid one timer per asset.

HIGH-LEVEL DESIGN



Represent deletion as a versioned tombstone with server-authoritative expiry. Hide assets from normal views, preserve encrypted media in a retention tier, and use bucketed lifecycle scans. Restore creates a newer state transition; permanent delete revokes keys and schedules physical cleanup.

DEEP DIVES

- Sync conflicts: compare versions when an offline restore meets a later permanent delete; destructive operations need explicit precedence and confirmation.
- Storage/power: evict local originals first under pressure, keep thumbnails minimal, and defer cleanup work appropriately.
- Privacy: delete encryption keys promptly where possible, document backup limits, and audit privileged restores.

LIKELY FOLLOW-UPS

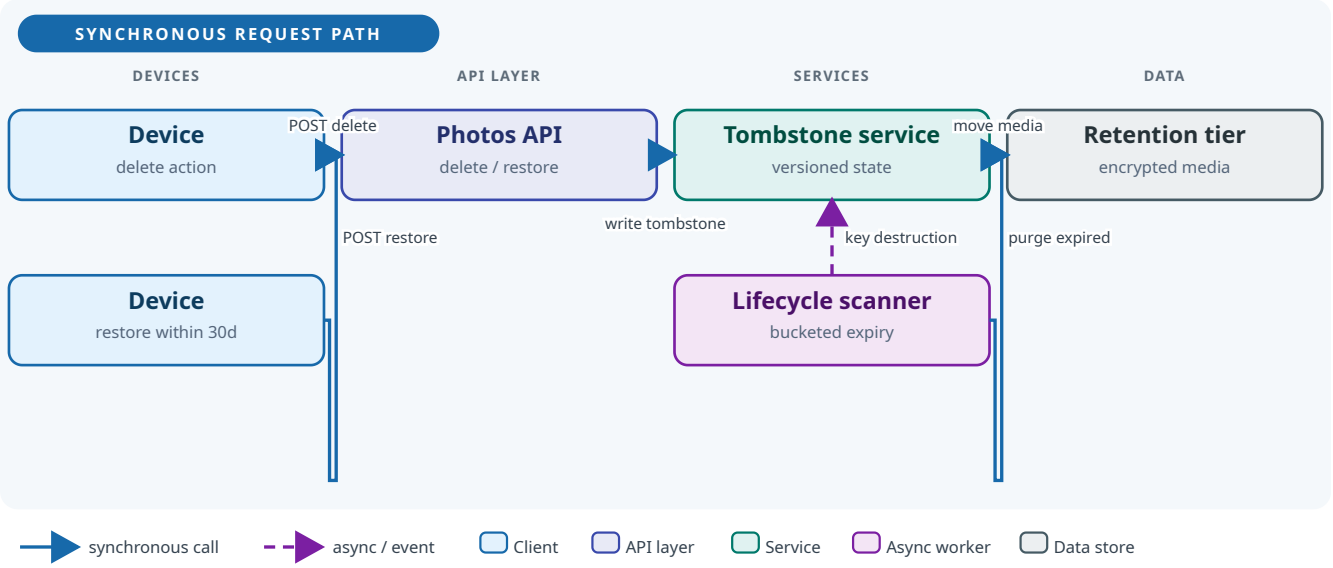
- The device clock is wrong—when does expiry happen?
- A shared owner deletes while a participant restores—who wins?
- How do you prove deletion completion?

TIE IT TO YOUR BACKGROUND

- Map tombstones and replay to your event-driven consistency work without claiming Photos experience.
- Use 30% FinOps savings to discuss retention-tier economics and lifecycle policies.
- Connect auditability and secure deletion to zero-trust, OWASP, and security-architecture experience.

Architecture diagram — question 20

Recently Deleted — data lifecycle



Deletion is a versioned state transition with server-authoritative expiry.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 20

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Hard delete vs soft delete with tombstones** → chosen soft delete plus tombstones because 'recently deleted' must be restorable for 30 days and converge across devices; tombstones make the deletion itself a replicated event.

2. **Decision: Reference counting vs mark-and-sweep garbage collection** → chosen mark-and-sweep after the grace period because distributed reference counting across devices is fragile; the sweeper reclaims only after the retention window and after all replicas acknowledge.

3. **Decision: Delete bytes first vs metadata first** → chosen metadata first because bytes are content-addressed and deduplicated across assets; deleting bytes first would corrupt other live references to the same bytes.

4. **Decision: Per-device expiry vs server-driven expiry log** → chosen server-driven expiry because device clocks skew; the server's ordered log is the single expiry authority and every device replays it.

5. **Decision: Synchronous purge API vs async purge jobs** → chosen async purge jobs because expiry is a background lifecycle concern; the user-facing delete only flips state, and the sweeper does the expensive reclamation off the critical path.

End-to-end flows and API contracts — question 20

End-to-end request flows

JOURNEY 1 — USER DELETES A PHOTO

1. The app calls POST /v1/assets/{id}/delete; the service writes a tombstone with server-side expiry.
2. The asset vanishes from normal views on every device via the change log.
3. Encrypted media moves to the retention tier: recoverable but invisible to sharing and search.
4. The tombstone carries deleter, timestamp, and expiry; edits during the window follow product policy.
5. A lifecycle scanner buckets tombstones by expiry so purge work spreads evenly.
6. Offline devices sync the tombstone on reconnect; deletes are versioned state transitions.

JOURNEY 2 — USER RESTORES WITHIN 30 DAYS

1. The user calls POST /v1/assets/{id}/restore from Recently Deleted.
2. The service validates the tombstone has not expired and writes a newer live state transition.
3. The asset reappears in place on all devices via the change log; no re-upload needed.
4. An expired tombstone returns 410 Gone with an honest "permanently deleted" message.
5. After expiry, the scanner revokes encryption keys, then schedules physical cleanup.
6. Key destruction is the real deletion: ciphertext is unrecoverable even before blocks are wiped.

API contracts

POST `/v1/assets/{id}/delete` Soft-delete into retention

Auth: iCloud token (device) · Content-Type: application/json

```
→ {"reason":"user"}
← 200 {"asset_id":"as_991","state":"recently_deleted","expires_at":"2026-10-18T10:00:00Z"}
```

GET `/v1/recently-deleted` List recoverable assets

Auth: iCloud token (device)

```
→ ?limit=100
← 200 {"items":[{"asset_id":"as_991","expires_at":"2026-10-18T10:00:00Z"}]}
```

POST `/v1/assets/{id}/restore` Restore a tombstoned asset

Auth: iCloud token (device) · Content-Type: application/json

```
→ {}
← 200 {"asset_id":"as_991","state":"active"}
```

DELETE `/v1/assets/{id}/purge` Permanently delete now

Auth: iCloud token (device)

```
→ —
← 200 {"asset_id":"as_991","state":"purged"}
```

How I would answer question 20

MODEL ANSWER - AS THE INTERVIEWEE

A8. Design “Recently Deleted” in Photos

I’ll start by narrowing the problem before drawing architecture. For Design “Recently Deleted” in Photos, I would ask: Exactly 30 days from which event and clock; Can users restore or delete immediately from any device; Does deleted media consume cloud and device quota; Shared-library behavior? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I’ll proceed with a focused first version and identify where the design evolves.

For scale, I’ll use round numbers and say them out loud: Assume billions of assets enter deletion state daily. Metadata and scheduled-expiry load are manageable, but full-resolution media retention has major storage cost. Avoid one timer per asset. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Delete → tombstone + expiry → hidden retention store Then Restore ——————————┘ └→ lifecycle purge → key destruction. The end-to-end design is: Represent deletion as a versioned tombstone with server-authoritative expiry. Hide assets from normal views, preserve encrypted media in a retention tier, and use bucketed lifecycle scans. Restore creates a newer state transition; permanent delete revokes keys and schedules physical cleanup. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Sync conflicts: compare versions when an offline restore meets a later permanent delete; destructive operations need explicit precedence and confirmation. The second is Storage/power: evict local originals first under pressure, keep thumbnails minimal, and defer cleanup work appropriately. The third is Privacy: delete encryption keys promptly where possible, document backup limits, and audit privileged restores. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, “The device clock is wrong—when does expiry happen?,” I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Map tombstones and replay to your event-driven consistency work without claiming Photos experience. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 21 of 28

Q21. Design Instagram: photo sharing and feed

WHY THEY ASK IT

Tests media ingestion, social-graph fan-out, ranking, hot accounts, storage, privacy, and cost. The director signal is choosing different delivery paths for ordinary users and celebrities while protecting feed freshness.

CLARIFYING QUESTIONS TO ASK

- Are we designing core upload and feed paths?
- How many daily users, posts, and feed reads?
- Chronological or ranked feed, and what freshness target?
- Photos only, or video and stories too?
- Privacy controls, deletion latency, and regional requirements?

SCALE ESTIMATION

Assume 500M daily users, 100M posts/day (~1.2K average writes/s), 20 feed opens/user/day (~116K average reads/s), 10× peak, and 3 MB per photo. Media adds ~300 TB/day before replicas and derivatives.

HIGH-LEVEL DESIGN

Upload → media service → object store/CDN → post metadata
Post event → fan-out workers → home-feed cache
Feed read → ranking service → hydrated posts → client

Use resumable uploads, media objects, asynchronous thumbnail/moderation pipelines, and a sharded post store. Fan out on write for ordinary accounts, fan out on read for high-follower accounts, then rank and hydrate from caches. Keep the social graph independently partitioned and propagate privacy changes through a priority invalidation path.

DEEP DIVES

- Fan-out: write-time delivery makes reads cheap but explodes for celebrities; a hybrid model bounds write amplification.
- Ranking and freshness: precomputed candidates reduce latency; late binding improves freshness and policy enforcement.
- Media lifecycle: content hashes, multiple renditions, CDN caching, tombstones, and auditable deletion control cost and compliance.

LIKELY FOLLOW-UPS

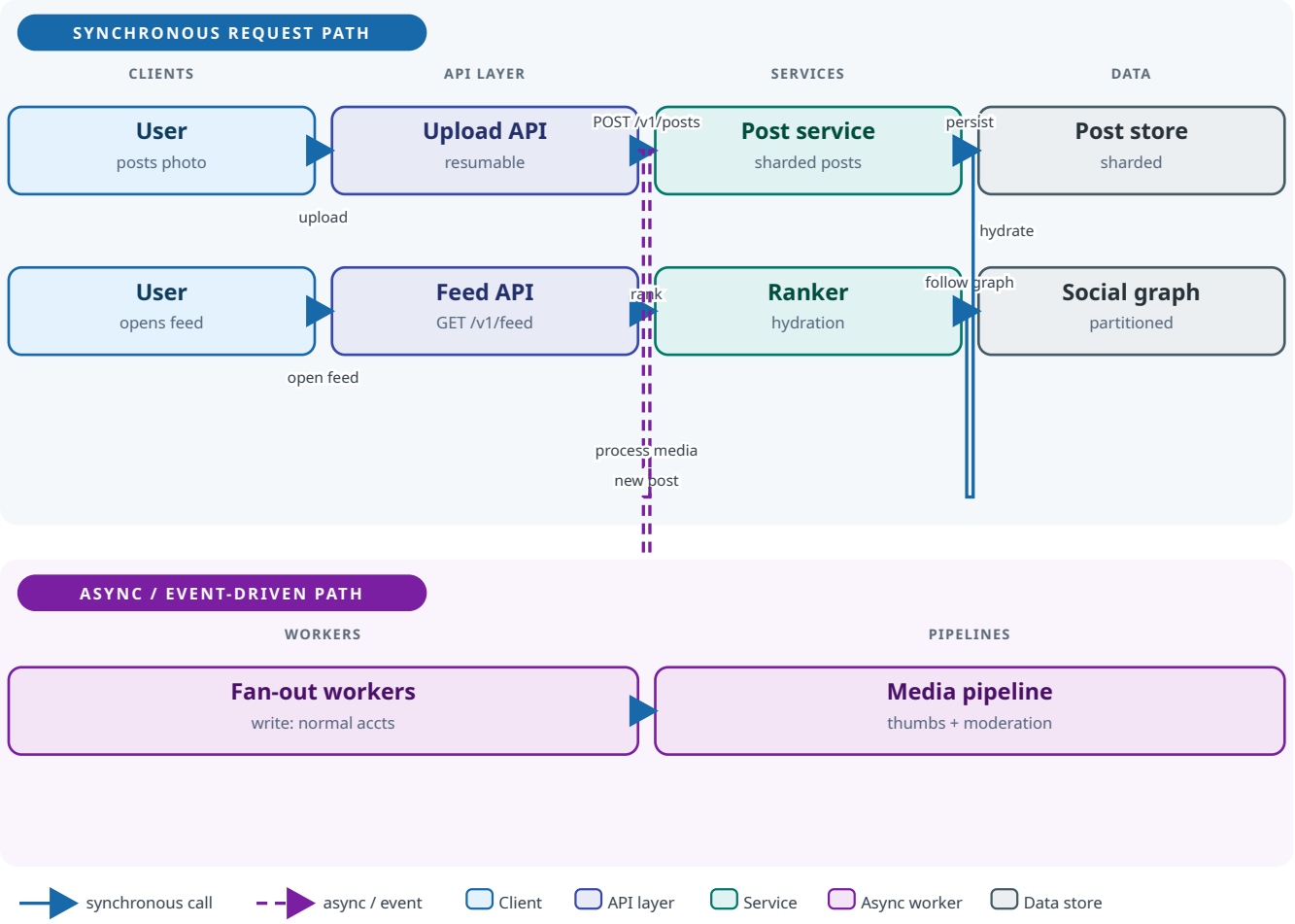
- A celebrity posts during a global event—what breaks first?
- How does a private-account change invalidate cached feeds?
- How would stories or short video alter storage and ranking?

TIE IT TO YOUR BACKGROUND

- Connect hybrid fan-out and event processing to your message-broker and event-driven platform experience.
- Use the 30% infrastructure-cost reduction to discuss CDN, rendition, and storage-tier economics.
- Relate the 30% latency and 40% API gains to feed-read budgets and cache discipline.

Architecture diagram — question 21

Instagram — post & feed



Fan-out on write for most; fan-out on read for celebrities; rank from cache.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 21

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Fan-out on write vs fan-out on read** → chosen hybrid (write for normal accounts, read for celebrities) because pure write fan-out multiplies one celebrity post into a hundred million writes; celebrities merge at read time while normal accounts keep cheap single-read feeds.

2. **Decision: Chronological vs ranked feed** → chosen ranked with a recency signal because pure chronological buries relevant posts under volume; the ranker blends recency, affinity, and freshness instead of picking one axis.

3. **Decision: Assemble feed on read vs precomputed materialized feeds** → chosen precomputed with incremental updates because assembling thousands of candidate posts per read is too slow at feed QPS; writes append to follower feeds asynchronously.

4. **Decision: Serve originals vs multi-rendition pipeline** → chosen multi-rendition because serving 4K originals to phones wastes bandwidth and battery; the media pipeline generates thumbnails and ladders once, and every read picks the right rendition.

5. **Decision: Relational vs adjacency store for the social graph** → chosen adjacency (NoSQL) store because follows are edge traversals, not joins; adjacency lists make 'all followers' a single range read for fan-out.

End-to-end flows and API contracts — question 21

End-to-end request flows

JOURNEY 1 — USER POSTS A PHOTO

1. The app creates a resumable upload, then PUTs image bytes in chunks.
2. On commit, the media service stores the original and writes a PENDING post row.
3. Async workers generate thumbnails, run moderation classifiers, and extract metadata.
4. Flagged posts stay invisible pending review; the user sees an honest status.
5. Once clean, the post flips ACTIVE and enters the fan-out pipeline.
6. Ordinary accounts fan out on write into each follower's feed cache.
7. Celebrity accounts skip write fan-out; followers merge the post at read time.

JOURNEY 2 — USER OPENS THEIR HOME FEED

1. The app calls GET /v1/feed with a cursor; the feed service reads the precomputed cache.
2. Recent posts from high-follower accounts merge in (read fan-out), then ranking applies.
3. Post ids hydrate via batched cache reads: metadata, signed media URLs, like counts.
4. Stories and suggestions interleave per ranking policy; ads slot at configured positions.
5. The response carries a next cursor; the client prefetches top media from the CDN.
6. If ranking degrades, the service falls back to reverse-chronological from the same caches.

API contracts

POST `/v1/media/uploads` Create resumable media upload

Auth: User OAuth · Content-Type: application/json

```
→ {"mime":"image/jpeg","bytes":3145728}
← 201 {"media_id":"m_551","session_uri":"https://up.example.com/s/m_551"}
```

POST `/v1/posts` Publish a post from uploaded media

Auth: User OAuth · Content-Type: application/json · Idempotency-Key: <uuid>

```
→ {"media_id":"m_551","caption":"Sunset over the lake","visibility":"public"}
← 202 {"post_id":"p_99120","status":"processing"}
```

GET `/v1/feed` Ranked home feed

Auth: User OAuth

```
→ ?cursor=c_881&limit=20
← 200 {"items":[{"post_id":"p_99120","likes":4821}],"next_cursor":"c_882"}
```

POST `/v1/posts/{id}/like` Like a post

Auth: User OAuth · Content-Type: application/json

```
→ {}
← 200 {"post_id":"p_99120","liked":true,"likes":4822}
```

How I would answer question 21

MODEL ANSWER - AS THE INTERVIEWEE

Q21. Design Instagram: photo sharing and feed

I'll start by narrowing the problem before drawing architecture. For Design Instagram: photo sharing and feed, I would ask: Are we designing core upload and feed paths; How many daily users, posts, and feed reads; Chronological or ranked feed, and what freshness target; Photos only, or video and stories too? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 500M daily users, 100M posts/day (~1.2K average writes/s), 20 feed opens/user/day (~116K average reads/s), 10× peak, and 3 MB per photo. Media adds ~300 TB/day before replicas and derivatives. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Upload → media service → object store/CDN → post metadata Then Post event → fan-out workers → home-feed cache Then Feed read → ranking service → hydrated posts → client. The end-to-end design is: Use resumable uploads, media objects, asynchronous thumbnail/moderation pipelines, and a sharded post store. Fan out on write for ordinary accounts, fan out on read for high-follower accounts, then rank and hydrate from caches. Keep the social graph independently partitioned and propagate privacy changes through a priority invalidation path. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Fan-out: write-time delivery makes reads cheap but explodes for celebrities; a hybrid model bounds write amplification. The second is Ranking and freshness: precomputed candidates reduce latency; late binding improves freshness and policy enforcement. The third is Media lifecycle: content hashes, multiple renditions, CDN caching, tombstones, and auditable deletion control cost and compliance. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "A celebrity posts during a global event—what breaks first?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Connect hybrid fan-out and event processing to your message-broker and event-driven platform experience. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 22 of 28

Q22. Design Netflix-style global video streaming

WHY THEY ASK IT

Tests ingest and encoding pipelines, global content delivery, adaptive bitrate, rights enforcement, playback quality, and the economics of massive egress.

CLARIFYING QUESTIONS TO ASK

- Video on demand only, or live events too?
- Regions, concurrent streams, and supported devices?
- Startup-time and rebuffering service-level objectives?
- Resolution, codec, digital-rights-management, and download requirements?
- Content licensing and regional availability rules?

SCALE ESTIMATION

Assume 250M subscribers, 50M concurrent streams at peak, and 5 Mbps average delivered bitrate. That is roughly 250 Tbps of edge traffic, so origin capacity cannot serve viewers directly.

HIGH-LEVEL DESIGN

Studio ingest → QC → encode ladder → package/encrypt → origin
Viewer → control plane → manifest → CDN edge → adaptive player

Create immutable masters, encode device-aware bitrate ladders asynchronously, package and encrypt segments, and place them behind multi-tier CDNs. A control plane handles identity, entitlements, recommendations, manifests, and telemetry; the player adapts bitrate using buffer and network signals.

DEEP DIVES

- Encoding: per-title optimization saves bandwidth but increases compute and workflow complexity.
- CDN: owned edge capacity improves economics and control at scale; multi-CDN improves reach and resilience.
- Playback: startup latency, rebuffering, and visual quality must be balanced; degrade video before audio continuity.

LIKELY FOLLOW-UPS

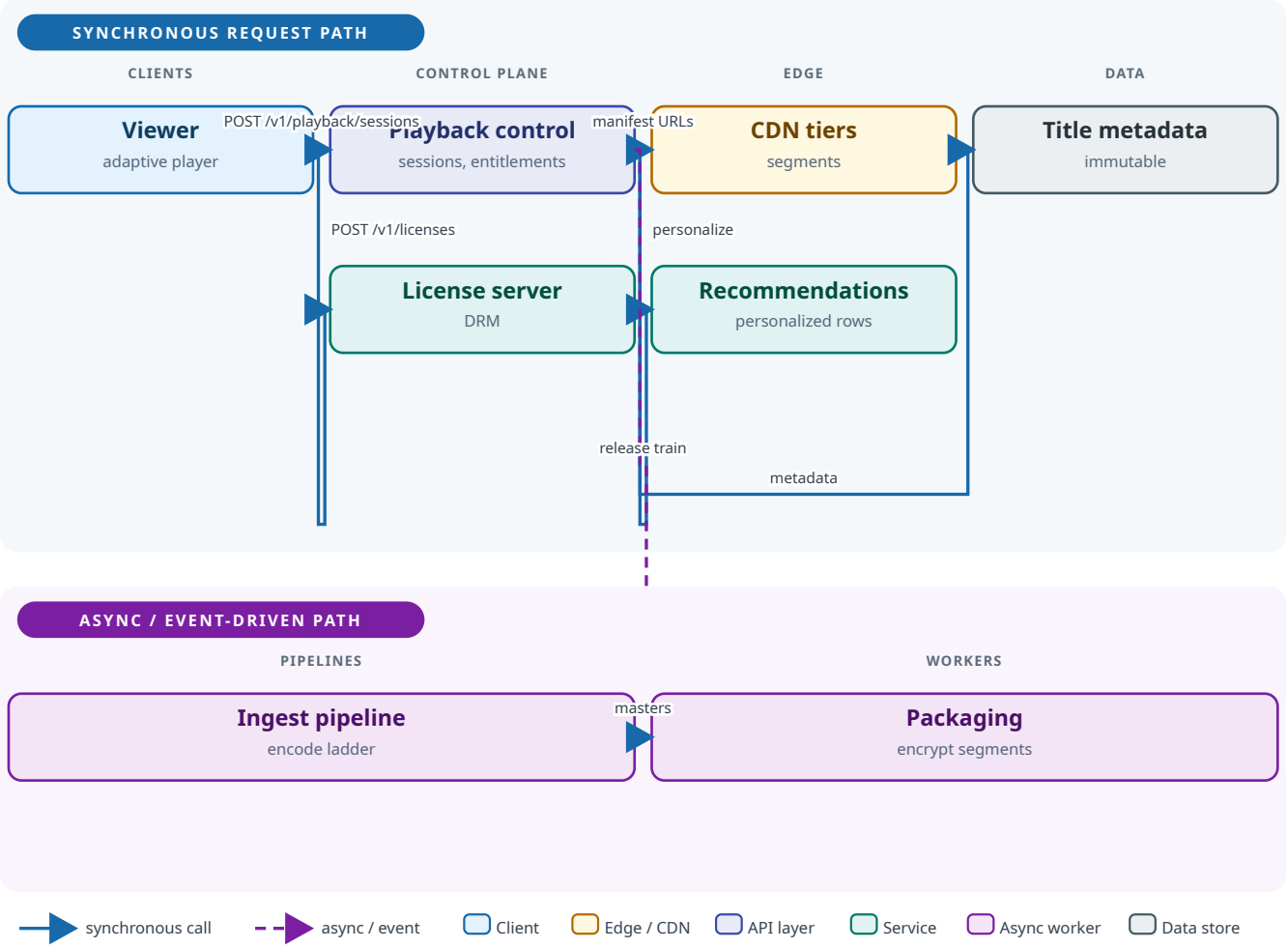
- A new series creates 50× regional demand—how do you prepare?
- How do you cut delivery cost in half?
- What happens when one CDN region fails?

TIE IT TO YOUR BACKGROUND

- Use your AWS modernization and FinOps results to frame compute-versus-egress trade-offs.
- Connect asynchronous encoding to your event-driven architecture and OpenShift automation.
- Use Marriott.com customer-facing experience to discuss global availability and user-impact metrics.

Architecture diagram — question 22

Netflix-scale streaming — control & data planes



The control plane decides; the data plane streams from the edge.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 22

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Central origin vs ISP-embedded edge appliances** → chosen embedded edge caches because video bytes are nearly all the traffic; serving from inside the ISP cuts transit cost and puts bytes milliseconds from the viewer.

2. **Decision: One encode vs per-title optimized ladders** → chosen per-title ladders because content complexity varies enormously; per-title encoding saves roughly a fifth of bandwidth at equal visual quality.

3. **Decision: Server-driven vs client-driven adaptive streaming** → chosen client-driven (DASH/HLS) because the client best knows its buffer health and measured bandwidth; the server stays stateless and any edge can serve any segment.

4. **Decision: Stateful streaming sessions vs stateless segment serving** → chosen stateless because stateful sessions complicate failover and scaling; the manifest plus independent segment fetches let clients hop edges mid-stream.

5. **Decision: Pull-fill vs predictive pre-positioning** → chosen predictive pre-positioning for hot titles because release-night pull-fill would crush origins; viewing models push likely content to edges overnight when transit is cheap.

End-to-end flows and API contracts — question 22

End-to-end request flows

JOURNEY 1 — VIEWER STARTS WATCHING A TITLE

1. The app POSTs to /v1/playback/sessions; the control plane checks identity and regional rights.
2. The session returns a manifest URL, DRM license URL, and device-bound session token.
3. The player fetches the manifest listing the bitrate ladder; segments live on the CDN.
4. It acquires the DRM license; keys are wrapped for the device's CDM.
5. Playback starts at a conservative bitrate and steps up as buffer and throughput allow.
6. Segments stream from the nearest edge; misses cascade edge to shield to origin.
7. QoE beacons (bitrate shifts, rebuffers) fire async; a failing edge fails over via the manifest.

JOURNEY 2 — NEW TITLE INGESTED AND PUBLISHED

1. The studio drops the mezzanine master; POST /v1/ingest/titles registers it and starts QC.
2. Automated QC checks integrity, captions, and rights metadata; failures return a report.
3. The encode farm builds the device-aware ladder in parallel: mobile, TV, HDR variants.
4. Segments are packaged, encrypted with per-title keys, and pushed to origin.
5. The control plane publishes availability windows per region, honoring licensing rights.
6. Pre-warming pushes first segments of anticipated hits to edges before release.
7. A release gate flips the title playable; the pipeline is idempotent per title version.

API contracts

POST `/v1/playback/sessions` Start playback session (control plane)

Auth: User OAuth · Content-Type: application/json

```
→ {"title_id":"t_7712","device_id":"tv_living","drm":"widevine"}
← 201 {"session_id":"s_441","manifest_url":"https://cdn.example.com/t/t_7712/master.mpd"}
```

POST `/v1/licenses` Acquire DRM license

Auth: Session token · Content-Type: application/octet-stream

```
→ <CDM license challenge bytes>
← 200 <license response bytes, bound to device CDM>
```

POST `/v1/telemetry/qoe` Player quality telemetry (async)

Auth: Session token · Content-Type: application/json

```
→ {"session_id":"s_441","events":[{"t":48,"type":"rebuffer","ms":900}]}
← 202 {"accepted":1}
```

POST `/v1/ingest/titles` Register a title for processing

Auth: Studio service account · Content-Type: application/json

```
→ {"title_id":"t_7712","master_ref":"s3://ingest/t_7712/master.mov"}
← 202 {"ingest_id":"ig_221","status":"qc_queued"}
```

How I would answer question 22

MODEL ANSWER - AS THE INTERVIEWEE

Q22. Design Netflix-style global video streaming

I'll start by narrowing the problem before drawing architecture. For Design Netflix-style global video streaming, I would ask: Video on demand only, or live events too; Regions, concurrent streams, and supported devices; Startup-time and rebuffering service-level objectives; Resolution, codec, digital-rights-management, and download requirements? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 250M subscribers, 50M concurrent streams at peak, and 5 Mbps average delivered bitrate. That is roughly 250 Tbps of edge traffic, so origin capacity cannot serve viewers directly. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Studio ingest → QC → encode ladder → package/encrypt → origin Then Viewer → control plane → manifest → CDN edge → adaptive player. The end-to-end design is: Create immutable masters, encode device-aware bitrate ladders asynchronously, package and encrypt segments, and place them behind multi-tier CDNs. A control plane handles identity, entitlements, recommendations, manifests, and telemetry; the player adapts bitrate using buffer and network signals. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Encoding: per-title optimization saves bandwidth but increases compute and workflow complexity. The second is CDN: owned edge capacity improves economics and control at scale; multi-CDN improves reach and resilience. The third is Playback: startup latency, rebuffering, and visual quality must be balanced; degrade video before audio continuity. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "A new series creates 50× regional demand—how do you prepare?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Use your AWS modernization and FinOps results to frame compute-versus-egress trade-offs. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 23 of 28

Q23. Design Uber: real-time driver-rider matching

WHY THEY ASK IT

Tests geospatial indexing, real-time state, matching, trip state machines, surge pricing, reliability, and marketplace fairness under rapidly changing supply and demand.

CLARIFYING QUESTIONS TO ASK

- Rider request and driver matching only, or pricing, navigation, and payment too?
- Target cities, active drivers, and requests per second?
- Maximum match latency and pickup-distance objective?
- Can a driver receive multiple offers?
- How fresh must location be, and how is privacy handled?

SCALE ESTIMATION

Assume 20M trips/day (~230 average requests/s), 2M active drivers, driver location every four seconds (~500K updates/s), and 20× event peaks. Location writes dominate.

HIGH-LEVEL DESIGN

Driver GPS → location stream → geo index
Rider request → matching service → candidate drivers → offer
Trip events → state machine → pricing/payment/telemetry

Ingest driver locations into regional streams and an in-memory geospatial index partitioned by city and cell. The matcher expands nearby cells, scores candidates by estimated arrival time and marketplace rules, reserves one driver with a short lease, and records trip transitions in an authoritative state machine.

DEEP DIVES

- Location freshness: memory-first indexes are fast but ephemeral; replayable streams rebuild state after failure.
- Matching: greedy assignment is fast; batched optimization improves marketplace efficiency but adds latency.
- Surge: use aggregated supply-demand windows, smoothing, caps, and experiment guardrails to avoid unstable or unfair pricing.

LIKELY FOLLOW-UPS

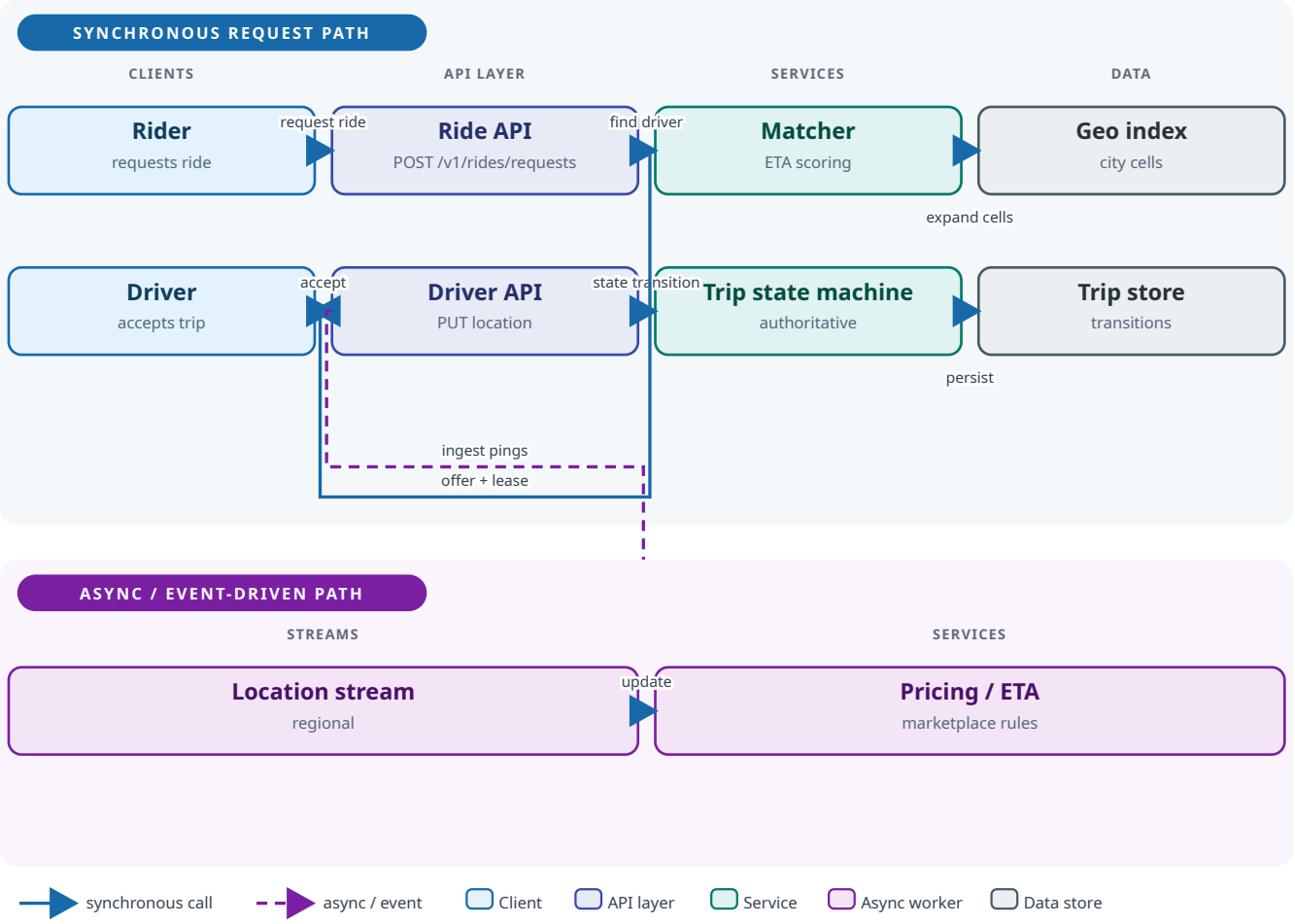
- A stadium empties and demand jumps 100×—what changes?
- How do you prevent double assignment?
- How do you recover a city after regional failure?

TIE IT TO YOUR BACKGROUND

- Map high-volume location streams to your event-driven and message-broker experience.
- Use platform ownership for a $500M business to discuss marketplace and customer trade-offs.
- Apply your global 80+ engineer leadership to regional cells, matching, pricing, and reliability ownership.

Architecture diagram — question 23

Uber — matching & trip lifecycle



Match on a live geo index; the trip state machine is the source of truth.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 23

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Driver polling vs push dispatch** → chosen push with batched offers because millions of drivers polling is a write storm; push targets only nearby eligible drivers and batches offers to avoid spamming.

2. **Decision: Exact optimal matching vs heuristic with time windows** → chosen heuristic with batched windows because exact optimal matching is intractable at city scale; greedy matching over short windows gets within a few percent of optimal in milliseconds.

3. **Decision: Naive lat/lng scan vs S2/geohash cell sharding** → chosen cell sharding because full scans don't scale; 'nearby drivers' becomes a set of cell lookups and drivers update only their cell, not a global index.

4. **Decision: Eventual vs strongly consistent trip state** → chosen strongly consistent trip lifecycle transitions because dispatching one rider to two drivers is a correctness failure; ETAs and positions stay eventual, but trip state moves under ACID transitions.

5. **Decision: Synchronous pricing vs async** → chosen synchronous with cached surge multipliers because the rider must see a price before confirming; surge multipliers refresh on a slower async loop so pricing never blocks on a recompute.

End-to-end flows and API contracts — question 23

End-to-end request flows

JOURNEY 1 — RIDER REQUESTS A RIDE

1. The app POSTs pickup, dropoff, and product to the regional matching service.
2. The matcher queries the in-memory geo index (city, then geohash cell) for nearby drivers.
3. It expands cells until enough candidates exist, scoring by ETA, rating, and marketplace rules.
4. The top driver gets an offer with a short acceptance lease and is soft-reserved.
5. On accept, the trip flips REQUESTED to ACCEPTED in the authoritative trip store.
6. An expired lease cascades to the next candidate; the rider sees transparent retries.
7. Surge is quoted upfront from local supply/demand; the rider accepts the fare before dispatch.

JOURNEY 2 — TRIP LIFECYCLE TO PAYMENT

1. Arrival and ride-start move the trip through ARRIVED to IN_PROGRESS as state-machine events.
2. Location streams at higher fidelity for route, safety, and fare computation.
3. Tolls, distance, and time accumulate; material deviations trigger fare review.
4. At dropoff the trip COMPLETES; pricing finalizes the fare and charges the rider.
5. Driver earnings update; payouts batch on the driver's schedule.
6. Both sides rate; disputes attach to the immutable trip event log, the source of truth.
7. Cancellation follows policy: free in the grace window, then a fee with driver compensation.

API contracts

POST `/v1/rides/requests` Request a ride with upfront fare

Auth: Rider OAuth · Content-Type: application/json · Idempotency-Key: <uuid>

```
→ {"pickup":{"lat":35.2271,"lng":-80.8431},"dropoff":{"lat":35.2031,"lng":-80.8392}}
← 201 {"ride_id":"r_55120","status":"matching","fare":{"amount":18.40}}
```

GET `/v1/rides/{id}` Ride status plus live driver location

Auth: Rider OAuth

```
→ —
← 200 {"ride_id":"r_55120","status":"accepted","driver":{"name":"Alex","rating":4.96}}
```

POST `/v1/rides/{id}/accept` Driver accepts the offer (lease)

Auth: Driver OAuth · Content-Type: application/json

```
→ {"offer_id":"of_991","lease_token":"lt_..."}
← 200 {"ride_id":"r_55120","status":"accepted"} · 409 if lease expired
```

PUT `/v1/drivers/location` Stream driver GPS (high frequency)

Auth: Driver OAuth · Content-Type: application/json

```
→ {"lat":35.2275,"lng":-80.8435,"heading":180,"speed":11.2}
← 202 {"indexed":true}
```

How I would answer question 23

MODEL ANSWER - AS THE INTERVIEWEE

Q23. Design Uber: real-time driver-rider matching

I'll start by narrowing the problem before drawing architecture. For Design Uber: real-time driver-rider matching, I would ask: Rider request and driver matching only, or pricing, navigation, and payment too; Target cities, active drivers, and requests per second; Maximum match latency and pickup-distance objective; Can a driver receive multiple offers? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 20M trips/day (~230 average requests/s), 2M active drivers, driver location every four seconds (~500K updates/s), and 20× event peaks. Location writes dominate. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Driver GPS → location stream → geo index Then Rider request → matching service → candidate drivers → offer Then Trip events → state machine → pricing/payment/telemetry. The end-to-end design is: Ingest driver locations into regional streams and an in-memory geospatial index partitioned by city and cell. The matcher expands nearby cells, scores candidates by estimated arrival time and marketplace rules, reserves one driver with a short lease, and records trip transitions in an authoritative state machine. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Location freshness: memory-first indexes are fast but ephemeral; replayable streams rebuild state after failure. The second is Matching: greedy assignment is fast; batched optimization improves marketplace efficiency but adds latency. The third is Surge: use aggregated supply-demand windows, smoothing, caps, and experiment guardrails to avoid unstable or unfair pricing. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "A stadium empties and demand jumps 100×—what changes?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Map high-volume location streams to your event-driven and message-broker experience. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 24 of 28

Q24. Design Google Docs collaborative editing

WHY THEY ASK IT

Tests real-time collaboration, concurrency control, operational transformation versus conflict-free replicated data types, presence, offline edits, and convergence.

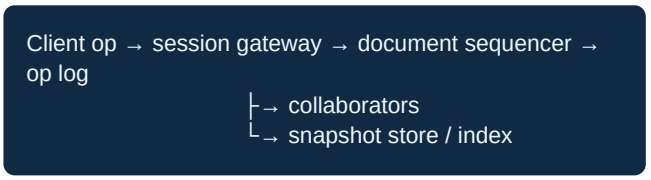
CLARIFYING QUESTIONS TO ASK

- Plain text first, or rich documents, comments, and images?
- Maximum simultaneous editors per document?
- Offline editing and multi-device support?
- Latency target for remote keystrokes?
- Version history, permissions, and data-residency needs?

SCALE ESTIMATION

Assume 100M daily users, 10M concurrent sessions, 5 operations/s per active editor, and 100 editors for hot documents. Operation traffic can reach tens of millions per second at peak.

HIGH-LEVEL DESIGN



Route a document to a collaboration shard, assign operations a document sequence, transform or merge concurrent edits, persist an append-only log, and periodically compact snapshots. Presence travels on an ephemeral channel; permissions and durable content use authoritative stores.

DEEP DIVES

- OT versus CRDT: operational transformation offers compact centralized control; CRDTs converge naturally offline but carry metadata and implementation cost.
- Recovery: clients reconnect with a cursor, receive a snapshot plus later operations, and resend idempotent pending edits.
- Hot documents: shard ancillary features, bound presence fan-out, and isolate expensive indexing from the editing path.

LIKELY FOLLOW-UPS

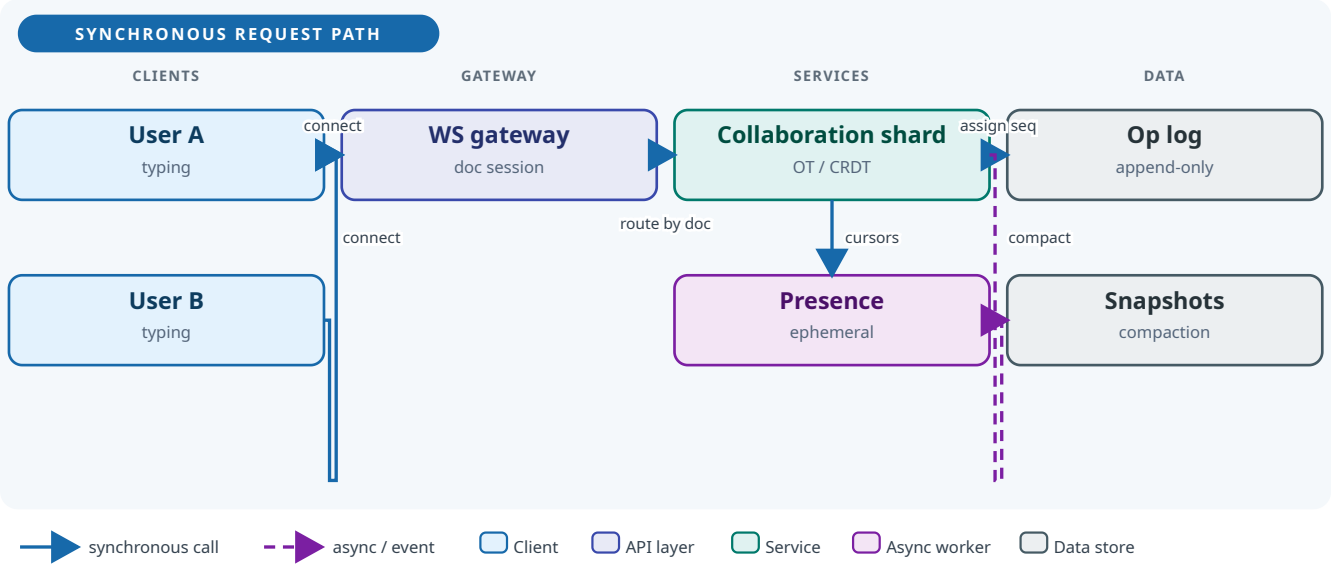
- A user edits offline for a week—how do changes merge?
- What breaks at 10,000 viewers?
- How do permission revocations reach active sessions?

TIE IT TO YOUR BACKGROUND

- Relate operation logs, replay, and idempotency to your event-driven platforms.
- Use seven-system integration experience to discuss clear contracts and source-of-truth boundaries.
- Be candid that collaborative editing is a transfer of distributed-systems principles, not claimed product ownership.

Architecture diagram — question 24

Google Docs — collaborative editing



One shard owns the document sequence; everything else is derived.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 24

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1 **Decision: Operational transform vs CRDTs** → chosen operational transform with a central server because OT gives intention-preserving merges with the server as the ordering authority; it matches the permission model where the server already arbitrates access.

2 **Decision: Word/commit sync vs keystroke-level ops** → chosen keystroke-level ops because collaboration only feels live with sub-100ms propagation; ops are tiny, transformable, and cheap to broadcast.

3 **Decision: Single document server vs sharded by document** → chosen sharded by document ID with sticky routing because one server cannot hold every open document; sticky routing keeps each document's OT authority on exactly one node.

4 **Decision: Polling vs WebSocket** → chosen WebSocket because polling at keystroke rates is prohibitive; one socket multiplexes operations, presence, and comments with minimal overhead.

5 **Decision: Snapshots only vs operation log with snapshots** → chosen operation log with periodic snapshots because the log is the source of truth for replay and undo; snapshots bound recovery time so a client never replays from genesis.

End-to-end flows and API contracts — question 24

End-to-end request flows

JOURNEY 1 — TWO USERS TYPE IN THE SAME DOCUMENT

1. Each client opens a WebSocket to the session gateway and joins the document's shard.
2. User A's keystroke becomes an op with a revision vector sent to the sequencer.
3. The sequencer assigns the next document sequence number and appends to the op log.
4. The op broadcasts to all collaborators, including B.
5. B's concurrent op is transformed against A's (OT) or merged by CRDT rules.
6. Both clients converge to identical text; cursors update without jumping.
7. Snapshots compact the log so late joiners bootstrap from snapshot plus tail.

JOURNEY 2 — USER EDITS OFFLINE, THEN RECONNECTS

1. Offline edits queue in the local outbox against the last known snapshot.
2. On reconnect, the client replays queued ops to the sequencer.
3. The sequencer interleaves them with ops that arrived meanwhile, transforming as needed.
4. The client receives missed ops since its snapshot and applies the merged result.
5. Every keystroke survives as an op; the merge policy only decides ordering.
6. Presence resumes over the ephemeral channel once the socket is back.
7. Revoked collaborators have sockets closed; their queued ops are rejected.

API contracts

WS `/v1/docs/{id}/session` Collaborative editing socket

Auth: User OAuth (doc ACL-checked) · Sec-WebSocket-Protocol: docs.collab.v1

```
→ → {"type":"op","op_id":"op_a1","base_rev":412,"op":{"insert":{"pos":5,"text":"hi"}}}
← ← {"type":"op_ack","op_id":"op_a1","seq":413}
```

POST `/v1/docs` Create a document

Auth: User OAuth · Content-Type: application/json

```
→ {"title":"Q4 planning","folder_id":"f_11"}
← 201 {"doc_id":"d_123","revision":0}
```

GET `/v1/docs/{id}/ops` Fetch ops since a revision

Auth: User OAuth

```
→ ?since=400&limit=1000
← 200 {"ops":[{"seq":401,"op":{...}}],"server_revision":414}
```

GET `/v1/docs/{id}/snapshot` Compacted snapshot for bootstrap

Auth: User OAuth

```
→ —
← 200 {"doc_id":"d_123","revision":400,"snapshot_seq":400}
```

How I would answer question 24

MODEL ANSWER - AS THE INTERVIEWEE

Q24. Design Google Docs collaborative editing

I'll start by narrowing the problem before drawing architecture. For Design Google Docs collaborative editing, I would ask: Plain text first, or rich documents, comments, and images; Maximum simultaneous editors per document; Offline editing and multi-device support; Latency target for remote keystrokes? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 100M daily users, 10M concurrent sessions, 5 operations/s per active editor, and 100 editors for hot documents. Operation traffic can reach tens of millions per second at peak. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Client op → session gateway → document sequencer → op log Then ├→ collaborators Then └→ snapshot store / index. The end-to-end design is: Route a document to a collaboration shard, assign operations a document sequence, transform or merge concurrent edits, persist an append-only log, and periodically compact snapshots. Presence travels on an ephemeral channel; permissions and durable content use authoritative stores. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is OT versus CRDT: operational transformation offers compact centralized control; CRDTs converge naturally offline but carry metadata and implementation cost. The second is Recovery: clients reconnect with a cursor, receive a snapshot plus later operations, and resend idempotent pending edits. The third is Hot documents: shard ancillary features, bound presence fan-out, and isolate expensive indexing from the editing path. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "A user edits offline for a week—how do changes merge?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Relate operation logs, replay, and idempotency to your event-driven platforms. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 25 of 28

Q25. Design a distributed cron and job scheduler

WHY THEY ASK IT

Tests durable scheduling, leases, idempotency, retries, fairness, clock semantics, and operability. A strong answer separates control-plane intent from execution-plane work.

CLARIFYING QUESTIONS TO ASK

- One-time and recurring jobs?
- Minimum scheduling granularity and acceptable lateness?
- Exactly-once effect or at-least-once execution?
- Job duration, payload size, dependencies, and tenant count?
- How are time zones, daylight saving, and missed runs handled?

SCALE ESTIMATION

Assume 1B schedules, 100M executions/day (~1.2K average; 100× spikes on minute boundaries), and jobs from seconds to hours. Hot timestamps and retry storms dominate.

HIGH-LEVEL DESIGN

```
Schedule API → metadata store → time buckets
Scheduler shards → ready queue → leased workers → result log
                                  └→ retry / dead-letter queue
```

Store schedules durably, index next-run time in coarse buckets, and let partition leaders move due work into ready queues. Workers claim leases, heartbeat long work, record idempotency keys, and write outcomes. The scheduler advances recurrence separately from execution success.

DEEP DIVES

- Semantics: exactly-once execution is unrealistic across crashes; target at-least-once delivery with idempotent effects.
- Time: evaluate recurrence in the owner's time zone, persist the chosen instant, and define daylight-saving behavior explicitly.
- Fairness: tenant queues, weighted capacity, concurrency caps, and reserved critical lanes prevent noisy-neighbor starvation.

LIKELY FOLLOW-UPS

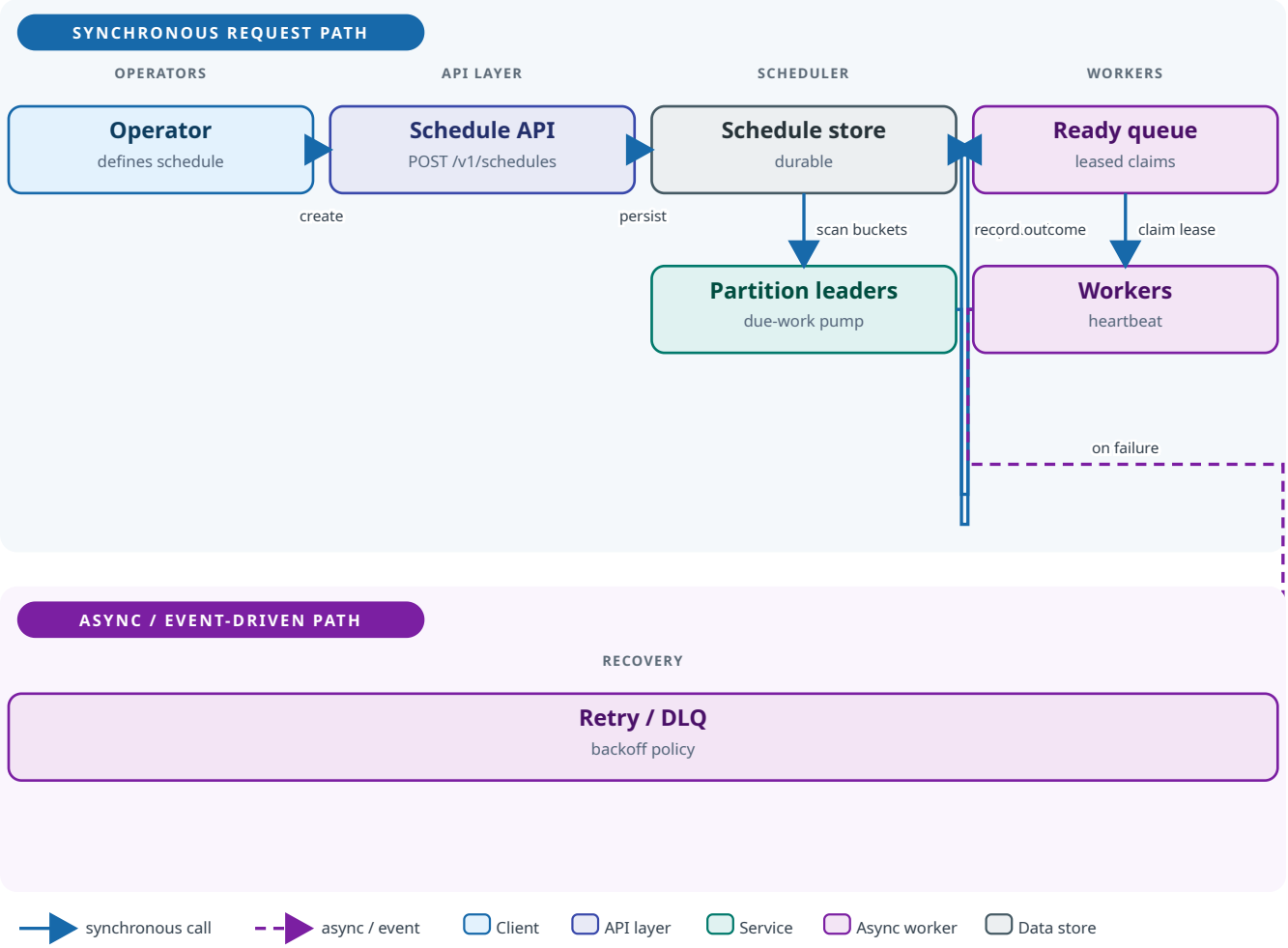
- A region fails at midnight UTC—how do you avoid duplicates?
- How do you handle jobs that run longer than their interval?
- How would you support workflow dependencies?

TIE IT TO YOUR BACKGROUND

- Use message brokers and OpenShift automation that eliminated 99% of manual work as the closest operational parallel.
- Connect 90% manual-effort reduction to durable scheduling and exception handling.
- Apply CI/CD and DevSecOps experience to safe worker rollout, tenant isolation, and auditability.

Architecture diagram — question 25

Distributed cron & job scheduler



Scheduling and execution are separate concerns with separate failure modes.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 25

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Single leader scheduler vs lock-per-job** → chosen lock-per-job with leases because a single leader is a bottleneck and a single point of failure; per-job leases let scheduling scale horizontally and survive any one scheduler dying.

2. **Decision: Exactly-once vs at-least-once execution** → chosen at-least-once with idempotent jobs because exactly-once across crashes is impossible; jobs carry run IDs for dedup and must be safe to retry.

3. **Decision: Interval vs cron expression with timezones** → chosen cron expressions resolved to UTC instants because business schedules are calendar-based; the scheduler owns timezone and DST resolution so jobs don't double-fire on clock changes.

4. **Decision: In-memory schedule vs durable replicated store** → chosen durable store because losing the schedule on restart silently drops jobs; the store is the source of truth that schedulers watch, and any scheduler can rebuild from it.

5. **Decision: Trigger inline vs queued execution** → chosen queued because job runtimes vary from milliseconds to hours; the queue decouples scheduling decisions from execution capacity and absorbs bursts.

End-to-end flows and API contracts — question 25

End-to-end request flows

JOURNEY 1 — A SCHEDULED JOB FIRES AND RUNS

1. The owner creates the schedule with a cron expression, payload, and retry policy.
2. The scheduler buckets the next-run time; the partition leader moves due jobs to the ready queue.
3. A worker claims the job with a lease; the claim records an idempotency key for the run.
4. The worker executes the payload and heartbeats to extend the lease on long work.
5. On success it writes the outcome to the result log; recurrence advances regardless of outcome.
6. One leader per partition and one lease per run structurally prevent duplicates.
7. Schedule edits apply to the next computed run; in-flight runs finish under the old definition.

JOURNEY 2 — WORKER CRASHES MID-RUN

1. Heartbeats stop; the lease expires without renewal.
2. The leader returns the run to the ready queue with the attempt counter incremented.
3. Another worker claims it; the idempotency key dedupes side effects.
4. Exhausted retries move the run to the dead-letter queue with full failure history.
5. Recurrence still advances; a failed 2 AM run never shifts the 3 AM run.
6. The owner gets an alert with run id, logs pointer, and the DLQ entry for manual replay.
7. Due checks use coarse buckets plus a monotonic sequencer, never raw wall-clock equality.

API contracts

POST `/v1/schedules` Create a schedule

Auth: Service account / user OAuth · Content-Type: application/json

```
→ {"name":"nightly-etl","cron":"0 2 * * *","tz":"America/New_York","retry":{"max_attempts":3}}
← 201 {"schedule_id":"sch_881","next_run":"2026-09-19T02:00:00-04:00"}
```

POST `/v1/schedules/{id}/trigger` Manually trigger an ad-hoc run

Auth: Service account / user OAuth · Content-Type: application/json

```
→ {"args":{"date":"2026-09-17"}}
← 202 {"run_id":"run_552","status":"queued"}
```

GET `/v1/runs/{id}` Inspect a run and its attempts

Auth: Service account / user OAuth

```
→ —
← 200 {"run_id":"run_551","status":"succeeded","attempts":1,"duration_s":482}
```

POST `/internal/v1/runs/{id}/heartbeat` Extend a run lease (internal)

Auth: mTLS worker identity · Content-Type: application/json

```
→ {"lease_id":"lse_311","extend_s":300}
← 200 {"lease_id":"lse_311"}
```

How I would answer question 25

MODEL ANSWER - AS THE INTERVIEWEE

Q25. Design a distributed cron and job scheduler

I'll start by narrowing the problem before drawing architecture. For Design a distributed cron and job scheduler, I would ask: One-time and recurring jobs; Minimum scheduling granularity and acceptable lateness; Exactly-once effect or at-least-once execution; Job duration, payload size, dependencies, and tenant count? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 1B schedules, 100M executions/day (~1.2K average; 100× spikes on minute boundaries), and jobs from seconds to hours. Hot timestamps and retry storms dominate. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Schedule API → metadata store → time buckets Then Scheduler shards → ready queue → leased workers → result log Then └→ retry / dead-letter queue. The end-to-end design is: Store schedules durably, index next-run time in coarse buckets, and let partition leaders move due work into ready queues. Workers claim leases, heartbeat long work, record idempotency keys, and write outcomes. The scheduler advances recurrence separately from execution success. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Semantics: exactly-once execution is unrealistic across crashes; target at-least-once delivery with idempotent effects. The second is Time: evaluate recurrence in the owner's time zone, persist the chosen instant, and define daylight-saving behavior explicitly. The third is Fairness: tenant queues, weighted capacity, concurrency caps, and reserved critical lanes prevent noisy-neighbor starvation. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "A region fails at midnight UTC—how do you avoid duplicates?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Use message brokers and OpenShift automation that eliminated 99% of manual work as the closest operational parallel. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 26 of 28

Q26. Design an API gateway

WHY THEY ASK IT

Tests request routing, authentication, authorization, rate limiting, observability, configuration safety, and how to keep a shared control point from becoming a bottleneck.

CLARIFYING QUESTIONS TO ASK

- North-south public traffic, internal east-west traffic, or both?
- Protocols, request rates, payload sizes, and latency budget?
- Authentication and tenant-isolation requirements?
- Can routes and policies change dynamically?
- Transformation, caching, and service-mesh boundaries?

SCALE ESTIMATION

Assume 5M requests/s globally, 10K services/routes, and less than 5 ms gateway overhead at p99. Configuration propagation must be fast without making the data plane depend on the control plane.

HIGH-LEVEL DESIGN

```
Config/policy → control plane → signed snapshots
Client → edge gateway → auth/rate limit/router → service
                      └→ metrics/traces/audit
```

Run stateless regional gateways with immutable signed configuration snapshots. Authenticate early, authorize against route policy, apply local or leased quotas, route with health-aware discovery, and emit sampled telemetry. Keep sensitive transformations in explicit plugins with strict budgets.

DEEP DIVES

- Control versus data plane: gateways continue safely on the last good config when control services fail.
- Extensibility: plugins accelerate teams but threaten latency and reliability; sandbox, version, and budget them.
- Observability: propagate trace context, record policy decisions, and protect high-cardinality telemetry costs.

LIKELY FOLLOW-UPS

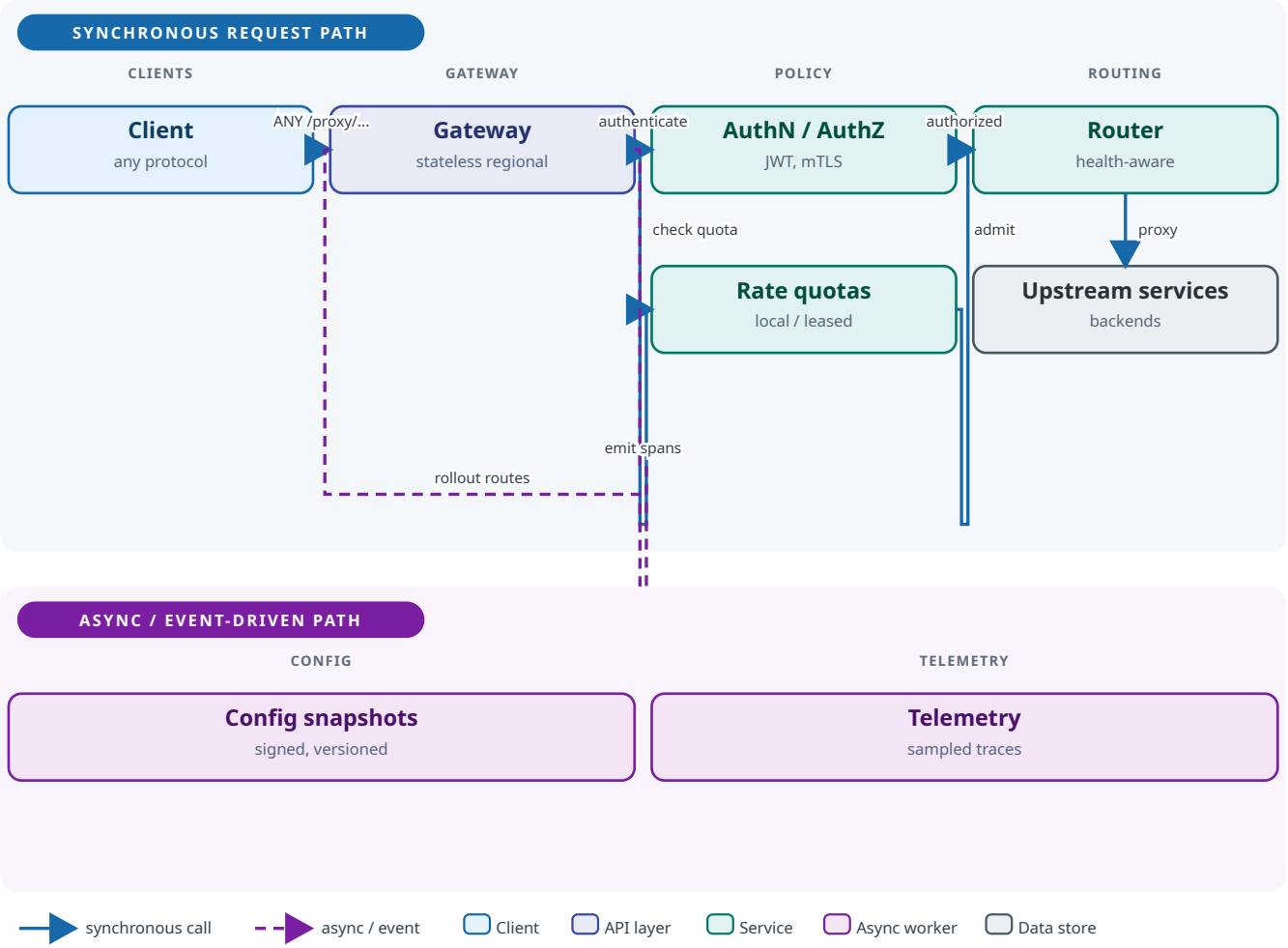
- A bad config reaches production—how do you contain it?
- How do you support 10× traffic without 10× cost?
- Which policies belong in the gateway versus services?

TIE IT TO YOUR BACKGROUND

- Lead with your API-first platforms and seven-system partner onboarding.
- Use AT&T OAuth 2.0, JWT, SSO, 2FA, and step-up-auth work for identity policy.
- Tie 40% faster APIs and 30% FinOps savings to gateway latency and telemetry-cost budgets.

Architecture diagram — question 26

API gateway — the front door



Authenticate and authorize before proxying; never let the gateway become stateful.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 26

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Centralized gateway vs sidecar per service** → chosen centralized edge gateway because auth, rate limiting, and routing in one place prevent per-service drift; sidecars add a hop and duplicated policy to every service.

2. **Decision: Auth-service call per request vs locally verified JWTs** → chosen locally verified JWTs with key rotation because a per-request auth call taxes every API call; JWTs verify in microseconds at the edge and rotation bounds key exposure.

3. **Decision: Fixed-window vs token-bucket rate limiting** → chosen token bucket per client and route because it absorbs legitimate bursts while capping sustained abuse; fixed windows allow 2× bursts at boundaries.

4. **Decision: Validate at the service vs at the gateway** → chosen gateway schema validation because rejecting malformed traffic at the edge protects services from junk load; services still own business-rule validation.

5. **Decision: Static config files vs dynamic control plane** → chosen dynamic control plane because static configs need a redeploy for every route change; the control plane pushes versioned updates with canary and rollback.

End-to-end flows and API contracts — question 26

End-to-end request flows

JOURNEY 1 — CLIENT REQUEST THROUGH THE GATEWAY

1. The client hits the edge; TLS terminates and the request enters the stateless gateway.
2. Authentication runs first: JWT, API key, or mTLS; unauthenticated traffic is rejected early.
3. The router matches the path against the signed route table from the config snapshot.
4. Authorization checks the route policy for required scopes or roles.
5. The rate limiter checks the local bucket, leasing from regional shards only when empty.
6. Plugins run within strict budgets: header injection, schema validation, PII redaction.
7. The gateway picks a healthy backend via discovery, applies timeout/retry budgets, and proxies; telemetry emits async.

JOURNEY 2 — OPERATOR ROLLS OUT A ROUTE CHANGE

1. The operator POSTs the route definition; the admin API validates before accepting.
2. The control plane builds a new immutable snapshot, signs it, and records the version.
3. Gateways load the new version atomically; none ever serves a half-applied config.
4. A canary region takes it first; error and latency diffs are compared automatically.
5. On regression, republishing the prior snapshot rolls back; it is a version-pointer move.
6. Gateways keep serving the last good snapshot if the control plane is unreachable.

API contracts

ANY `/proxy/{service}/{path*}` Proxied data-plane request (example)

Auth: Bearer JWT / API key / mTLS · X-Request-Id: <uuid> (injected if absent)

```
→ GET /proxy/billing/v1/invoices?status=open
← 200 <backend response> · X-RateLimit-Remaining: 812
```

POST `/admin/v1/routes` Define or update a route

Auth: Admin token (RBAC: gateway-admin) · Content-Type: application/json

```
→ {"path_prefix":"/v1/billing","service":"billing","auth":"jwt","rate_limit":{"rps":500}}
← 201 {"route_id":"rt_771","snapshot":"cfg_2042","rollout":"canary"}
```

GET `/admin/v1/config/versions` List signed config snapshots

Auth: Admin token

```
→ —
← 200 {"serving":"cfg_2041","canary":"cfg_2042"}
```

POST `/admin/v1/config/rollback` Roll back to a prior snapshot

Auth: Admin token (RBAC: gateway-admin) · Content-Type: application/json

```
→ {"version":"cfg_2040","reason":"elevated 5xx on /v1/billing"}
← 202 {"serving":"cfg_2040","rollout":"global"}
```

How I would answer question 26

MODEL ANSWER - AS THE INTERVIEWEE

Q26. Design an API gateway

I'll start by narrowing the problem before drawing architecture. For Design an API gateway, I would ask: North-south public traffic, internal east-west traffic, or both; Protocols, request rates, payload sizes, and latency budget; Authentication and tenant-isolation requirements; Can routes and policies change dynamically? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 5M requests/s globally, 10K services/routes, and less than 5 ms gateway overhead at p99. Configuration propagation must be fast without making the data plane depend on the control plane. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Config/policy → control plane → signed snapshots Then Client → edge gateway → auth/rate limit/router → service Then └→ metrics/traces/audit. The end-to-end design is: Run stateless regional gateways with immutable signed configuration snapshots. Authenticate early, authorize against route policy, apply local or leased quotas, route with health-aware discovery, and emit sampled telemetry. Keep sensitive transformations in explicit plugins with strict budgets. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Control versus data plane: gateways continue safely on the last good config when control services fail. The second is Extensibility: plugins accelerate teams but threaten latency and reliability; sandbox, version, and budget them. The third is Observability: propagate trace context, record policy decisions, and protect high-cardinality telemetry costs. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "A bad config reaches production—how do you contain it?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Lead with your API-first platforms and seven-system partner onboarding. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 27 of 28

Q27. Design iMessage end-to-end encrypted multi-device sync

WHY THEY ASK IT

Tests key management, trusted device enrollment, asynchronous encrypted delivery, multi-device consistency, metadata privacy, recovery, and Apple-quality behavior under offline conditions.

CLARIFYING QUESTIONS TO ASK

- One-to-one first, or groups, attachments, edits, and reactions?
- How many devices per account, and must all receive history?
- How is a new device enrolled and verified?
- What recovery or encrypted backup is allowed?
- Which metadata may the service retain, and for how long?

SCALE ESTIMATION

Assume 1B users, 100 messages/user/day (~1.2M average messages/s), three devices/user, and 10× peak. Multi-device envelope fan-out multiplies work; media follows a separate encrypted object path.

HIGH-LEVEL DESIGN

```
Device keys → directory of public prekeys
Sender → encrypt per recipient device → relay queues → devices
Attachment → client encryption → object store → opaque pointer
```

Each trusted device owns hardware-protected keys and publishes signed prekeys. The sender establishes sessions and encrypts an envelope for each recipient device. Regional relays hold bounded ciphertext until acknowledged; push carries only opaque wake data. Device enrollment requires an existing trusted device or a recovery policy with explicit risk.

DEEP DIVES

- Enrollment: convenience competes with account-takeover resistance; require strong authentication, user-visible approval, and key transparency.
- Offline delivery: per-device cursors and idempotency preserve order without server plaintext; expired ciphertext follows a clear retention rule.
- Groups: sender keys reduce encryption cost, but membership changes trigger rekeying and transparent security events.

LIKELY FOLLOW-UPS

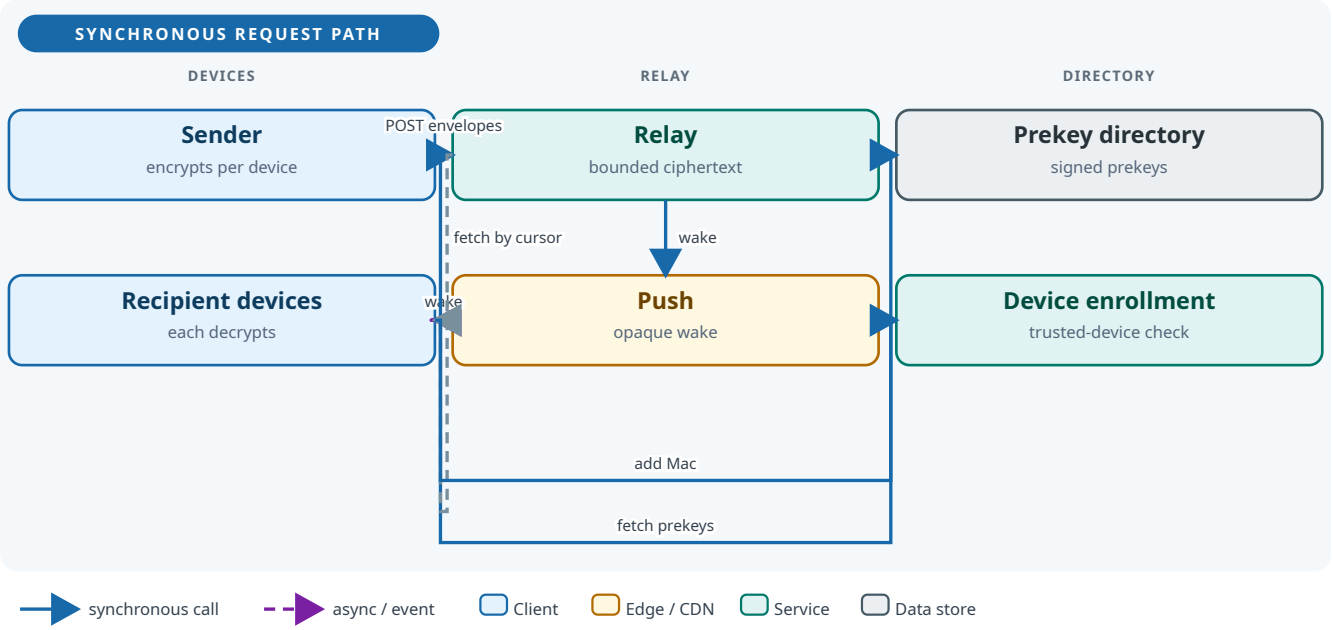
- A new device needs old history—what options preserve privacy?
- How do you revoke a stolen device immediately?
- How can users report abuse when the server cannot read messages?

TIE IT TO YOUR BACKGROUND

- Use AT&T mobile-security architecture—2FA, step-up auth, JWT, TLS, and device trust—as your strongest foundation.
- Apply zero-trust, OWASP, and bug-bounty remediation to enrollment and privileged key-directory operations.
- Connect relay queues and idempotency to your event-driven platform background without claiming iMessage delivery.

Architecture diagram — question 27

iMessage — E2E multi-device sync



Every recipient device gets its own encrypted envelope; relays never see plaintext.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 27

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Shared user key vs per-device keys** → chosen per-device keys with a directory because a shared key cannot be revoked for one lost device; the directory maps devices to keys and revocation is per device.

2. **Decision: Server-side fan-out vs sender-side fan-out** → chosen sender-side fan-out because server-side fan-out would require the server to see plaintext or hold keys; the sender encrypts once per recipient device.

3. **Decision: iCloud backup restore vs live sync** → chosen live sync via push plus pull because backups are hours stale; live sync keeps every device within seconds without a full restore.

4. **Decision: Server-ordered vs client-ordered messages** → chosen client-side ordering with per-device sequence numbers because the server cannot see content to order it; clients merge per-device sequences deterministically.

5. **Decision: Scheduled rotation only vs rotation on membership change** → chosen rotation on every device add/remove plus scheduled rotation because a removed device must immediately lose access to future messages; scheduled rotation bounds long-term key exposure.

End-to-end flows and API contracts — question 27

End-to-end request flows

JOURNEY 1 — USER SENDS A MESSAGE TO A MULTI-DEVICE CONTACT

1. The sender fetches the recipient's public prekey bundles, one per trusted device.
2. It encrypts the message separately for each recipient device session.
3. The device POSTs the envelopes to the regional relay, keyed by recipient device inbox.
4. Push carries only an opaque wake token; no sender, snippet, or metadata.
5. Each device fetches by cursor, decrypts with hardware-protected keys, and renders.
6. The device acks its cursor; acknowledged envelopes become eligible for compaction.
7. Attachments encrypt client-side into blobs; envelopes carry only opaque blob pointers.

JOURNEY 2 — RECIPIENT ADDS A NEW MAC

1. Sign-in on the new Mac requires approval from an existing trusted device.
2. The Mac generates its keypair in hardware and publishes signed prekeys to the directory.
3. The trusted device countersigns, binding the Mac to the account trust chain.
4. History is not retroactively re-encrypted; the Mac receives new messages from enrollment on.
5. Senders discover the new bundle on their next directory fetch and include it.
6. A lost device is revoked; the directory marks its keys revoked and senders exclude it.
7. Full account recovery visibly resets device trust for contacts to re-verify.

API contracts

GET `/v1/directory/prekeys/{user}` Prekey bundles for all trusted devices

Auth: Device session token

```
→ —
← 200 {"user_id":"u_991","devices":[{"device_id":"dev_iphone","identity_key":"base64..."}]}
```

POST `/v1/relay/envelopes` Submit per-device encrypted envelopes

Auth: Device session token · Content-Type: application/json

```
→ {"envelopes":[{"to_device":"dev_iphone","message_id":"m_77a","ciphertext":"base64..."}]}
← 202 {"accepted":["m_77a"]}
```

POST `/v1/devices/enroll` Enroll a new trusted device

Auth: Approval signature from trusted device · Content-Type: application/json

```
→ {"device_id":"dev_mac2","identity_key":"base64...","approval":{"by_device":"dev_iphone"}}
← 201 {"device_id":"dev_mac2","status":"trusted"}
```

POST `/v1/attachments` Upload client-encrypted attachment blob

Auth: Device session token · Content-Type: application/octet-stream

```
→ <encrypted bytes> · X-Blob-Sha256: e3b0...
← 201 {"blob_id":"b_221","bytes":1844220}
```

How I would answer question 27

MODEL ANSWER - AS THE INTERVIEWEE

Q27. Design iMessage end-to-end encrypted multi-device sync

I'll start by narrowing the problem before drawing architecture. For Design iMessage end-to-end encrypted multi-device sync, I would ask: One-to-one first, or groups, attachments, edits, and reactions; How many devices per account, and must all receive history; How is a new device enrolled and verified; What recovery or encrypted backup is allowed? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 1B users, 100 messages/user/day (~1.2M average messages/s), three devices/user, and 10× peak. Multi-device envelope fan-out multiplies work; media follows a separate encrypted object path. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Device keys → directory of public prekeys Then Sender → encrypt per recipient device → relay queues → devices Then Attachment → client encryption → object store → opaque pointer. The end-to-end design is: Each trusted device owns hardware-protected keys and publishes signed prekeys. The sender establishes sessions and encrypts an envelope for each recipient device. Regional relays hold bounded ciphertext until acknowledged; push carries only opaque wake data. Device enrollment requires an existing trusted device or a recovery policy with explicit risk. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Enrollment: convenience competes with account-takeover resistance; require strong authentication, user-visible approval, and key transparency. The second is Offline delivery: per-device cursors and idempotency preserve order without server plaintext; expired ciphertext follows a clear retention rule. The third is Groups: sender keys reduce encryption cost, but membership changes trigger rekeying and transparent security events. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "A new device needs old history—what options preserve privacy?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Use AT&T mobile-security architecture—2FA, step-up auth, JWT, TLS, and device trust—as your strongest foundation. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

Question 28 of 28

Q28. Design a Siri-style voice assistant request pipeline

WHY THEY ASK IT

Tests on-device versus cloud partitioning, streaming speech, latency budgets, privacy, model routing, tool execution, and graceful degradation.

CLARIFYING QUESTIONS TO ASK

- Simple commands, open-domain questions, or multi-step actions?
- What must work fully on-device?
- Wake-word, speech recognition, understanding, and response latency targets?
- Which actions require user confirmation?
- What audio or transcripts may be retained for improvement?

SCALE ESTIMATION

Assume 500M daily users, 20 requests/user/day (~116K average requests/s), 10× peak, and a 1–2 second perceived-response target. Streaming audio and model inference dominate latency and cost.

HIGH-LEVEL DESIGN

```
Mic → on-device wake/VAD → local ASR/intent or encrypted stream
Cloud gateway → ASR/LLM/router → tool sandbox → response
                                  └→ safety/evaluation telemetry
```

Detect wake and voice activity on-device, handle common private intents locally, and stream only when cloud capability is needed. A gateway applies identity, consent, routing, and budgets; models produce a plan; tools execute in a permissioned sandbox; responses stream back while sensitive actions require confirmation.

DEEP DIVES

- Partitioning: on-device processing improves latency/privacy and offline use; cloud models improve capability but add network, cost, and data risk.
- Latency: speculative endpointing and streaming reduce perceived delay, but incorrect early actions must never escape confirmation gates.
- Privacy and safety: minimize retention, separate evaluation samples, redact sensitive data, and make tool permissions explicit and revocable.

LIKELY FOLLOW-UPS

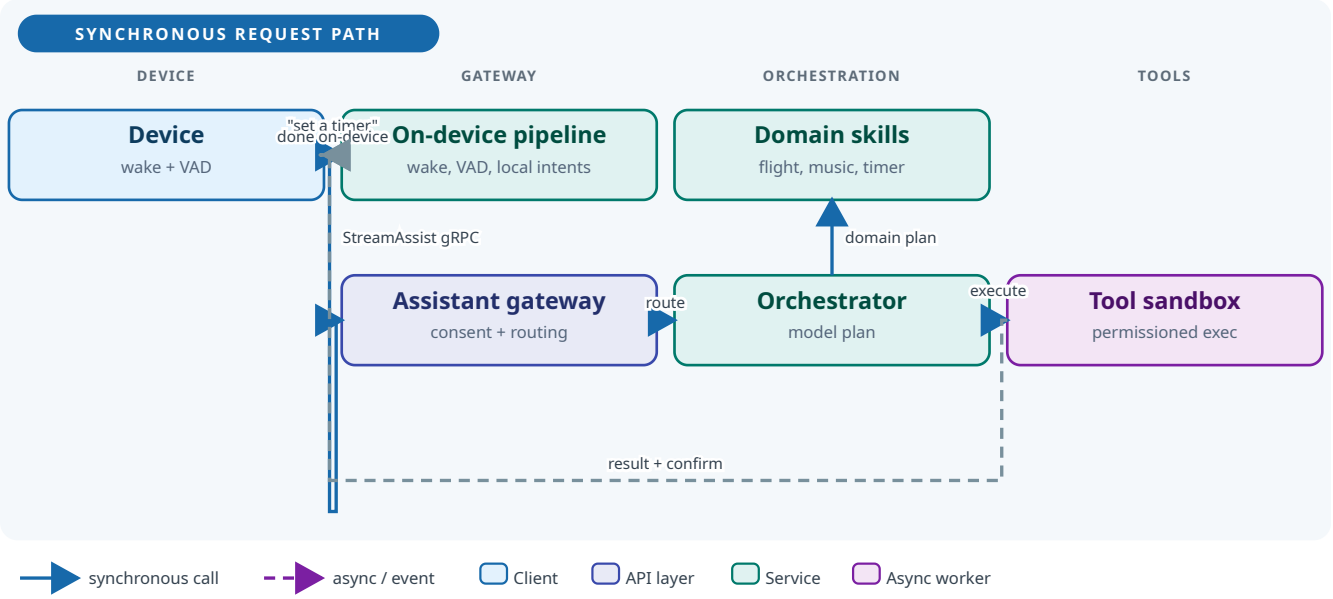
- The network disappears after the wake word—what still works?
- How do you cut inference cost in half?
- How do you prevent a model from executing an unsafe action?

TIE IT TO YOUR BACKGROUND

- Lead with your GenAI, agentic-AI, LLM-orchestration, and MLOps portfolio.
- Use intelligent document processing—hours to minutes—to show how you connect models to measurable workflow outcomes.
- Bring in AT&T identity/security work and FinOps savings for permission gates, model routing, and cost-per-successful-task.

Architecture diagram — question 28

Siri-style voice assistant pipeline



Default to on-device; escalate to cloud only when capability demands it.

> **Whiteboard tip:** narrate each arrow as you draw it — name the source of truth, the partition key, and where the path turns asynchronous.

Trade-offs — question 28

Key trade-offs — say the decision out loud

Interviewers score whether you see the decision, not whether you pick their favorite answer. For each one, name both options, state your default, and tie the reason to the requirement: latency, cost, consistency, reliability, or operability.

1. **Decision: Cloud for everything vs on-device first with cloud escalation** → chosen on-device first because it is faster, private, and works offline; only intents the device cannot satisfy escalate to the cloud gateway.

2. **Decision: Batch ASR vs streaming ASR** → chosen streaming because partial transcripts let the orchestrator start planning before the user finishes speaking, which cuts perceived latency far more than any model speedup.

3. **Decision: Monolithic model vs orchestrated domain skills** → chosen orchestrated skills because no single model is best at flights, music, and timers; the orchestrator routes to specialized executors that share one conversation context.

4. **Decision: Auto-execute tools vs confirm side effects** → chosen confirmation for side effects because auto-executing purchases or messages from misheard speech is dangerous; reads run free, writes ask first.

5. **Decision: One synchronous pipeline vs staged pipeline with per-stage deadlines** → chosen staged with deadlines because ASR, planning, and tool execution have different latency budgets; per-stage deadlines let the system degrade gracefully (skip personalization) instead of timing out wholesale.

End-to-end flows and API contracts — question 28

End-to-end request flows

JOURNEY 1 — "SET A TIMER FOR 10 MINUTES" (ON-DEVICE)

1. The mic stream runs through the on-device wake-word detector; nothing leaves until it matches.
2. Voice activity detection segments the utterance; on-device ASR transcribes it locally.
3. The intent classifier recognizes a timer intent with high confidence.
4. The local executor sets the timer against the device clock; no network request is made.
5. The response synthesizes on-device; the interaction completes in well under a second.
6. A privacy-safe log (intent type, latency, success) syncs for quality metrics; no audio.
7. Low confidence would have escalated to the cloud path instead of guessing.

JOURNEY 2 — "BOOK ME A FLIGHT TO CHICAGO FRIDAY" (CLOUD PLUS TOOLS)

1. Wake and VAD run on-device, but the classifier marks this intent cloud-requiring.
2. The device opens an encrypted streaming gRPC session, sending audio frames as they arrive.
3. The gateway authenticates, checks consent scopes, and enforces per-user budgets.
4. Streaming ASR yields a partial transcript; the LLM plans: resolve date, search flights.
5. The tool sandbox runs flight search with scoped credentials; results stream back progressively.
6. Booking, a sensitive action, requires explicit user confirmation before executing.
7. On confirmation the booking tool runs; the confirmation number returns on the same stream.

API contracts

gRPC `assistant.v1.Assistant/StreamAssist` Bidirectional streaming voice session

Auth: Device identity token (handshake)

```
→ → {config:{locale:"en-US",consent:["travel"]}} → {audio_chunk:"base64..."}
← ← {partial_transcript:"book me a flight..."} ← {tool_result:{...}}
```

POST `/v1/intents/classify` On-device intent classification (local)

Auth: None (on-device)

```
→ {"transcript":"set a timer for 10 minutes"}
← 200 {"intent":"create_timer","confidence":0.98,"route":"on_device"}
```

POST `/internal/v1/tools/{tool}/execute` Sandboxed tool execution (internal)

Auth: mTLS + scoped consent token · Content-Type: application/json

```
→ {"tool":"flight_search","args":{"to":"ORD","date":"2026-09-25"}}
← 200 {"flights":[{"airline":"UA","price":212.40}]}
```

GET `/v1/capabilities` On-device vs cloud intent routing

Auth: Device identity token

```
→ ?os_version=26.1
← 200 {"on_device":["timer","alarm"],"cloud":["flight_booking"],"model_version":"siri-26.1"}
```

How I would answer question 28

MODEL ANSWER - AS THE INTERVIEWEE

Q28. Design a Siri-style voice assistant request pipeline

I'll start by narrowing the problem before drawing architecture. For Design a Siri-style voice assistant request pipeline, I would ask: Simple commands, open-domain questions, or multi-step actions; What must work fully on-device; Wake-word, speech recognition, understanding, and response latency targets; Which actions require user confirmation? I also want to confirm the most important service-level objective and what we are intentionally leaving out. Asking these questions is not ceremony; it determines whether I optimize for latency, correctness, privacy, cost, or some combination. If those assumptions are acceptable, I'll proceed with a focused first version and identify where the design evolves.

For scale, I'll use round numbers and say them out loud: Assume 500M daily users, 20 requests/user/day (~116K average requests/s), 10× peak, and a 1–2 second perceived-response target. Streaming audio and model inference dominate latency and cost. I am not looking for false precision. I am identifying the dominant resource, the peak-to-average ratio, and whether the system is read-heavy, write-heavy, bandwidth-heavy, or compute-heavy. That tells me which path deserves the tightest latency budget and which state can be processed asynchronously.

At the whiteboard I would sketch this flow: Mic → on-device wake/VAD → local ASR/intent or encrypted stream Then Cloud gateway → ASR/LLM/router → tool sandbox → response Then └→ safety/evaluation telemetry. The end-to-end design is: Detect wake and voice activity on-device, handle common private intents locally, and stream only when cloud capability is needed. A gateway applies identity, consent, routing, and budgets; models produce a plan; tools execute in a permissioned sandbox; responses stream back while sensitive actions require confirmation. I would narrate each arrow and name the source of truth, partition key, idempotency boundary, and asynchronous boundary.

The first trade-off I would raise proactively is Partitioning: on-device processing improves latency/privacy and offline use; cloud models improve capability but add network, cost, and data risk. The second is Latency: speculative endpointing and streaming reduce perceived delay, but incorrect early actions must never escape confirmation gates. The third is Privacy and safety: minimize retention, separate evaluation samples, redact sensitive data, and make tool permissions explicit and revocable. I would choose a default based on the stated user outcome, but keep the alternative visible. At director level, I want the decision to be reversible: version the contract, isolate the risky component, canary changes, and define an exit criterion before scaling the investment.

For reliability, I would define service-level indicators for latency, availability, correctness, saturation, and cost per successful user action. I would add structured logs, metrics, traces, bounded retries with jitter, dead-letter handling where work is asynchronous, load shedding, and a tested rollback path. If the interviewer says, "The network disappears after the wake word—what still works?," I would revisit the bottleneck first, change one component at a time, and preserve correctness while degrading nonessential features. I would also assign clear on-call ownership and write runbooks for dependency and regional failures.

I can connect this to my background without overstating it: Lead with your GenAI, agentic-AI, LLM-orchestration, and MLOps portfolio. In my current role I lead a global organization of more than 80 engineers and a $5–7M portfolio, so I would sequence delivery as a thin vertical slice, prove the critical risk, then scale teams around durable service boundaries. I would close by saying: the proposed design keeps the primary path simple, moves expensive or failure-prone work off that path, makes the key trade-offs explicit, and links technical choices to user experience, operating burden, and business value. The next deep dive should be the highest-risk assumption we identified.

The repeatable practice loop

Do not memorize diagrams. Train a stable sequence until you can think clearly under interruption, then use the model answers as a diagnostic—not a script to recite.

Read only the question 01

Hide the guide. Restate the problem and identify the user outcome before seeing any hints.

Clarify for three minutes 02

Ask categories out loud. State assumptions only after the interviewer confirms or declines to specify.

Design for 45 minutes 03

Speak continuously while sketching. Use the five-step arc: clarify, estimate, design, deep-dive, stress and close.

Compare and score 04

Read the model answer only after finishing. Write down missing requirements, math, trade-offs, and failure modes.

Repair out loud 05

Re-answer only the weak portions immediately. Do not silently reread; retrieval and explanation are the training.

Revisit on schedule 06

Repeat the whole question in the next rotation. Move weak prompts forward for an extra mock.

> **Score each attempt from 0 to 2:** clarified before designing; quantified scale; explained the primary path; named at least three trade-offs; handled failure and operability; connected decisions to users and business; closed with a crisp summary. Target 11/14 before calling a prompt interview-ready.

Use the model answer correctly

Borrow its structure and transitions, not every sentence. Your answer should remain responsive to the interviewer’s constraints. If the prompt changes, change the design.

The clarifying questions playbook

Asking the right questions before designing is itself a scored behavior. It shows that you can convert ambiguity into an engineering contract.

Users and scale

"Who is the primary user?" "How many daily and concurrent users?" "Are there regional or device segments?"

Read versus write

"What is the read/write ratio?" "Which path is burstier?" "Are writes user-driven, streamed, or batch?"

Latency

"What is the p50 and p99 target?" "Which step determines perceived latency?" "Can results stream or degrade?"

Consistency versus availability

"Which data must be strongly consistent?" "What staleness is acceptable?" "During a partition, should we reject or serve stale?"

Data size and growth

"What is the typical and maximum object size?" "What is daily growth and retention?" "What access skew should I expect?"

Must-have versus nice-to-have

"Which journey must the first design support?" "What is explicitly out of scope?" "Which capability can arrive in phase two?"

> **Thirty-second opening:** "I'll first clarify users, scope, scale, and the non-functional priorities. Then I'll estimate the dominant loads, sketch the end-to-end design, deep-dive into the two hardest risks, and close with failure modes, trade-offs, and an evolution path. Does that approach work for you?"

The answer delivery formula

0:00–0:30	**Frame.** Say how you will approach the problem. This makes your structure visible and gives the interviewer an early chance to redirect.
0:30–5:00	**Clarify.** Ask, listen, and restate. State every assumption out loud; do not silently invent requirements.
5:00–10:00	**Quantify.** Estimate average and peak requests, payload, storage, bandwidth, and the dominant cost. Round numbers are fine; invisible math is not.
10:00–22:00	**Sketch and narrate.** Draw the primary read and write paths. Never go silent. Name state, interfaces, partition keys, and asynchronous boundaries.
22:00–38:00	**Deep-dive.** Choose the hardest two risks. Name trade-offs proactively before the interviewer asks and explain why one option fits the requirement.
38:00–44:00	**Operate.** Cover failure modes, observability, security, rollout, on-call burden, and cost without prompting.
44:00–45:00	**Close.** Give a 60-second summary of the design, its key trade-off, the primary risk, and the next evolution.

Close like a director: “The design protects the critical user path, uses asynchronous work to isolate expensive processing, and accepts bounded staleness where availability matters more. I would validate the biggest risk with a thin slice, measure p99 latency, correctness, and unit cost, then scale the organization around stable service boundaries.”

Handling follow-ups, curveballs, and the close

A follow-up is not evidence that your first answer failed. It is the interviewer testing whether your mental model is alive.

When they say “Now make it 10× larger”

Think out loud. Recalculate the dominant load, identify what saturates first, and change one component at a time. Add partitioning, hierarchy, caching, batching, or regionalization only where the new math requires it.

When they ask “What breaks first?”

Name a concrete queue, dependency, hot partition, cache miss path, or control-plane limit. Describe the signal that detects it, the user-visible consequence, and the containment mechanism.

When they ask “Cut cost in half”

Start with the unit-cost driver: egress, memory, GPU tokens, storage, external messages, or overprovisioned compute. Protect the service-level objective while testing compression, tiering, routing, utilization, or managed-versus-owned options.

When they challenge your choice

Do not defend reflexively. Restate the changed constraint, compare the alternatives against it, and revise the design if the evidence changes. Tie the reasoning to real experience when it genuinely fits: “In our AWS modernization, we protected the critical API path first and used measured latency and cost to sequence the migration.”

> **Director habit:** answer the technical curveball, then add staffing, rollout, and on-call consequences in one sentence. This shows you understand that architecture creates organizational load.

Questions to ask the interviewer

Use the final minutes to learn how the organization actually makes and operates systems. Choose two or three questions that fit the conversation; do not fire all eight mechanically.

1. What is the team's hardest scaling or reliability challenge right now?
2. Which architectural decision has changed most as the product matured?
3. Where does the team carry the greatest on-call burden, and what is being done about it?
4. How do you decide which platform capabilities to build versus buy?
5. What trade-off regularly creates healthy disagreement between product and engineering?
6. How are major designs reviewed, tested in production, and revisited after launch?
7. What would you want the person in this role to improve in the first six months?
8. How do senior leaders balance near-term delivery with multi-year architecture investments?

> **Listen for evidence:** ownership clarity, honest incident learning, investment in platform health, room to revisit decisions, and whether leadership treats reliability and developer productivity as product work.

Your one-page story bank

Use these as brief credibility bridges, not as a claim that you built the exact consumer product. Name the transferable principle, your verified result, and the boundary of the analogy.

1 YouTube	AWS migration; FinOps; event processing	**2 Maps**	Low-latency APIs; streams; global org
3 Typeahead	API latency; event overlay; OWASP	**4 URL shortener**	7-system onboarding; API speed; security
5 WhatsApp	AT&T identity; event platforms	**6 Rate limiter**	Partner APIs; zero trust
7 Distributed cache	Latency gains; FinOps; invalidation	**8 Notifications**	AT&T tokens; broker automation
9 Sponsored ads	$500M business; AI governance	**10 Web crawler**	Mainframe/cloud migration; sandboxing
11 AI chatbot	$5–7M AI portfolio; FinOps	**12 Enterprise RAG**	Document AI; 7-system integration
13 iCloud Photos	Replay/idempotency; mobile security	**14 Notes sync**	Message brokers; 99% automation
15 FaceTime	AT&T auth; low-latency APIs	**16 APNs**	Mobile tokens; queue isolation
17 Apple Pay	AT&T security; CEH; integration	**18 Offline E2EE chat**	Mobile identity; event-driven design
19 App Store	Marriott.com; DevSecOps; AI review	**20 Recently Deleted**	Event tombstones; lifecycle cost
21 Instagram	Event fan-out; CDN economics	**22 Netflix**	AWS/FinOps; consumer reliability
23 Uber	Streams; business trade-offs; org design	**24 Google Docs**	Logs/replay; integration contracts
25 Distributed cron	OpenShift automation; reliability	**26 API gateway**	API-first; OAuth/JWT; latency
27 iMessage	AT&T device trust; zero trust	**28 Voice assistant**	Agentic AI; MLOps; identity; FinOps

> **Bridge formula:** “I have not operated this exact product, but I faced the same architectural pressure in [real system]. We chose [decision], measured [verified result], and learned [transferable principle]. I would apply that principle here while partnering with domain specialists on the product-specific constraint.”

Four-week rotation: every question twice

Run two full passes through all 28 prompts. The first pass builds coverage; the second removes hesitation and strengthens weak decisions.

Week 1 — first pass, Q1–Q14

Mon–Sun: two questions per day in order. Score both. On Wednesday and Saturday, add a 15-minute repair drill on the lowest score so far.

Week 2 — first pass, Q15–Q28

Mon–Sun: two questions per day in order. End the week with one peer mock chosen randomly from all 28.

Week 3 — second pass, Q1–Q14

Mon–Sun: two per day. Change one requirement—10× scale, half the cost, regional failure, or stronger privacy—and adapt without restarting.

Week 4 — second pass, Q15–Q28

Mon–Sun: two per day. Require a director close: staffing, 90-day slice, on-call model, build versus buy, and three-year evolution. Add two mocks from your weakest quartile.

Mock interview protocol

The peer selects the prompt and likely follow-ups. The candidate gets no guide. The peer interrupts, changes a constraint, and scores structure—not just correctness. Feedback must name one strength, one missing decision, one unclear trade-off, and one moment where the candidate stopped driving.

Record and review

Record audio, then answer: Did I clarify first? Do scale math? Name three or more trade-offs? Cover failure modes and operability? Tie decisions to user impact? Close with a summary? Keep a score sheet and repeat anything below 11/14.

Director calibration and day-before checklist

Staffing and sequencing 1

Name durable team boundaries, dependencies, and the 90-day thin slice. Explain what can proceed in parallel and which capability is the critical path.

Cost and build-versus-buy 2

Identify the dominant unit cost. Build differentiating or risk-critical capabilities; buy commodities only after evaluating lock-in, portability, and on-call burden.

Operability 3

Define service-level objectives, error budgets, ownership, runbooks, safe deployment, rollback, and incident learning before declaring the design complete.

Three-year evolution 4

Describe the first release, the scale threshold that triggers the next architecture, and how contracts remain stable while implementations change.

Executive explanation 5

Translate one key trade-off into customer impact, risk, cost, and decision timing without losing technical truth.

Organizational realism 6

Call out skill gaps, platform leverage, security review, data governance, and who carries the pager.

Day-before checklist

- Rehearse the 30-second framing and 60-second close twice.
- Review the six clarification categories and your scale-math shortcuts.
- Choose three authentic stories: modernization, automation/AI, and security.
- Review your weakest two prompts only; do not cram all 28.
- Prepare two interviewer questions that fit the team.
- Set up paper or a whiteboard; practice drawing large, labeled boxes and arrows.
- Sleep. Clear reasoning and listening matter more than one extra memorized component.

> **Selection mindset:** You are not trying to guess the interviewer's architecture. You are demonstrating that you can turn ambiguity into a measurable system, make trade-offs visible, lead the conversation, and own the operational consequences.

Make the design operable by your organization

At Senior Engineering Manager or Director level, the architecture is only the opening bid. Your differentiator is showing how an 80+ engineer global organization would deliver, operate, govern, and evolve it within a $5–7M multi-year portfolio.

Shape the teams around durable boundaries 1

Propose 4–6 accountable domains—for example, client experience, core platform, data/AI, security, and reliability—each with a clear service contract. Cite your growth from 20 to 80+ engineers as evidence that you have built leadership capacity, not simply added headcount.

Sequence value before completeness 2

Describe a 90-day thin slice, a year-one scalable platform, and a three-year evolution path. Use your mainframe and monolith migrations to explain strangler patterns, parallel run, rollback, and measurable exit criteria.

Make operability part of the design 3

Define service-level objectives, error budgets, telemetry, incident ownership, and dependency runbooks before launch. Connect this to your API latency outcomes and automation that reduced manual work by 90–99%.

Treat cost as an architecture constraint 4

Identify the dominant unit cost—egress, memory, GPU tokens, storage, or vendor messages—and attach it to a product metric. Cite the 30% infrastructure-cost reduction that you reinvested into AI as your proof point.

Choose build versus buy deliberately 5

Build where capability differentiates the product or controls critical risk; buy managed primitives where operational burden does not create strategic advantage. Include exit costs, data portability, skill concentration, and vendor failure.

Explain one trade-off to an executive 6

“We can optimize for the lowest first-year cost or the fastest path to a resilient platform. I recommend the second, with a capped investment and explicit milestones, because it reduces operating risk while preserving a reversible path.”

> **Your three proof stories:** modernization (latency −30%, APIs +40%), automation (hours to minutes; 90–99% manual reduction), and disciplined economics (infrastructure cost −30%, reinvested into AI).

Sources and interpretation

1. DesignGurus, “Google System Design Interview: What Changed, What They Ask, and How to Pass” (2026).
2. IGotAnOffer, “Google System Design Interviews” (updated June 15, 2026).
3. Aced / Exponent, Apple Engineering Program Manager Interview Guide (2026).
4. Interview Show, Apple SDE interview experience (September 2026).
5. System Design Trainer, Top 20 System Design Interview Questions for 2026.

Apple and Google do not publish fixed question banks. This guide combines recently reported patterns with high-frequency design prompts and current AI/ML trends. Exact interviews vary by team, level, and domain.